L O G 4 3 0

Architecture logicielle — Été 2026

Phase 1 — Architecture basée par services (microservices)

Dossier d'architecture — gabarit Arc42 + vues 4+1 + ADR

Projet CanTelcoX — Système de support commercial (BSS) télécom

Équipe / N° d'équipe	*N/A*
Membres (nom, code permanent)	Harout Dabbaghian – DABH79330201
Cours / Groupe	LOG430 — Groupe 01
Session	Été 2026
Date de remise	8 juillet 2026, 23 h 59 (Moodle)
Version du document	*v1.0 — 2026-06-30*

École de technologie supérieure — Département de génie logiciel et des TI

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Sommaire

Mettez à jour le sommaire dans Word (clic droit → « Mettre à jour les champs ») une fois le document complété.

Sommaire 2
Comment utiliser ce gabarit 4
Correspondance Tâches de l'énoncé → Sections du document 4
1. Introduction et objectifs 6
1.1 Aperçu des exigences 6
1.2 Cas d'utilisation Must (MoSCoW) 7
1.3 Objectifs de qualité 21
1.4 Parties prenantes 22
2. Contraintes d'architecture 23
2.1 Contraintes techniques 23
2.2 Contraintes réglementaires & conformité 23
2.3 Contraintes organisationnelles & conventions 23
3. Contexte et périmètre 23
3.1 Contexte métier 25
3.2 Contexte technique 26
3.3 Cartographie du domaine (DDD) 27
4. Stratégie de solution 29
4.1 Style architectural & découpage en services 29
4.2 Communication inter-services 30
4.3 Stratégie de l'API REST 30
4.4 Référence industrielle TMF (optionnelle) 31
5. Vue des blocs de construction 32
5.1 Niveau 1 — Décomposition en services 32
5.2 Niveau 2 — Structure interne d'un service (hexagonal) 33
5.3 Modèle de domaine 35
5.4 Organisation du code (vue Développement) 37
6. Vue d'exécution 40
6.1 Scénario bout-en-bout (E2E) 40
6.2 Authentification & MFA (UC-02) 42
6.3 Idempotence & exactly-once à l'exécution 44
7. Vue de déploiement 46
7.1 Topologie de déploiement 46
7.2 API Gateway 47
7.3 Load balancing & tolérance aux pannes 48
8. Concepts transversaux 50

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

8.1 Persistance & intégrité 50
8.2 Idempotence (commandes & activations) 51
8.3 Exactly-once (facturation) 52
8.4 Journal d'audit append-only (CRTC / Loi 25) 52
8.5 Sécurité applicative 53
8.6 Contrôles anti-fraude 54
8.7 Gestion d'erreurs & versionnement 55
8.8 Observabilité (4 Golden Signals) 56
8.9 Caching 57
9. Décisions d'architecture (ADR) 58
ADR-001 — Style architectural & découpage en services 58
ADR-002 — Persistance & transactions (idempotence / exactly-once / append-only) 59
ADR-003 — Stratégie d'erreurs & versionnement d'API 60
10. Exigences de qualité 61
10.1 Arbre de qualité 61
10.2 Scénarios de qualité 62
10.3 Cibles NFR & résultats 63
10.4 Stratégie de tests & couverture 63
10.5 Résultats des campagnes de charge 66
10.6 Load balancing (N = 1..4) & tolérance aux pannes 69
10.7 Caching (on / off) 72
10.8 Appels directs vs via Gateway 73
11. Risques et dette technique 75
12. Glossaire 79
12.1 Langage métier 79
12.2 Acronymes 81
Annexe A — Contrats d'API & catalogue d'endpoints 82
Annexe B — Schéma de persistance 84
Annexe C — CI/CD, conteneurisation & exploitation 90
Annexe D — Traçabilité, Définition de Fini & auto-évaluation 90
D.1 Matrice de traçabilité (livrables & critères d'acceptation) 92
D.2 Définition de Fini (DoD) 94
D.3 Auto-évaluation (grille de l'énoncé) 94
Annexe E — Soutenance et démonstration (20 %) 96
E.1 Déroulé attendu 96
E.2 Grille de soutenance 96
E.3 Plan de soutenance (à préparer) 96

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Comment utiliser ce gabarit

Ce gabarit structure le dossier d'architecture de la Phase 1 selon le modèle Arc42 (sections 1 à 12) et y intègre les cinq vues 4+1. Chaque section rappelle, dans une boîte « Repères de l'énoncé », les tâches concernées, les livrables attendus et les critères d'acceptation (CA) à cocher. Remplacez chaque mention [À compléter — …] par votre contenu, insérez vos diagrammes là où indiqué, puis cochez les CA au fur et à mesure.

Conventions

- Texte en gris entre crochets = consigne à remplacer par votre contenu.
- Cases ☐ = critères d'acceptation de l'énoncé ; cochez-les lorsque la preuve est dans le dépôt ou le rapport.
- Insérez les diagrammes (PlantUML, Mermaid, draw.io…) sous forme d'images exportées ; nommez-les et numérotez les figures.
- Le dossier final est remis en PDF ; ce gabarit Word sert de source. Visez la reproductibilité < 30 min sur la VM.
- **Le gabarit est une structure minimale, pas un plafond :** ajoutez toute sous-section ou tout contenu pertinent demandé par l'énoncé. Les justifications de décisions (« pourquoi ce choix plutôt qu'un autre ») vont en priorité dans les ADR (§9) et la colonne « Justification (→ ADR) » du §4 ; les sections descriptives (§1–§8) présentent surtout l'architecture retenue.

Correspondance Tâches de l'énoncé → Sections du document

Tableau de navigation : où documenter chaque tâche du devoir. La matrice de traçabilité détaillée (avec statut) figure en Annexe D.

Tâche (énoncé)	Où la documenter
1.1 Cas d'utilisation Must (MoSCoW)	**§1 Introduction & objectifs · §10 Scénarios de qualité**
1.2 Cartographie DDD	**§3 Carte des contextes · §5 Modèle de domaine · §12 Glossaire**
2.1 Découpage microservices	**§4 Stratégie de solution · §5 Vue des blocs**
2.2 Couche API REST	**§4.4 · Annexe A (contrats OpenAPI/Postman)**
2.3 Documentation 4+1 + Arc42	Ensemble du document
2.4 ADR (≥ 3)	**§9 Décisions d'architecture**
3.1 Schéma de persistance	**§8.1 · Annexe B (ER/UML, migrations, seeds)**
3.2 Persistance & intégrité	**§8.1–8.4 · Annexe B**
4.1 Domaine & services applicatifs	**§5.3 Modèle de domaine**
4.2 Ports/adapters	**§3.2 Contexte technique · §5.2 Structure interne**
4.3 API REST publique (E2E)	**§6 Vue d'exécution · Annexe A**
4.4 Sécurité applicative	**§8.5 Sécurité · §8.6 Anti-fraude**
5.1 Observabilité	**§8.8 · §10.5 (dashboards)**
5.2 Tests de charge	**§10.5**

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Tâche (énoncé)	Où la documenter
5.3 Load balancing	**§7.3 · §10.6**
5.4 Caching	**§8.9 · §10.7**
6.1 API Gateway	**§7.2**
6.2 Direct vs Gateway	**§10.8**
7.1 Stratégie de tests	**§10.4**
7.2 Sécurité & gestion d’erreurs	**§8.5–8.7**
8.1–8.3 CI/CD & conteneurisation	**§7.1 · Annexe C**
9.1 Documentation finale	Ensemble · Annexe C (runbook)
9.2 Rapport comparatif	**§10.6–10.8**

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

1. Introduction et objectifs

Objectif Arc42 — *Présenter le système, ses objectifs métier essentiels, les objectifs de qualité prioritaires et les parties prenantes.*

Repères de l'énoncé

Tâches de l'énoncé : 1.1 Clarifier le périmètre & cas d'utilisation Must.

Vue 4+1 : Scénarios (cas d'utilisation).

Livrables attendus :

- UC textuels (scénario principal + alternatifs) pour ≥ 5 UC Must.
- Priorisation MoSCoW alignée sur les domaines du cahier.

Critères d'acceptation (à cocher)

☑ ≥ 5 UC Must (parmi UC-01 à UC-08) entièrement décrits et validés.

☑ Priorisation MoSCoW justifiée et matrice de traçabilité UC ↔ domaines complétée.

1.1 Aperçu des exigences

CanTelcoX, opérateur mobile canadien fictif, déploie un BSS moderne pour abonnés particuliers et PME couvrant le cycle de vie des lignes mobiles : souscription, activation, consultation d'usage, prise de commande de forfaits/options, paiement de facture et conformité réglementaire.

La mission du système est de fournir une plateforme BSS minimale mais complète permettant à CanTelcoX de gérer le cycle de vie initial d'un abonné mobile particulier ou PME. Le périmètre de la Phase 1 couvre les capacités indispensables à l'ouverture et à l'exploitation commerciale d'une ligne mobile: inscription du client, vérification d'identité, authentification sécurisée, activation de ligne, prise de commande de forfaits ou d'options, paiement de facture et traçabilité des opérations critiques. Les fonctions avancées, comme la détection automatisée de fraude, la consultation détaillée d'usage ou l'exécution complète du cycle mensuel de facturation, peuvent être simulées, limitées ou reportées si elles ne sont pas nécessaires à la démonstration du périmètre minimal viable.
La plateforme doit exposer ces capacités par une API REST sécurisée et documentée, orchestrée par une API Gateway, tout en s'intégrant à des infrastructures télécoms simulées telles que le réseau mobile, la portabilité, la passerelle de paiement et la médiation d'usage. L'architecture doit aussi soutenir les exigences de disponibilité, performance, sécurité, auditabilité et conformité réglementaire.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

1.2 Cas d’utilisation Must (MoSCoW)

Catalogue de référence des UC du cahier :

UC	Intitulé	Priorité MoSCoW
UC-01	Inscription & vérification d’identité	Must
UC-02	Authentification & MFA	Must
UC-03	Activation d’une ligne	Must
UC-04	Consultation usage / factures	Should
UC-05	Prise de commande	Must
UC-06	Paiement de facture	Must
UC-07	Détection de fraude	Could
UC-08	Cycle de facturation mensuel	Should

Justification de la priorisation MoSCoW

Les UC-01, UC-02, UC-03, UC-05 et UC-06 sont priorisés comme **Must**, car ils constituent le périmètre minimal viable du BSS CanTelcoX. Ils couvrent les étapes essentielles permettant de créer un client, sécuriser son accès, activer une ligne, commander une offre et encaisser un paiement. Ces cas d’utilisation portent aussi les principales exigences réglementaires et architecturales de la Phase 1: vérification d’identité, authentification forte, auditabilité des opérations, intégration avec les systèmes externes simulés, idempotence des commandes/activations et disponibilité des services critiques.

Les UC-04, UC-07 et UC-08 sont classés **Should** ou **Could** selon leur profondeur d’implémentation, car ils enrichissent l’exploitation du BSS mais ne bloquent pas la démonstration du cycle minimal de souscription, activation, commande et paiement. En particulier, le paiement de facture relève du UC-06 et reste Must, tandis que le cycle mensuel complet de facturation relève du UC-08 et peut être limité ou reporté en Phase 1.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Matrice de traçabilité UC ↔ domaines du cahier (bounded contexts)

Cochez (✓) le ou les domaines couverts par chaque UC, pour démontrer l'alignement avec le cahier. Ajustez selon les UC réellement retenus.

UC	Clients & Identité	Lignes & Services	Commandes & Activations	Catalogue & Offres	Usage / Facturation	Conformité & Audit
UC-01 Inscription & identité	✓					✓
UC-02 Authentification & MFA	✓					✓
UC-03 Activation d'une ligne		✓	✓	✓		✓
UC-04 Consultation usage / factures					✓	✓
UC-05 Prise de commande	✓	✓	✓	✓		✓
UC-06 Paiement de facture					✓	✓
UC-07 Détection de fraude	✓	✓	✓		✓	✓
UC-08 Cycle de facturation					✓	✓

Champ	Contenu
UC retenu	UC-01 — Inscription & vérification d'identité
Acteurs	Acteur principal: abonné particulier ou représentant PME. Acteurs secondaires : Service Clients & Identité, service de vérification d'identité simulé, service Conformité & Audit.
Préconditions	L'abonné n'a pas encore de compte actif avec la même adresse courriel. Les services Clients & Identité et Conformité & Audit sont disponibles.
Scénario principal	

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Champ	Contenu
	1. L'abonné soumet ses informations d'inscription : courriel, mot de passe, nom affiché, nom légal, adresse canadienne, type et numéro de document d'identité simulé, type de client. *2. Le système valide les champs obligatoires, le format du courriel, la longueur minimale du mot de passe, la province canadienne, le code postal et le document d'identité simulé.* *3. Le système vérifie qu'aucun client n'existe déjà avec la même adresse courriel.* *4. Le système effectue une vérification d'identité simulée synchrone pour la Phase 1.* *5. Le système hache le numéro de document d'identité et le mot de passe avant persistance.* *6. Le système crée le profil client avec* **status=ACTIVE**, **identityStatus=VERIFIED** et **riskLevel=LOW**. *7. Le système enregistre un événement d'audit* **CUSTOMER_REGISTERED** *avec* **TraceId**, *source, résultat, statut d'identité et niveau de risque.* *8. Le système retourne* **201 Created** *avec le profil public du client, sans exposer le numéro de document ni le mot de passe.*
Scénarios alternatifs / exceptions	**FA1 — Informations manquantes ou invalides:** le système détecte des champs obligatoires manquants ou invalides, refuse la création du client et retourne **400 validation_error**. Aucun client n'est créé. **FA2 — Courriel déjà utilisé:** le système détecte qu'un client existe déjà avec la même adresse courriel, refuse la demande et retourne **409 customer_exists**. Aucun doublon n'est créé. **FA3 — Service d'audit indisponible:** en exécution Docker Compose normale, le service dépend de l'audit en état **healthy**. Si l'appel audit échoue pendant la requête, l'erreur est traitée comme une indisponibilité transitoire et doit être journalisée côté service. **FA4 — KYC avancé ou risque élevé:** non implémenté en Phase 1. Une vérification KYC asynchrone, un statut **Pending** ou une révision manuelle sont documentés comme extensions futures.
Postconditions / règles métier	Un profil client est créé seulement si les données minimales sont valides et si l'adresse courriel est unique. En Phase 1, la vérification d'identité est simulée de manière synchrone et le client est créé directement avec **status=ACTIVE**, **identityStatus=VERIFIED** et **riskLevel=LOW.** Le mot de passe et le numéro de document d'identité ne sont jamais stockés en clair. Un événement d'audit append-only est produit avec le TraceId.

Ce diagramme montre le comportement réellement implémenté en Phase 1: validation des données, vérification d'unicité du courriel, vérification d'identité simulée synchrone, hachage des données sensibles, création d'un client ACTIVE/VERIFIED et journalisation append-only. OTP/MFA est traité séparément par UC-02.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

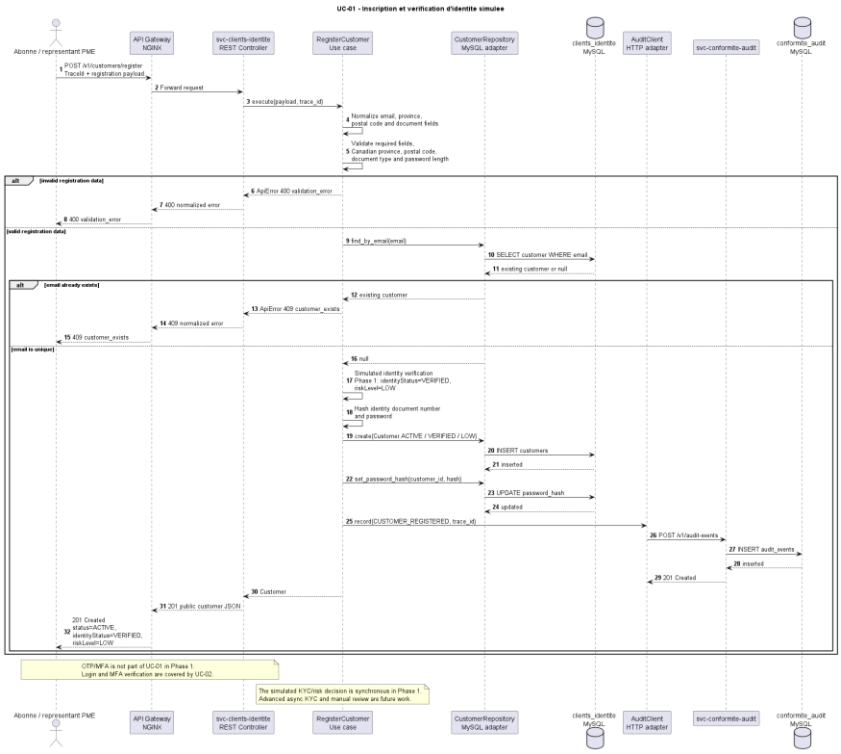


Figure 1.1 — Diagramme de séquence UC-01 Inscription et vérification d'identité.

Champ	Contenu
UC retenu	UC-02 — Authentification & MFA
Acteurs	Acteur principal: abonné ou représentant PME. Acteurs secondaires : Service Clients & Identité, fournisseur MFA simulé, service Conformité & Audit.
Préconditions	Le client possède un compte existant. Le compte est actif. Les services Clients & Identité et Conformité & Audit sont disponibles.
Scénario principal	1. L'utilisateur saisit son courriel et son mot de passe. 2. Le système valide le format des données d'authentification. 3. Le système vérifie que le client existe, que le compte est actif et que le mot de passe correspond au hachage stocké. 4. Le système évalue le risque de connexion selon les règles simplifiées de la Phase 1. 5. Le système crée un challenge MFA de type OTP simulé. En Phase 1, le code OTP de démonstration est **123456** et peut être retourné dans le champ **devOtp** pour faciliter les tests. 6. Le système retourne une réponse indiquant que le MFA est requis, avec le niveau **mfaStrength**, par exemple **STEP_UP** pour une connexion

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Champ	Contenu
	jugée plus sensible. 7. L'utilisateur soumet le code OTP associé au challenge MFA. 8. Le système valide le code OTP, vérifie que le challenge n'est pas expiré et qu'il n'a pas déjà été utilisé. 9. Après MFA réussi, le système crée une session de démonstration sous forme de jeton **dev-session-{challengeId}.** 10. Les événements de login, de challenge MFA et de validation MFA sont journalisés dans le service Conformité & Audit avec le **TraceId**. 11. Le système retourne le jeton de session à utiliser comme Bearer token sur les endpoints protégés.
Scénarios alternatifs / exceptions	**FA1 — Identifiants invalides:** si le courriel ou le mot de passe est invalide, le système refuse l'accès et retourne **401 invalid_credentials**. La tentative est auditée. **FA2 — Compte non actif:** si le client existe mais que son compte n'est pas actif, le système refuse l'accès et retourne **403 customer_not_active.** **FA3 — Trop de tentatives échouées:** si le nombre de tentatives échouées dépasse la limite prévue, le compte est temporairement bloqué et le système retourne **423 customer_temporarily_locked**. L'événement est journalisé. **FA4 — OTP invalide:** si le code OTP fourni ne correspond pas au challenge, le système refuse la validation MFA et retourne **401 mfa_invalid.** **FA5 — OTP expiré:** si le challenge MFA est expiré, le système retourne **422 mfa_expired.** **FA6 — Challenge MFA déjà utilisé:** si le même challenge MFA est rejoué après une validation réussie, le système retourne **409 mfa_already_used.** **FA7 — Connexion à risque élevé:** si le système détecte un contexte plus risqué, il retourne un niveau **mfaStrength=STEP_UP**. En Phase 1, cela reste un renforcement logique du scénario MFA simulé, sans fournisseur SMS, push ou WebAuthn réel.
Postconditions / règles métier	Une session est créée uniquement après authentification par mot de passe et validation MFA réussie. Le jeton retourné en Phase 1 est un jeton de démonstration **dev-session-{challengeId}** et non un JWT /OIDC de production. Les tentatives de connexion, la création du challenge MFA et la validation MFA sont journalisées avec le **TraceId**. Les challenges MFA expirés ou déjà utilisés ne peuvent pas créer de session.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Ce diagramme montre le comportement réellement implémenté : validation du courriel et du mot de passe, évaluation du risque de connexion, création d'un challenge MFA OTP simulé, validation du challenge, journalisation audit et création d'une session de démonstration dev-session-{challengeId}.

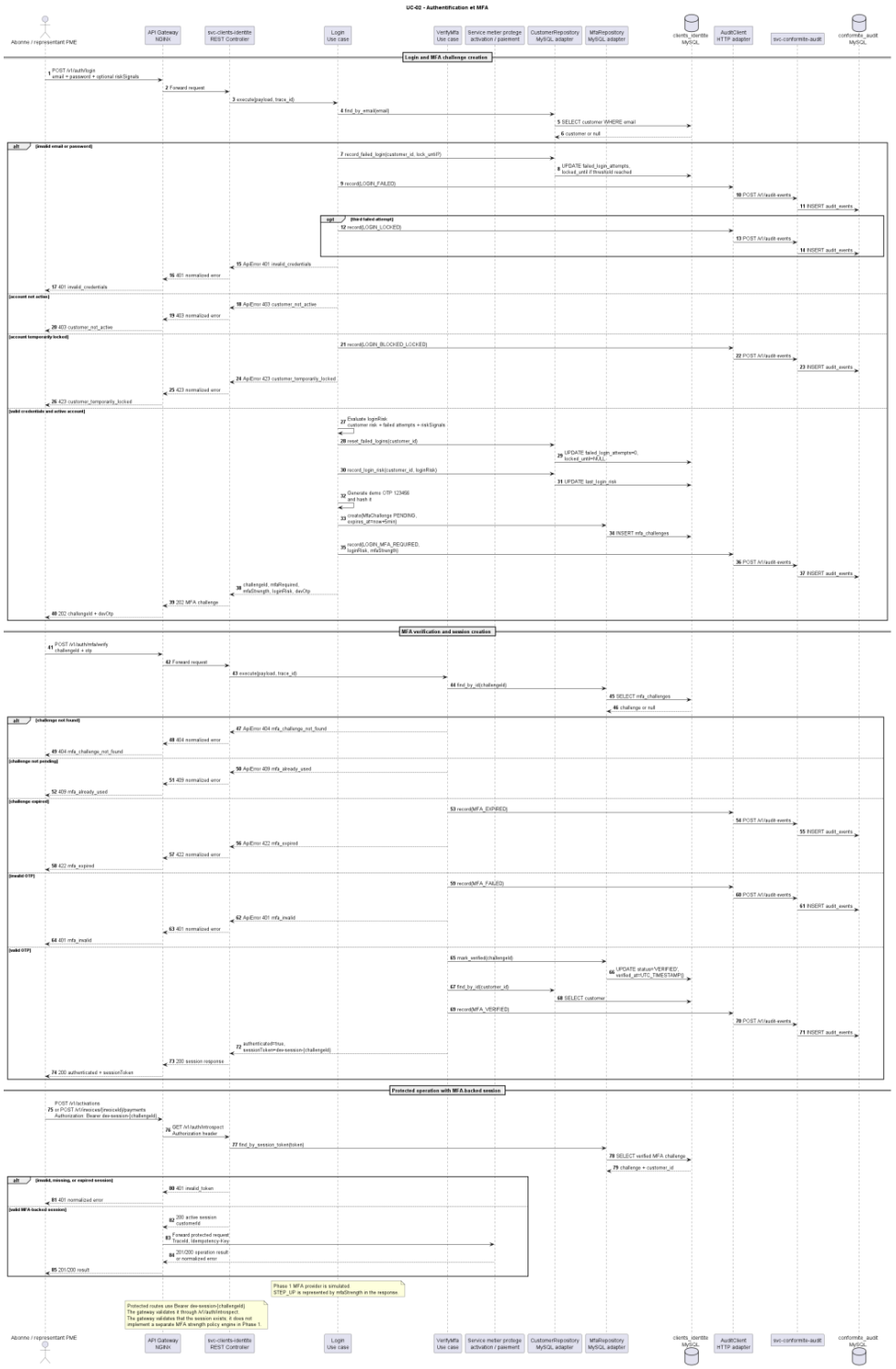


Figure 1.2 — Diagramme de séquence UC-02 Authentification et MFA.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Champ	Contenu
UC retenu	UC-03 — Activation d'une ligne
Acteurs	Acteur principal: abonné particulier, représentant PME ou agent back-office. Acteurs secondaires : Service Commandes & Activations, Service Clients & Identité, Service Catalogue & Offres, Service Lignes & Services, réseau simulé HLR/free5GC, service Conformité & Audit.
Préconditions	Le client existe, son compte est **ACTIVE** et son identité est **VERIFIED**. L'offre demandée existe et est active. La requête contient un **TraceId** et une **Idempotency-Key**. Les services nécessaires sont disponibles.
Scénario principal	1. Le client envoie **POST /v1/activations** avec **customerId**, **offerId**, **simType**, **numberMode** optionnel, **msisdn** optionnel, **TraceId** et **Idempotency-Key**. 2. Le service Commandes & Activations valide la clé d'idempotence et refuse les doublons conflictuels. 3. Le service vérifie que le client existe, que son compte est **ACTIVE** et que son identité est **VERIFIED**. 4. Le service bloque l'activation si le niveau de risque du client est **HIGH**. 5. Le service vérifie que l'offre existe et que son statut est **ACTIVE**. 6. Le service Commandes & Activations appelle le service Lignes & Services pour créer la ligne. 7. Si **numberMode=NEW**, le service Lignes & Services génère un MSISDN, un IMSI et un identifiant SIM/eSIM. 8. Si **numberMode=PORT_IN**, le service Lignes & Services accepte le MSISDN fourni et marque la portabilité simulée comme **ACCEPTED**. 9. Le service Lignes & Services crée la ligne en statut **PENDING**. 10. Le service Lignes & Services appelle l'adaptateur réseau HLR/free5GC pour provisionner la ligne. 11. Si le provisioning réussit, la ligne passe au statut **ACTIVE** avec **hlrStatus=PROVISIONED**. 12. Le service Commandes & Activations enregistre le résultat rejouable de l'activation. 13. Les événements importants sont journalisés dans le service Conformité & Audit avec le **TraceId**. 14. Le système retourne le résultat de l'activation au client.
Scénarios alternatifs / exceptions	**FA1 — Requête invalide ou clé d'idempotence manquante:** le système détecte une requête invalide ou une **Idempotency-Key** absente et retourne une erreur de validation. Aucune activation n'est créée. **FA2 — Requête répétée identique:** le système reçoit une demande

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Champ	Contenu
	déjà traitée avec la même clé d'idempotence et le même contenu. Il retourne le résultat déjà connu sans déclencher une nouvelle activation. **FA3 — Requête répétée conflictuelle:** le système reçoit la même clé d'idempotence avec un contenu différent. Il refuse la demande afin d'éviter une activation ambiguë. **FA4 — Client inexistant**, non actif ou identité non vérifiée : le système refuse l'activation si le client n'existe pas, si son compte n'est pas **ACTIVE** ou si son identité n'est pas **VERIFIED**. Le refus est journalisé. **FA5 — Risque client élevé:** si le niveau de risque du client est **HIGH**, l'activation est bloquée. L'événement est journalisé dans Conformité & Audit. **FA6 — Offre inexistante ou inactive**: si l'offre demandée n'existe pas ou n'est pas **ACTIVE**, l'activation est refusée. **FA7 — MSISDN déjà attribué:** si le MSISDN généré ou fourni est déjà associé à une ligne existante, la création de ligne est refusée. **FA8 — Type de SIM invalide:** si **simType** n'est pas accepté, la demande est refusée. En Phase 1, le système génère l'identifiant SIM/eSIM ; il ne gère pas un inventaire SIM complet. **FA9 — Échec du réseau simulé HLR/free5GC**: si l'adaptateur réseau retourne une erreur ou ne répond pas, la ligne passe au statut **FAILED** et l'activation retourne une erreur, par exemple **503**. Aucun mécanisme de reprise automatique n'est exposé en Phase 1.
Postconditions / règles métier	Une activation réussie crée une ligne mobile associée au client, à une offre, à un MSISDN, à un IMSI et à un identifiant SIM/eSIM. La ligne passe de **PENDING** à **ACTIVE** lorsque le provisioning réseau simulé réussit. Une même demande ne doit pas être exécutée deux fois grâce à **l'Idempotency-Key**. Les événements critiques sont journalisés avec le **TraceId**.

Ce diagramme montre le comportement réellement implémenté : réception d'une demande directe POST /v1/activations, validation de l'idempotence, vérification du client ACTIVE/VERIFIED, validation de l'offre active, blocage du risque HIGH, création de la ligne par svc-lignes-services, provisioning via l'adaptateur HLR/free5GC, passage de la ligne à ACTIVE et journalisation audit avec TraceId.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

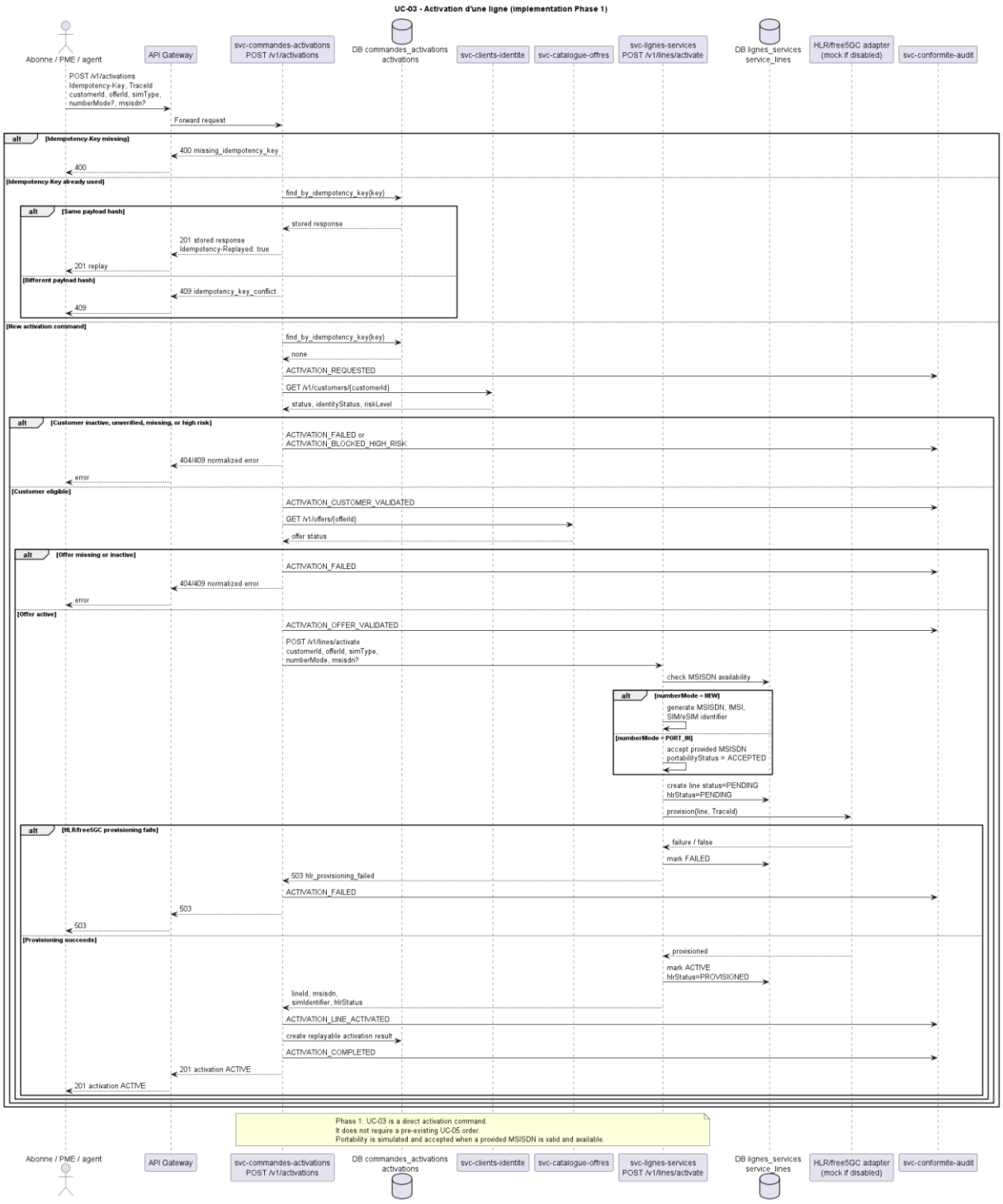


Figure 1.3 — Diagramme de séquence UC-03 Activation d'une ligne mobile.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Champ	Contenu
UC retenu	UC-05 — Prise de commande
Acteurs	Acteur principal: abonné particulier, représentant PME ou agent back-office. Acteurs secondaires: Service Catalogue & Offres, Service Commandes & Activations, Service Clients & Identité, service fraude/audit.
Préconditions	Le client est authentifié ou identifié. Le catalogue contient au moins une offre active. Le client est admissible à l'offre choisie.
Scénario principal	1. L'utilisateur consulte les offres disponibles. 2. L'utilisateur soumet **POST /v1/orders** avec **customerId**, **offerId**, **orderType**, **quantity**, **TraceId** et **Idempotency-Key**. 3. Le service Commandes & Activations valide la clé d'idempotence et rejoue la réponse si une commande identique existe déjà. 4. Le système vérifie que le client existe, que son compte est **ACTIVE** et que son identité est **VERIFIED**. 5. Le système vérifie le niveau de risque du client. 6. Le système vérifie que l'offre existe dans le catalogue et que son statut est **ACTIVE**. 7. Pour **NEW_PLAN**, le système calcule le montant total à partir du prix mensuel de l'offre et de la quantité demandée. 8. Le système crée la commande avec le statut **ACCEPTED**. 9. La commande reçoit **orchestrationStatus=READY_FOR_ACTIVATION**. 10. La confirmation est simulée avec **confirmationStatus=SIMULATED_SENT**. 11. Les événements de commande sont journalisés dans le service Conformité & Audit avec le **TraceId**. 12. Le système retourne la commande créée au client.
Scénarios alternatifs / exceptions	**FA1 — Requête invalide ou clé d'idempotence manquante:** le système détecte une requête invalide ou une **Idempotency-Key** absente et retourne une erreur de validation. Aucune commande n'est créée. **FA2 — Requête répétée identique:** le système reçoit une demande déjà traitée avec la même clé d'idempotence et le même contenu. Il retourne la commande déjà créée sans créer de doublon. **FA3 — Requête répétée conflictuelle:** le système reçoit la même clé d'idempotence avec un contenu différent. Il refuse la demande pour éviter une commande ambiguë. **FA4 — Client inexistant, non actif ou identité non vérifiée:** si le client n'existe pas, si son compte n'est pas **ACTIVE** ou si son identité n'est pas **VERIFIED**, la commande est refusée et l'événement est journalisé.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Champ	Contenu
	FA5 — Risque client élevé: si le niveau de risque du client est **HIGH**, la commande est bloquée. L'événement est journalisé dans Conformité & Audit. **FA6 — Offre inexistante ou inactive:** si l'offre demandée n'existe pas ou n'est pas **ACTIVE**, la commande est refusée. **FA7 — Type de commande non supporté:** les types **CHANGE_PLAN** et **ADD_OPTION** sont reconnus comme cas métier, mais leur exécution complète n'est pas implémentée en Phase 1. Le système retourne une erreur de compatibilité ou de fonctionnalité non supportée. **FA8 — Échec de confirmation:** la confirmation est simulée en Phase 1. Aucun fournisseur réel de notification courriel/SMS/push n'est appelé. Un échec réel de notification est donc hors périmètre Phase 1.
Postconditions / règles métier	Une commande valide référence un client, une offre, un type de commande, une quantité et un montant calculé. Une commande acceptée reçoit le statut **ACCEPTED** et **orchestrationStatus=READY_FOR_ACTIVATION.** Une même demande ne doit pas créer plusieurs commandes grâce à **l'Idempotency-Key**. Les événements critiques sont journalisés avec le **TraceId**.

Ce diagramme montre le comportement réellement implémenté : réception d'une demande POST /v1/orders, validation de l'idempotence, vérification du client ACTIVE/VERIFIED, contrôle du risque client, validation de l'offre active, calcul du montant pour NEW_PLAN, création d'une commande ACCEPTED avec orchestrationStatus=READY_FOR_ACTIVATION, confirmation simulée et journalisation audit avec TraceId.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

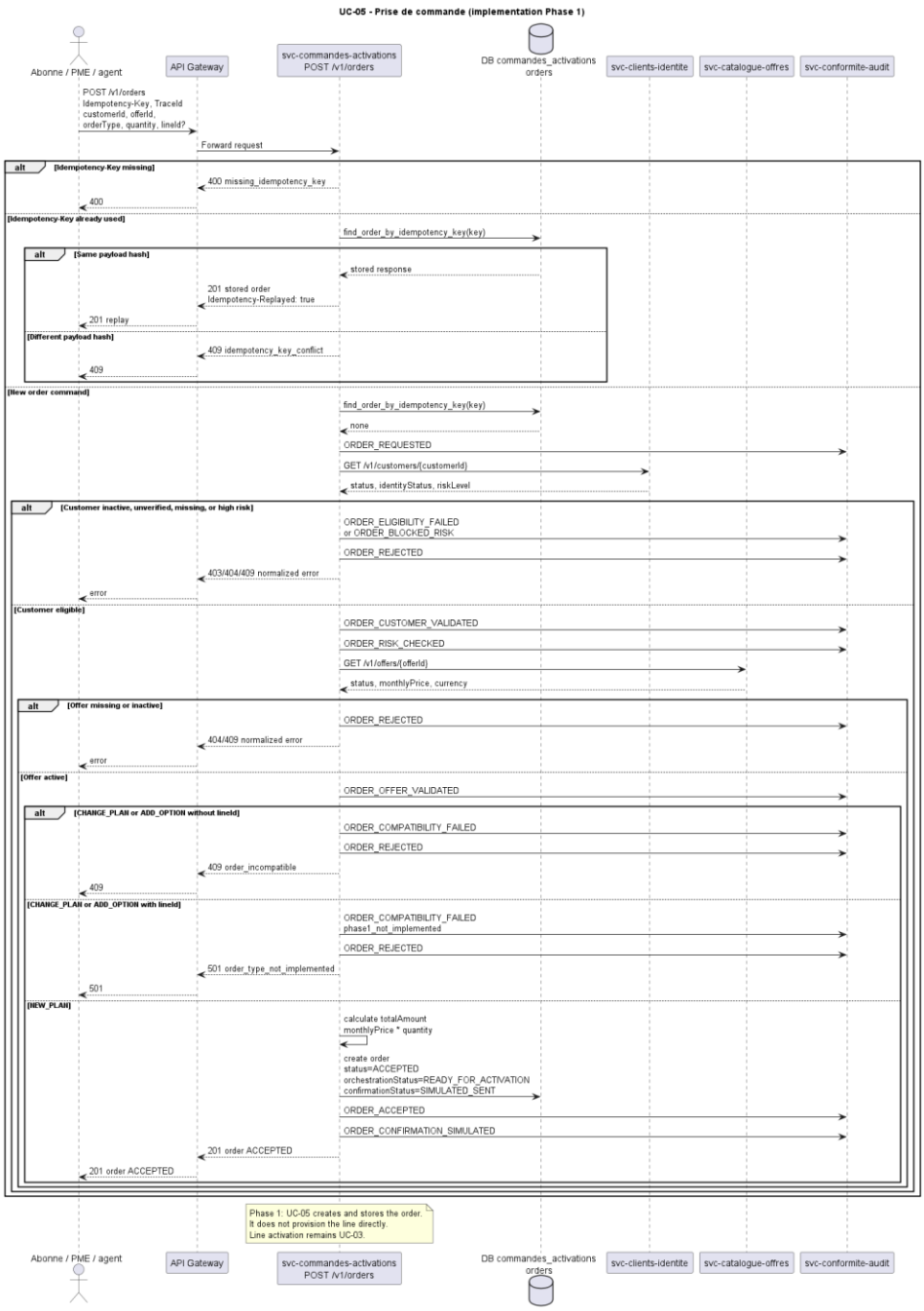


Figure 1.4 — Diagramme de séquence UC-05 Prise de commande / souscription à un forfait.

Champ	Contenu
UC retenu	UC-06 — Paiement de facture
Acteurs	Acteur principal: abonné particulier ou représentant PME.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Champ	Contenu
	Acteurs secondaires : Service Usage / Facturation, passerelle de paiement simulée, service Conformité & Audit.
Préconditions	Le client existe et peut être identifié. Une facture au statut **OPEN** existe pour ce client. La requête contient un **TraceId** et une **Idempotency-Key.** La passerelle de paiement simulée est disponible.
Scénario principal	1. L'utilisateur consulte ou sélectionne une facture existante. 2. L'utilisateur envoie **POST /v1/invoices/{invoice_id}/payments** avec customerId, **paymentMethod=SIMULATED_CARD**, **amount**, **currency=CAD**, **TraceId** et **Idempotency-Key**. 3. Le système valide la clé d'idempotence et rejoue le résultat si la même demande a déjà été traitée. 4. Le système vérifie que la facture existe. 5. Le système vérifie que la facture appartient au client indiqué. 6. Le système vérifie que la facture est au statut **OPEN**. 7. Le système vérifie que le montant et la devise correspondent exactement à la facture. 8. Le système journalise l'événement **PAYMENT_REQUESTED**. 9. Le système transmet la demande à la passerelle de paiement simulée. 10. Si la passerelle accepte, le système crée le paiement avec le statut **ACCEPTED**. 11. Le système marque la facture au statut **PAID**. 12. Le système journalise les événements **PAYMENT_ACCEPTED** et **INVOICE_PAID**. 13. Le système retourne la confirmation avec **paymentId**, **gatewayTransactionId** et **invoiceStatus=PAID**.
Scénarios alternatifs / exceptions	**FA1 — Requête invalide ou clé d'idempotence manquante:** le système détecte une requête invalide ou une **Idempotency-Key** absente et retourne une erreur de validation. Aucun paiement n'est créé. **FA2 — Requête répétée identique:** le système reçoit une demande déjà traitée avec la même clé d'idempotence et le même contenu. Il retourne le résultat déjà connu sans créer de nouvelle transaction. **FA3 — Requête répétée conflictuelle**: le système reçoit la même clé d'idempotence avec un contenu différent. Il refuse la demande afin d'éviter un paiement ambigu. **FA4 — Facture inexistante:** si la facture n'existe pas, le système refuse le paiement et retourne une erreur appropriée. Aucun paiement n'est créé. **FA5 — Facture appartenant à un autre client:** si la facture ne correspond pas au **customerId** fourni, le paiement est refusé. **FA6 — Facture déjà payée ou non ouverte**: si la facture n'est pas

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Champ	Contenu
	au statut **OPEN**, le système refuse le paiement afin d'éviter un double paiement. **FA7 — Montant ou devise invalide:** si le montant ne correspond pas exactement au montant dû, ou si la devise n'est pas **CAD**, le système refuse le paiement. **FA8 — Passerelle de paiement simulée indisponible:** si la passerelle simulée échoue ou ne répond pas, le système retourne **503 payment_gateway_unavailable**. Aucun statut **PENDING** ni mécanisme de réconciliation n'est implémenté en Phase 1.
Postconditions / règles métier	Un paiement accepté crée une écriture de paiement avec le statut **ACCEPTED** et marque la facture comme **PAID**. Une facture **OPEN** ne peut être payée qu'une seule fois avec succès. La même demande ne doit pas générer plusieurs transactions grâce à **l'Idempotency-Key**. Les événements critiques sont journalisés avec le **TraceId**.

Ce diagramme montre le comportement réellement implémenté : réception d'une demande POST /v1/invoices/{invoice_id}/payments, validation de l'idempotence, vérification de la facture OPEN, contrôle du client, du montant exact et de la devise CAD, appel à la passerelle de paiement simulée, création du paiement ACCEPTED, passage de la facture à PAID et journalisation audit avec TraceId.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

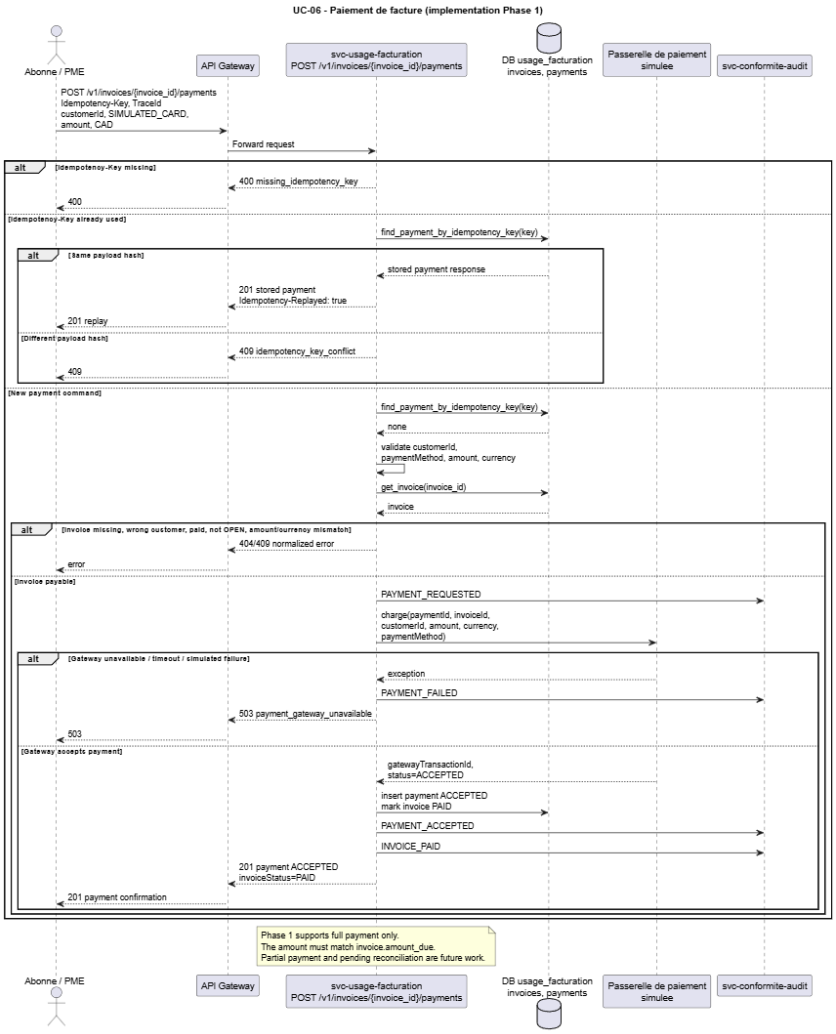


Figure 1.5 — Diagramme de séquence UC-06 Paiement de facture mobile.

1.3 Objectifs de qualité

Les trois objectifs de qualité prioritaires (cibles NFR du cahier) :

Objectif de qualité	Scénario / motivation	Cible mesurable
Performance	Réactivité des parcours critiques Phase 1 via API Gateway: MFA, commande, activation, paiement	Latence P95 ≤ 500 ms
Débit / capacité	Soutenir un trafic de démonstration multi-services sans erreurs	≥ 600 opérations/s
Disponibilité	Continuité de service en cas de panne d'instance répliquée derrière Gateway	Disponibilité 95 %
Sécurité & auditabilité	Protéger les opérations sensibles: inscription, MFA, activation, commande,	Authentification + MFA avant session;

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Objectif de qualité	Scénario / motivation	Cible mesurable
	paiement et accès aux données personnelles	Idempotency-Key sur commandes sensibles; secrets via variables d'environnement; audit append-only des événements critiques

1.4 Parties prenantes

Partie prenante	Rôle / attente	Section concernée
Abonnés particuliers / PME	Souscrire, s'authentifier, commander un forfait, activer une ligne, consulter une facture et effectuer un paiement	**§1, §6, Annexe A**
Opérateur CanTelcoX / exploitation	Assurer la disponibilité, la supervision, la traçabilité et l'exploitation quotidienne de la plateforme.	**§7, §8, §10, Annexe C**
Régulateur (CRTC, Loi 25)	Vérifier la traçabilité, la protection des données personnelles et l'auditabilité des opérations sensibles.	**§2, §7, §9, Annexe B**
Équipe de développement	Concevoir, implémenter, tester et maintenir les services, les API REST, les contrats et les tests automatisés.	**§4, §5, §7, Annexes A-B**
Équipe DevOps / SRE	Déployer la stack Docker Compose, maintenir la CI/CD, gérer les healthchecks, le load balancing, l'observabilité et le rollback.	**§7, §8, §10, Annexe C**
Systèmes externes simulés	Fournir les intégrations simulées: HLR/free5GC, passerelle de paiement, portabilité simplifiée et données d'usage/facturation de démonstration.	**§3, §4, §6**
Équipe sécurité / conformité	Définir les contrôles MFA, idempotence, anti-fraude simple, protection des données sensibles et audit append-only.	**§2, §7, §9, ADR-002**

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

2. Contraintes d’architecture

Objectif Arc42 — *Recenser les contraintes techniques, réglementaires et organisationnelles qui limitent les choix d’architecture.*

2.1 Contraintes techniques

Contrainte	Description / impact
Déploiement	Environnement Docker propre de type VM; conteneurisation Docker; lancement reproductible avec docker compose up; déploiement scripté en une commande.
Communication (Phase 1)	Communication synchrone REST entre services via API Gateway; pas de bus événementiel lourd en Phase 1.
Documentation	Documentation OpenAPI/Swagger, modèle Arc42, vues 4+1, diagrammes PlantUML et décisions d’architecture documentées.
Observabilité	Prometheus + Grafana pour métriques et tableaux de bord; logs structurés; healthchecks /health; suivi des 4 Golden Signals.
Pile technologique	Python/Flask pour les microservices REST; MySQL comme base relationnelle; Redis pour le cache catalogue; NGINX comme reverse proxy/load balancer; Prometheus + Grafana pour l’observabilité; Locust pour les tests de charge; Docker Compose pour l’orchestration; GitHub Actions pour la CI/CD.

2.2 Contraintes réglementaires & conformité

Le cadre télécom canadien impose des contraintes structurantes (à tracer vers les ADR §9) :

Cadre	Exigence dérivée
CRTC / Loi sur les télécommunications	Traçabilité et auditabilité des opérations sensibles: inscription, authentification, commande, activation, paiement et consultation des données.
Loi 25 / LPRPDE	Protection des renseignements personnels; minimisation des données exposées; secrets hors dépôt; journalisation sans fuite de données sensibles.
Intégrité transactionnelle	**Idempotency-Key** obligatoire pour commandes, activations et paiements; contraintes d’unicité en base; paiement d’une facture protégé contre les doublons; journal d’audit append-only.
Lutte à la fraude	MFA avant session; blocage des clients à risque élevé; signaux de risque MFA simulés **(newDevice, roaming, unusualLocation, simSwapRecen**t); fraude avancée conservée comme extension future.

2.3 Contraintes organisationnelles & conventions

[À compléter — conventions d’équipe : versionnement Git, normes de code, gestion des secrets via variables d’environnement, langue de la doc, etc.]

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Contrainte	Description / impact
Versionnement Git	Le code source, les fichiers Docker, les contrats API, la documentation et les scripts d'exploitation sont versionnés dans un dépôt Git commun. Les changements importants sont regroupés en commits cohérents et traçables.
Organisation du dépôt	Le dépôt sépare les services (services/), l'infrastructure Docker, les workflows CI/CD, les contrats OpenAPI/Postman, les ADR, les diagrammes et les runbooks.
Normes de code	Le code suit des conventions communes de structure, de nommage, de validation d'entrée, de gestion d'erreurs normalisées et de tests automatisés.
Gestion des secrets	Aucun secret réel ne doit être commité. Les paramètres sensibles passent par variables d'environnement ou .env local exclu du versionnement; .env.example documente les variables attendues.
Documentation	La documentation d'architecture est rédigée en français selon Arc42, vues 4+1, ADR, annexes API/persistance/CI-CD et diagrammes PlantUML. Les API sont documentées avec OpenAPI/Swagger et Postman.
Reproductibilité	La stack doit être reproductible dans un environnement Docker propre avec docker compose up; le déploiement VM est scripté avec rollback simple.
Usage de l'IA générative	L'IA générative peut assister le setup, le débogage, la compréhension et la rédaction, mais l'équipe reste responsable de la validation, de l'intégration et de la soutenance du contenu final.

Note SIAG (encadrement labos): l'usage des systèmes d'IA générative est encadré — assistance au setup, au débogage et à la complétion permise, mais la production directe des livrables évalués peut faire l'objet d'une vérification orale ou écrite. Documentez votre usage.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

3. Contexte et périmètre

Objectif Arc42 — *Délimiter le système vis-à-vis de ses partenaires (utilisateurs et systèmes externes) — contextes métier et technique — et cartographier le domaine (DDD).*

Repères de l'énoncé

Tâches de l'énoncé : 1.2 Cartographier le domaine (DDD) ; 4.2 adapters externes simulés.

Vue 4+1 : Contexte (alimente Logique et Déploiement).

Livrables attendus :

- Diagramme de contexte (système + acteurs + systèmes externes).
- Bounded contexts et carte des contextes (DDD).
- Ubiquitous language (reporté au glossaire §12).

Critères d'acceptation (à cocher)

☑ Frontières de service tracées à partir des bounded contexts.

3.1 Contexte métier

Ce diagramme présente CanTelcoX comme une boîte noire représentant la plateforme commerciale BSS. Il montre les principaux acteurs externes : abonnés particuliers/PME, back-office, équipe sécurité/conformité, régulateur/auditeur et équipe exploitation DevOps-SRE. Il délimite aussi les dépendances externes simulées de la Phase 1 : réseau mobile HLR/HSS/UDM, hub de portabilité MSISDN, passerelle de paiement, médiation d'usage, service OTP/MFA et service KYC/scoring. L'observabilité Prometheus/Grafana est représentée comme une dépendance d'exploitation pour suivre les métriques, journaux et états de service.

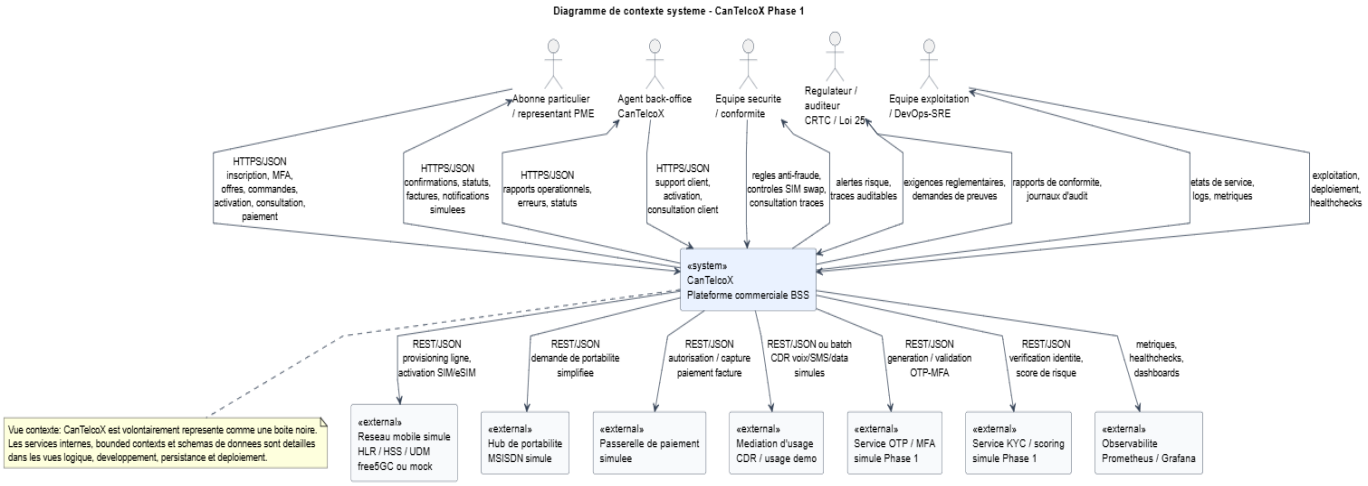


Figure 2.1 — Diagramme de contexte système de CanTelcoX Phase 1

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

3.2 Contexte technique

Systèmes externes simulés à intégrer via adapters (ports/adapters) :

Système externe simulé	Rôle	Protocole / adapter
Réseau mobile HLR/HSS/UDM, free5GC ou mock	Activation et état des lignes mobiles ; provisioning de la ligne ; association SIM/eSIM, IMSI/SUPI et profil abonné.	Adapter REST simulé **MobileNetworkAdapter**; appels HTTP/JSON vers les endpoints de provisioning, activation SIM/eSIM et consultation d'état.
Hub de portabilité MSISDN simulé	Portabilité d'un numéro mobile existant ; validation ou refus d'une demande de portabilité MSISDN.	Adapter REST simulé **PortabilityAdapter** ; appels HTTP/JSON pour créer une demande de portabilité et consulter son statut.
Passerelle de paiement simulée	Paiement de facture mobile dans UC-06; autorisation, confirmation ou échec de paiement.	Adapter REST simulé **PaymentGatewayAdapter**; appels HTTP/JSON pour autoriser/capturer un paiement et consulter le statut de transaction.
Médiation d'usage / CDR simulés	Fournir les usages voix/SMS/data nécessaires à la consultation d'usage et au cycle de facturation.	Adapter REST ou batch simulé **UsageMediationAdapter**; récupération HTTP/JSON ou import périodique de CDR simulés.
Service KYC / scoring simulé	Vérification d'identité client, statut KYC et score de risque pour l'inscription et les contrôles anti-fraude.	Adapter REST simulé **KycScoringAdapter**; appels HTTP/JSON pour vérifier l'identité et obtenir un score de risque.
Service OTP / MFA simulé	Génération et validation des codes OTP/MFA pour l'authentification et les opérations sensibles. En Phase 1, l'OTP est simulé.	Adapter REST simulé **MfaAdapter**; appels HTTP/JSON pour générer, valider et expirer un facteur MFA.
Observabilité Prometheus / Grafana	Collecte des métriques, healthchecks, tableaux de bord et support aux 4 Golden Signals : latence, trafic, erreurs et saturation.	Exposition HTTP **/metrics** et endpoints de santé ; scraping Prometheus et visualisation Grafana.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

En Phase 1, certains systèmes externes sont documentés comme cibles d'adapters, mais leur implémentation est simplifiée : KYC/scoring et OTP/MFA sont simulés dans svc-clients-identite, la portabilité est simulée dans svc-lignes-services, et la médiation CDR reste limitée ou future.

3.3 Cartographie du domaine (DDD)

Bounded contexts du domaine télécom (frontières de service candidates) :

Bounded context	Responsabilité	Service candidat
Clients & Identité	Profils abonnés, vérification d'identité simulée, authentification et MFA	svc-clients-identite
Lignes & Services	Lignes mobiles, MSISDN, SIM/eSIM, activation et état réseau	svc-lignes-services
Commandes & Activations	Prise de commande, préparation à l'activation et idempotence	svc-commandes-activations
Catalogue & Offres	Forfaits, options, prix et règles d'éligibilité	svc-catalogue-offres
Usage / Rating / Facturation	Factures, paiements, données d'usage de démonstration et garanties exactly-once pratiques	svc-usage-facturation
Conformité & Audit	Journal append-only, traces réglementaires et contrôles anti-fraude	svc-conformite-audit

Carte des contextes (relations: upstream/downstream, ACL, conformist…):

La figure suivante présente la carte des contextes DDD. Les rectangles bleus représentent les bounded contexts internes, le rectangle jaune représente le contexte transversal de conformité/audit, et les rectangles gris représentent les systèmes externes simulés accessibles via ACL/adapters.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

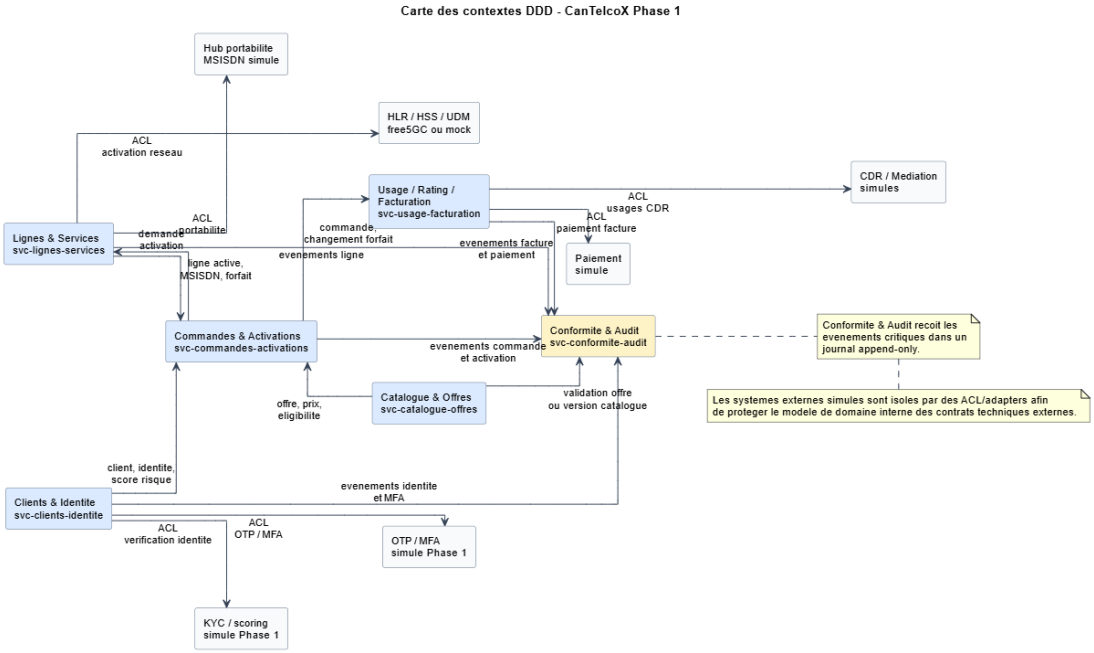


Figure 3.2 — Carte des contextes DDD de CanTelcoX Phase 1

Clients & Identité et Catalogue & Offres sont en amont de Commandes & Activations : une commande a besoin d'un client actif, d'une identité vérifiée, d'un score de risque acceptable et d'une offre active. Commandes & Activations accepte les commandes, applique l'idempotence et prépare l'activation. L'activation effective de la ligne est ensuite prise en charge par Lignes & Services, qui gère les MSISDN, les SIM/eSIM, l'IMSI/SUPI et le provisionnement réseau simulé.

Usage / Rating / Facturation gère les factures et les paiements. En Phase 1, l'usage détaillé, le rating complet et le cycle mensuel automatisé sont limités ou simulés, tandis que le paiement complet d'une facture ouverte est couvert par UC-06.

Conformité & Audit est un contexte transversal. Il reçoit les événements critiques des autres contextes, notamment l'inscription, le MFA, les commandes, les activations, la validation des offres, les factures et les paiements. Ces événements alimentent un journal append-only utilisé pour l'audit, la conformité CRTC/Loi 25 et la traçabilité des opérations sensibles.

Les systèmes externes simulés sont intégrés par des ACL/adapters afin de protéger le modèle de domaine interne. Clients & Identité utilise des adapters vers KYC/scoring et OTP/MFA. Lignes & Services utilise des adapters vers HLR/HSS/UDM/free5GC et le hub de portabilité simulé. Usage / Rating / Facturation utilise des adapters vers la médiation CDR simulée et la passerelle de paiement simulée.

Les blocs DDD à matérialiser dans le code sont le langage ubiquitaire, les Value Objects, les Aggregates et les Repositories. Les mêmes noms métier doivent rester cohérents entre la documentation, les diagrammes, le code et les contrats d'API.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

4. Stratégie de solution

Objectif Arc42 — *Résumer les décisions fondamentales : style architectural, découpage, technologies, atteinte des objectifs de qualité.*

Repères de l'énoncé

Tâches de l'énoncé : 2.1 Concevoir l'architecture microservices ; 2.2 Concevoir la couche API REST.

Vue 4+1 : Transversale (préfigure §5–§7).

Livrables attendus :

- Choix de découpage en services (≥ 2) et style par service (Hexagonal/MVC).
- Stratégie de communication inter-services (sync REST en Phase 1).
- Routes versionnées /v1, codes HTTP normalisés, format d'erreur JSON unifié, contrats OpenAPI.

Critères d'acceptation (à cocher)

☑ Dépendances dirigées (pas de cycles), couplage contrôlé aux frameworks.

☑ Bounded contexts mappés sur les services.

☑ Swagger publié, collection Postman fonctionnelle, séparation domaine ↔ infra.

4.1 Style architectural & découpage en services

Décision	Choix retenu	Justification (→ ADR)
Style global	Architecture basée par services (microservices)	Ce style permet d'aligner l'architecture sur les bounded contexts DDD, de séparer les responsabilités métier et de déployer les services de façon indépendante. Il répond aux besoins de modularité, d'évolutivité, d'observabilité et de tolérance aux pannes attendus en Phase 1. Voir ADR-001.
Découpage (BC → service)	**svc-clients-identite, svc-lignes-services, svc-commandes-activations, svc-catalogue-offres, svc-usage-facturation, svc-conformite-audit**	Chaque service correspond à un bounded context du domaine CanTelcoX. Ce découpage limite le couplage, clarifie la propriété des données et facilite la traçabilité entre les exigences métier, les API et le code. Voir ADR-001.
Style par service	Architecture hexagonale, ports/adapters, avec contrôleurs HTTP de type MVC	L'architecture hexagonale sépare le domaine métier de l'infrastructure : API REST, bases de données, adapters externes, cache et observabilité. Les contrôleurs HTTP servent de couche d'entrée, mais la logique

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Décision	Choix retenu	Justification (→ ADR)
		métier reste dans les couches application et domaine. Voir ADR-001.

4.2 Communication inter-services

En Phase 1, la communication inter-services est principalement synchrone par REST/HTTP en JSON. Les services exposent des routes versionnées sous **/v1**, documentées dans le contrat OpenAPI central du projet. Cette approche est retenue pour garder l'architecture simple, testable et compatible avec les contraintes de démonstration sur VM.

Les dépendances entre services sont dirigées et suivent la carte des contextes DDD. **svc-commandes-activations** consomme les informations de **svc-clients-identite** et de **svc-catalogue-offres**, puis délègue l'activation de ligne à **svc-lignes-services**. **svc-usage-facturation** valide les clients, gère les factures et les paiements, et utilise une passerelle de paiement simulée. **svc-conformite-audit** reçoit les événements critiques des autres services pour alimenter le journal append-only.

Les cycles de dépendance sont évités : un service consommateur appelle un service fournisseur, mais le fournisseur ne rappelle pas directement le consommateur dans le même flux. Les événements d'audit sont envoyés vers **svc-conformite-audit**, qui agit comme récepteur transversal sans dépendance retour vers les services métier.

Les systèmes externes et simulateurs sont isolés par des adapters lorsque le code Phase 1 les implémente. **svc-lignes-services** utilise un adapter HLR/free5GC ou mock pour le provisionnement réseau. **svc-usage-facturation** utilise une passerelle de paiement simulée. La portabilité, le KYC/scoring, l'OTP/MFA et la médiation CDR sont simplifiés ou simulés en Phase 1, avec des adapters externes complets conservés comme extensions futures.

4.3 Stratégie de l'API REST

Aspect	Convention retenue
Versionnement	Les routes exposées par la Gateway et les services utilisent le préfixe **/v1/....** Les changements incompatibles seront publiés dans une version majeure ultérieure, par exemple **/v2/....**
Codes HTTP	**200 OK** pour une lecture ou une action réussie ; **201 Created** pour une création ; **400 Bad Request** pour une requête mal formée ; **401 Unauthorized** si l'utilisateur n'est pas authentifié ; **403 Forbidden** si l'accès est interdit ; **404 Not Found** si la ressource n'existe pas **; 405 Method Not Allowed** pour une méthode non supportée ; **409 Conflict** pour un conflit d'état ou d'idempotence ; **422 Unprocessable** Entity pour une erreur métier ; **423 Locked** pour un compte temporairement verrouillé ; **500 Internal Server Error** pour une erreur serveur ; **503 Service Unavailable** pour une dépendance temporairement indisponible. **202 Accepted** et **204 No Content** restent des conventions prévues si des opérations asynchrones ou sans corps sont ajoutées.
Format d'erreur	

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Aspect	Convention retenue
	JSON unifié contenant **code**, **message**, **details**, **traceId** et **timestamp**. Le champ traceId permet de corréler l'erreur avec les logs et les traces d'observabilité.
Documentation	Un contrat OpenAPI central est fourni dans **docs/api/openapi.yaml**. Une collection Postman est fournie en Annexe A pour tester les cas d'utilisation Must via les routes **/v1**.

4.4 Référence industrielle TMF (optionnelle)

Les TMF Open APIs peuvent inspirer la conception. Mapping indicatif :

TMF Open API	Domaine	Service interne
TMF620 Product Catalog Management	Catalogue & Offres	svc-catalogue-offres
TMF622 Product Ordering Management	Commandes & Activations	svc-commandes-activations
TMF629 Customer Management	Clients & Identité	*svc-clients-identite*
TMF637 Product Inventory Management / TMF638 Service Inventory Management	Lignes & Services	svc-lignes-services
TMF678 Customer Bill Management	Factures	svc-usage-facturation
TMF676 Payment Management	Paiements	svc-usage-facturation

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

5. Vue des blocs de construction

Objectif Arc42 — *Décomposition statique du système : niveau 1 (services), niveau 2 (structure interne d'un service), modèle de domaine. Couvre les vues Logique et Développement de 4+1.*

Repères de l'énoncé

Tâches de l'énoncé : 1.2 modèle de domaine ; 2.1 couches & dépendances ; 4.1 domaine ; 4.2 ports/adapters.

Vue 4+1 : Logique + Développement.

Livrables attendus :

- Décomposition en services et couches (hexagonal : domaine / application / infrastructure).
- Modèle de domaine par service (entités, agrégats, value objects).
- Organisation du code (structure des dépôts / modules).

Critères d'acceptation (à cocher)

☑ Agrégats clés identifiés (Profils abonnés, Lignes/MSISDN, Commandes, Forfaits, Factures).

☑ Séparation nette domaine ↔ infra ; pas de logique métier dans les controllers.

5.1 Niveau 1 — Décomposition en services

La figure suivante présente la décomposition « whitebox » du système CanTelcoX en services métier, avec l'API Gateway, les dépendances dirigées, les bases de données détenues par service et les principaux simulateurs externes.

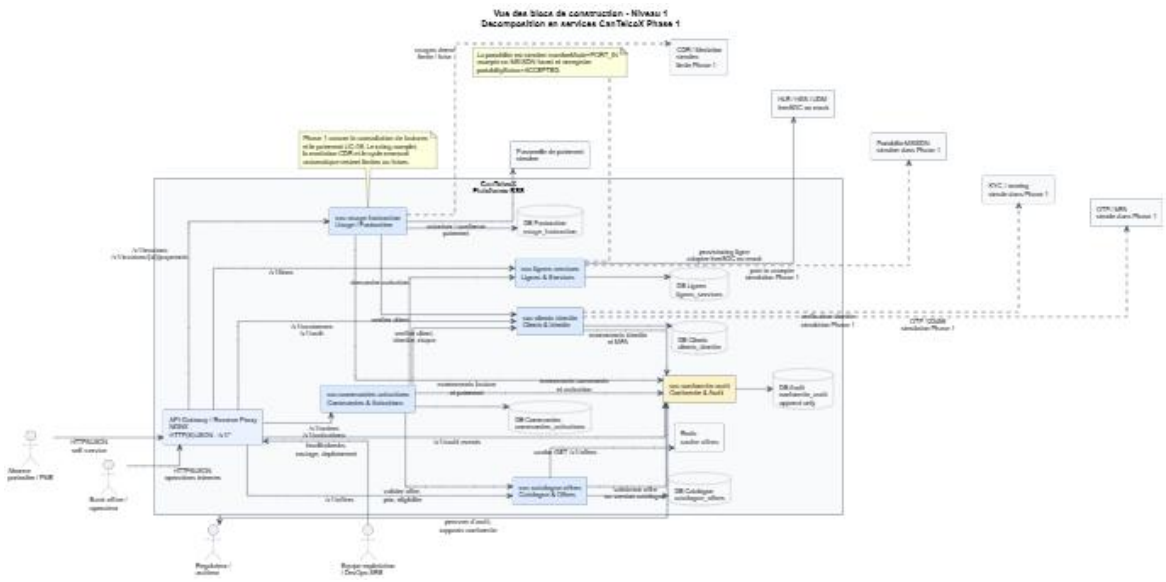


Figure 5.1 — Vue des blocs de construction niveau 1 de CanTelcoX Phase 1

CanTelcoX est exposé à travers une API Gateway / reverse proxy NGINX. Les acteurs externes, notamment l'abonné particulier/PME, le back-office et l'équipe d'exploitation, accèdent au système via des API HTTP(S)/JSON versionnées sous **/v1/*.** L'API Gateway centralise l'entrée du trafic, propage les en-têtes techniques comme **TraceId**, **Authorization** et **Idempotency-Key**, applique le routage vers les services appropriés et sert de point d'entrée unique. En Phase 1, la validation fine du jeton est principalement déléguée aux services applicatifs.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Le service **svc-clients-identite** est responsable de la gestion des clients, de l'identité, de l'authentification et du MFA. Il persiste ses données dans **DB Clients (clients_identite)** et publie les événements liés à l'identité, à l'inscription et au MFA vers **svc-conformite-audit**. En Phase 1, le KYC/scoring et l'OTP/MFA sont simulés dans le périmètre du service afin de rendre la démonstration reproductible.

Le service **svc-catalogue-offres** gère le catalogue des offres, des forfaits, des prix et des règles d'éligibilité. Il utilise **DB Catalogue** (**catalogue_offres**) comme source de données principale et Redis pour optimiser les lectures fréquentes de **/v1/offers.**

Le service **svc-commandes-activations** gère la prise de commande et l'orchestration des activations. Il valide le client auprès de **svc-clients-identite**, vérifie l'offre auprès de **svc-catalogue-offres,** applique l'idempotence sur les commandes et activations, persiste l'état dans **DB Commandes** (**commandes_activations**) et délègue l'activation de ligne à **svc-lignes-services**. Il envoie aussi les événements de commande et d'activation vers **svc-conformite-audit**.

Le service **svc-lignes-services** est responsable des lignes mobiles, des MSISDN, des SIM/eSIM, de l'IMSI/SUPI et de l'activation réseau. Il maintient ses données dans **DB Lignes (lignes_services)** et utilise un adapter HLR/free5GC ou mock pour le provisionnement réseau. La portabilité MSISDN est simulée en Phase 1 : un **numberMode=PORT_IN** accepte un MSISDN fourni et enregistre **portabilityStatus=ACCEPTED**.

Le service **svc-usage-facturation** couvre la consultation des factures et le paiement UC-06. Il valide le client, persiste les factures et paiements dans **DB Facturation (usage_facturation)**, utilise une passerelle de paiement simulée et publie les événements de facture/paiement vers **svc-conformite-audit**. En Phase 1, le rating complet, la médiation CDR et le cycle mensuel automatisé sont limités ou conservés comme extensions futures.

Enfin, le service **svc-conformite-audit** centralise les événements d'audit provenant des autres services. Il conserve les traces dans **DB Audit (conformite_audit)** en mode append-only afin d'assurer la traçabilité, la conformité CRTC/Loi 25 et l'analyse des opérations sensibles.

5.2 Niveau 2 — Structure interne d'un service (hexagonal)

Le service représentatif choisi est **svc-commandes-activations**, responsable de la prise de commande, de l'idempotence et de l'orchestration des activations. Il suit une structure hexagonale pragmatique : les routes REST restent en périphérie, les cas d'utilisation applicatifs coordonnent le flux métier, le domaine porte les objets métier, et les dépendances techniques sont injectées sous forme de clients ou repositories.

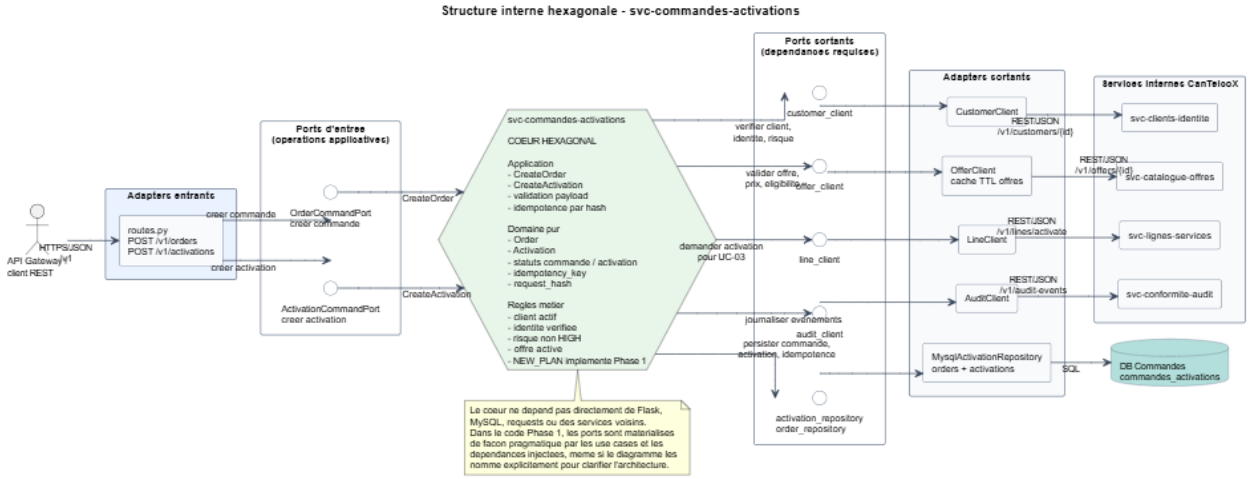


Figure 5.2 — Structure interne hexagonale de svc-commandes-activations

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

La figure représente le service svc-commandes-activations selon une architecture hexagonale. Le cœur du service est représenté par l'hexagone : il contient la couche application et le domaine pur. Les ports d'entrée et de sortie définissent les contrats utilisés par le cœur. Les adapters REST, SQL et interservices restent à l'extérieur de l'hexagone afin d'éviter que le domaine dépende directement de Flask, MySQL, requests ou des autres services.

Couche	Responsabilité	Exemples
Adapters entrants	Recevoir les appels externes, convertir les requêtes HTTP en commandes applicatives, valider le format de la requête et retourner les réponses HTTP. Ils ne contiennent pas de logique métier durable.	**routes.py, POST /v1/orders, POST /v1/activations**
Ports entrants	Définir les opérations offertes par le cœur du service aux adapters entrants. Ils représentent les cas d'utilisation accessibles depuis l'extérieur.	**OrderCommandPort, ActivationCommandPort**
Application	Orchestrer les cas d'utilisation, gérer l'idempotence, coordonner les appels aux ports sortants et construire les réponses applicatives.	**CreateOrder, CreateActivation**
Domaine — cœur pur	Porter les objets métier et les règles indépendantes des frameworks, de REST, de MySQL et des autres services.	**Order**, **Activation**, statuts commande/activation, **idempotency_key, request_hash**
Ports sortants	Définir les dépendances nécessaires au cœur applicatif sans dépendre des technologies concrètes. En Phase 1, ces ports sont matérialisés par des dépendances injectées.	**customer_client, offer_client, line_client, audit_client, activation_repository, order_repository**
Adapters sortants	Implémenter les ports sortants vers les technologies concrètes : base de données, API REST internes, audit et services voisins.	**CustomerClient, OfferClient, LineClient, AuditClient, MysqlActivationRepository**
Infrastructure technique	Fournir les mécanismes techniques transversaux utilisés par les adapters :	Flask, **requests**, MySQL, métriques Prometheus, logs

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

	configuration, logs, métriques, clients HTTP, connexion MySQL et variables d'environnement.	structurés, variables d'environnement

5.3 Modèle de domaine

Agrégats clés à modéliser (un diagramme par bounded context retenu) :

Agrégat	Entités / Value Objects	Invariants
Profil abonné	Entités: Customer, MfaChallenge. Value Objects conceptuels: CustomerId, Email, Telephone, Adresse, IdentityStatus, RiskLevel.	Un profil client doit posséder une identité vérifiée pour être utilisé dans les commandes et activations. Les données sensibles, comme les documents d'identité, ne doivent pas être stockées en clair. Les opérations d'inscription, authentification et MFA doivent être auditables.
Ligne / MSISDN	Entité : Line. Value Objects conceptuels : LineId, MSISDN, SIM/eSIM, IMSI, SUPI, LineStatus, HlrStatus, PortabilityStatus.	Un MSISDN, un identifiant SIM et un IMSI doivent être uniques. Une ligne ne peut passer à **ACTIVE** qu'après provisionnement réseau réussi via HLR/free5GC ou mock. En Phase 1, la portabilité est simulée avec **numberMode=PORT_IN** et **portabilityStatus=ACCEPTED**.
Commande / Activation	Entités : Order, Activation. Value Objects conceptuels : OrderId, ActivationId, CustomerId, OfferId, IdempotencyKey, RequestHash, OrderStatus, Money.	Une même clé d'idempotence ne doit pas créer deux commandes ou activations différentes. Une commande ou activation doit référencer un client actif, une identité vérifiée et une offre active. Les clients à risque élevé sont bloqués. Chaque opération sensible produit une trace d'audit.
Offre	Entité : Offer. Value Objects conceptuels : OfferId, Price, DataQuota, OfferStatus.	Une offre sélectionnable doit être active et posséder un prix exploitable. Une commande doit référencer une offre existante et active. Les options, versions complexes et règles

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Agrégat	Entités / Value Objects	Invariants
		d’éligibilité avancées restent des extensions futures.
Facture / Paiement	Entité principalement implémentée : Payment; factures persistées en base. Value Objects conceptuels : InvoiceId, BillingPeriod, Amount, Currency, PaymentId, PaymentStatus, TransactionReference, IdempotencyKey.	Un paiement ne doit pas être appliqué deux fois. Une clé d’idempotence rejouée avec le même contenu retourne le résultat existant ; rejouée avec un contenu différent, elle produit un conflit. Les paiements et changements de statut de facture doivent être auditables.
Conformité & Audit	Entité : AuditEvent. Value Objects conceptuels : AuditEventId, TraceId, Timestamp, ActorId, EventType, RiskLevel.	Le journal d’audit est append-only : un événement ne doit jamais être modifié ni supprimé. Chaque opération sensible doit produire une trace horodatée avec acteur, source, résultat, métadonnées et identifiant de corrélation.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

5.4 Organisation du code (vue Développement)

L'organisation du code suit le decoupage en services defini a partir des bounded contexts. Chaque service possede son propre dossier sous services/, son Dockerfile, ses dependances Python et ses tests locaux. Le depot contient aussi des dossiers transversaux pour l'infrastructure, la documentation, les tests systeme et les scripts de demonstration.

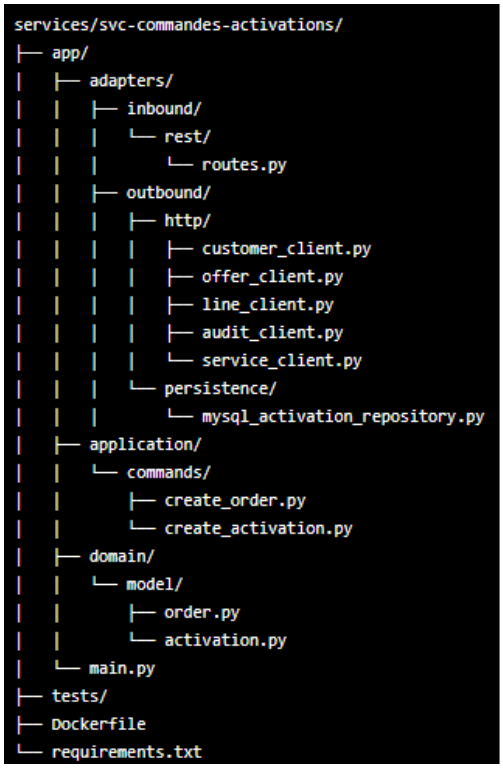
Arborescence proposée du dépôt

```
cantelcox/
├── docker-compose.yml
├── .env.example
├── README.md
├── docs/
│   ├── adr/
│   ├── api/
│   │   ├── openapi.yaml
│   │   └── postman/
│   ├── architecture/
│   │   └── diagrams/
│   ├── evidence/
│   └── runbooks/
├── infra/
│   ├── nginx/
│   │   └── nginx.conf
│   ├── mysql/
│   │   └── init/
│   ├── prometheus/
│   │   └── prometheus.yml
│   └── grafana/
├── services/
│   ├── shared/
│   ├── svc-clients-identite/
│   ├── svc-catalogue-offres/
│   ├── svc-commandes-activations/
│   ├── svc-lignes-services/
│   ├── svc-usage-facturation/
│   └── svc-conformite-audit/
├── tests/
│   ├── e2e/
│   ├── integration/
│   └── load/
└── scripts/
```


© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Structure interne type d'un service

Le service svc-commandes-activations est representatif de l'organisation interne retenue.



Conventions de dépendances entre couches

Couche	Responsabilité	Dépendances autorisées
adapters/inbound/rest	Expose les endpoints REST, lit les requetes HTTP, appelle les cas d'utilisation et retourne les reponses HTTP.	Peut dependre de **application** et des composants configures dans **main.py**. Ne doit pas contenir la logique metier durable.
application	Orchestre les cas d'utilisation, l'idempotence, les validations applicatives et les appels aux dependances sortantes.	Peut dependre de **domain** et de dependances injectees comme repositories ou clients HTTP.
domain	Contient les objets metier et les invariants independants des frameworks.	Ne depend pas de Flask, MySQL, Redis, requests ou des autres services.
adapters/outbound/http	Encapsule les appels REST vers les autres services.	Peut dependre de bibliotheques HTTP et du format d'erreur commun.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

adapters/outbound/persistence	Implemente la persistance MySQL et les contraintes techniques comme les cles uniques d'idempotence.	Peut dependre du connecteur MySQL et mapper les objets du domaine vers les tables.
services/shared	Fournit les fonctions transversales communes: erreurs JSON, logs structures, metriques, hooks HTTP, connexion MySQL.	Utilise par les services, sans porter de logique metier propre a unbounded context.

Règles de développement

- Les noms utilises dans le code doivent rester coherents avec le langage ubiquitaire du rapport : **Customer, MfaChallenge, Line, MSISDN, Order, Activation, Offer, Payment, AuditEvent.**
- Les details techniques comme MySQL, Redis, clients REST, Prometheus, logs et variables d'environnement restent dans **adapters/, services/shared, infra/ ou main.py.**
- Le contrat OpenAPI central est conserve dans **docs/api/openapi.yaml;** la collection Postman est dans **docs/api/postman/.**
- Les tests unitaires propres a chaque service sont dans **services/<service>/tests/.**
- Les tests transversaux sont dans **tests/e2e/, tests/integration/** et **tests/load/.**
- Les scripts d'automatisation et de demonstration sont dans **scripts/.**
- Les secrets reels ne sont pas versionnes; le depot conserve seulement des valeurs d'exemple comme **.env.example.**

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

6. Vue d’exécution

Objectif Arc42 — *Comportement dynamique : interactions entre composants pour les scénarios clés. Couvre la vue Processus (C&C) de 4+1.*

Repères de l’énoncé

Tâches de l’énoncé : 4.3 Exposer l’API REST publique (scénario bout-en-bout) ; 3.2 idempotence à l’exécution.

Vue 4+1 : Processus (C&C).

Livrables attendus :

- Diagrammes de séquence pour les scénarios clés.
- Scénario E2E : inscription → MFA → activation ligne → souscription forfait → consultation usage.

Critères d’acceptation (à cocher)

☑ Scénario bout-en-bout démontrable dans la VM, erreurs normalisées JSON.

☑ Double soumission d’une commande sans effet de bord (idempotence) illustrée.

6.1 Scénario bout-en-bout (E2E)

La figure suivante présente le scénario E2E démontrable en Phase 1 à travers l’API Gateway. Le parcours commence par l’inscription d’un abonné, puis l’authentification avec MFA simulé. Une fois le **sessionToken** obtenu, l’abonné consulte les offres actives, crée une commande idempotente, active une ligne mobile, consulte ses factures et effectue un paiement de facture. Les opérations sensibles produisent des événements vers **svc-conformite-audit**, consultables ensuite par **TraceId.**

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

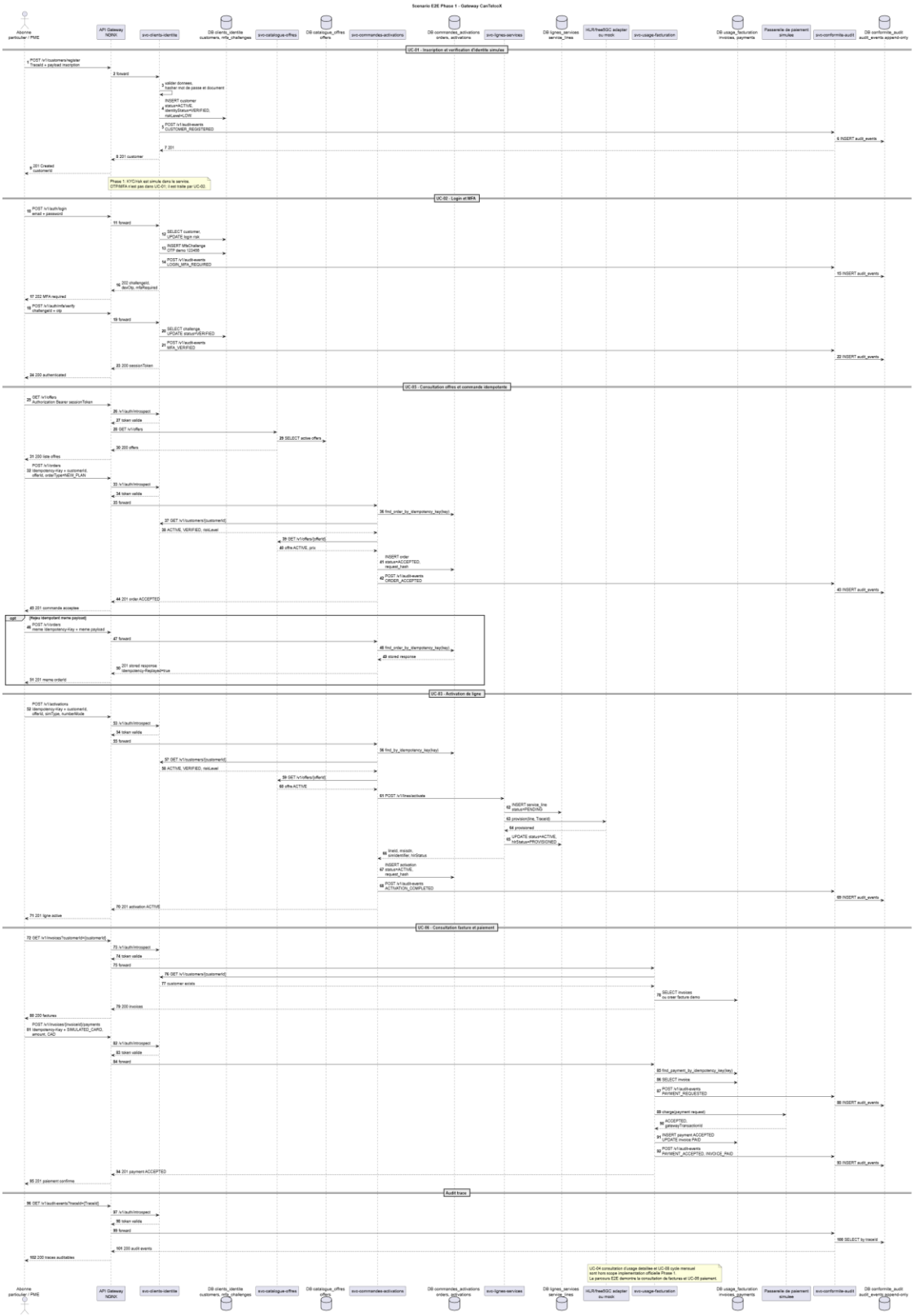


Figure 6.1 — Scénario E2E Phase 1 via API Gateway

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Ce scénario illustre la traversée des principaux services: **svc-clients-identite, svc-catalogue-offres, svc-commandes-activations, svc-lignes-services, svc-usage-facturation et svc-conformite-audit**. L'API Gateway centralise l'accès aux routes **/v1**, propage les en-têtes de corrélation et valide les routes protégées par introspection du jeton de session.

L'idempotence est illustrée par le rejeu d'une commande avec la même **Idempotency-Key** et le même payload: le service retourne la réponse déjà persistée sans créer de doublon. La même stratégie est appliquée aux activations et paiements. En Phase 1, la consultation d'usage détaillée UC-04 et le cycle mensuel automatisé UC-08 restent limités ou futurs. Le scénario E2E officiel couvre plutôt la consultation de factures et le paiement UC-06, conformément au comportement actuellement implémenté dans le dépôt.

6.2 Authentification & MFA (UC-02)

La figure présente le scénario UC-02 d'authentification et MFA. L'abonné soumet d'abord son courriel et son mot de passe à **POST /v1/auth/login.** Si les identifiants sont valides et que le compte est actif, **svc-clients-identite** crée un challenge MFA simulé avec un OTP de démonstration et journalise l'événement **LOGIN_MFA_REQUIRED**.

L'abonné valide ensuite le challenge avec **POST /v1/auth/mfa/verify**. Une session de démonstration `**dev-session-{challengeId}** est créée uniquement si le challenge existe, n'est pas expiré, est encore en statut **PENDING** et que l'OTP est correct. Les échecs MFA, expirations et validations réussies sont journalisés dans **svc-conformite-audit**.

Pour les opérations protégées, comme l'activation d'une ligne ou le paiement d'une facture, l'API Gateway exige un jeton **Authorization: Bearer**. Elle valide ce jeton auprès de **svc-clients-identite** via **/v1/auth/introspect** avant de transmettre la requête au service métier concerné. En Phase 1, le MFA est simulé et le niveau **mfaStrength** est retourné par l'authentification, mais la Gateway ne met pas encore en œuvre un moteur séparé de politique MFA par niveau de risque.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

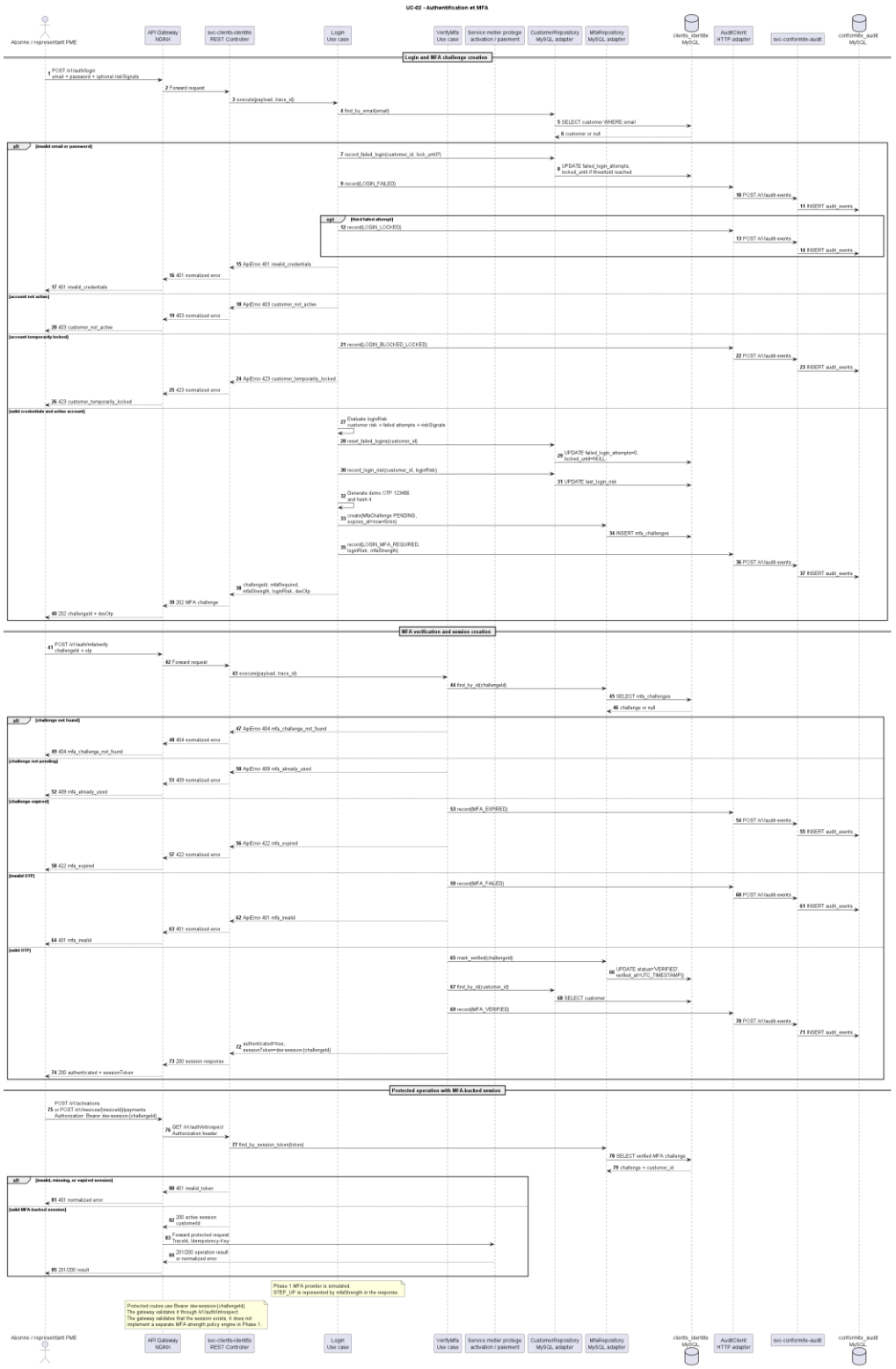


Figure 6.2 — Diagramme de séquence UC-02 Authentification et MFA.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

6.3 Idempotence & exactly-once à l'exécution

Cette section illustre deux mécanismes d'intégrité à l'exécution: l'idempotence des commandes avec **Idempotency-Key** et **l'exactly-once pratique** pour les écritures de paiement.

Lors de la première soumission, **svc-commandes-activations** calcule un **request_hash** à partir du payload, vérifie que la clé **Idempotency-Key** n'existe pas encore, valide le client et l'offre, puis persiste la commande avec la clé, le hash, la réponse JSON et le statut HTTP retourné. La table **orders** possède une contrainte unique sur **idempotency_key**.

Lors d'une soumission répétée avec la même clé et le même payload, le service retrouve la commande existante et retourne la réponse déjà enregistrée avec l'en-tête **Idempotency-Replayed=true**. Aucune nouvelle commande n'est créée. Si la même clé est réutilisée avec un payload différent, le hash ne correspond plus et le service retourne **409 idempotency_key_conflict**.

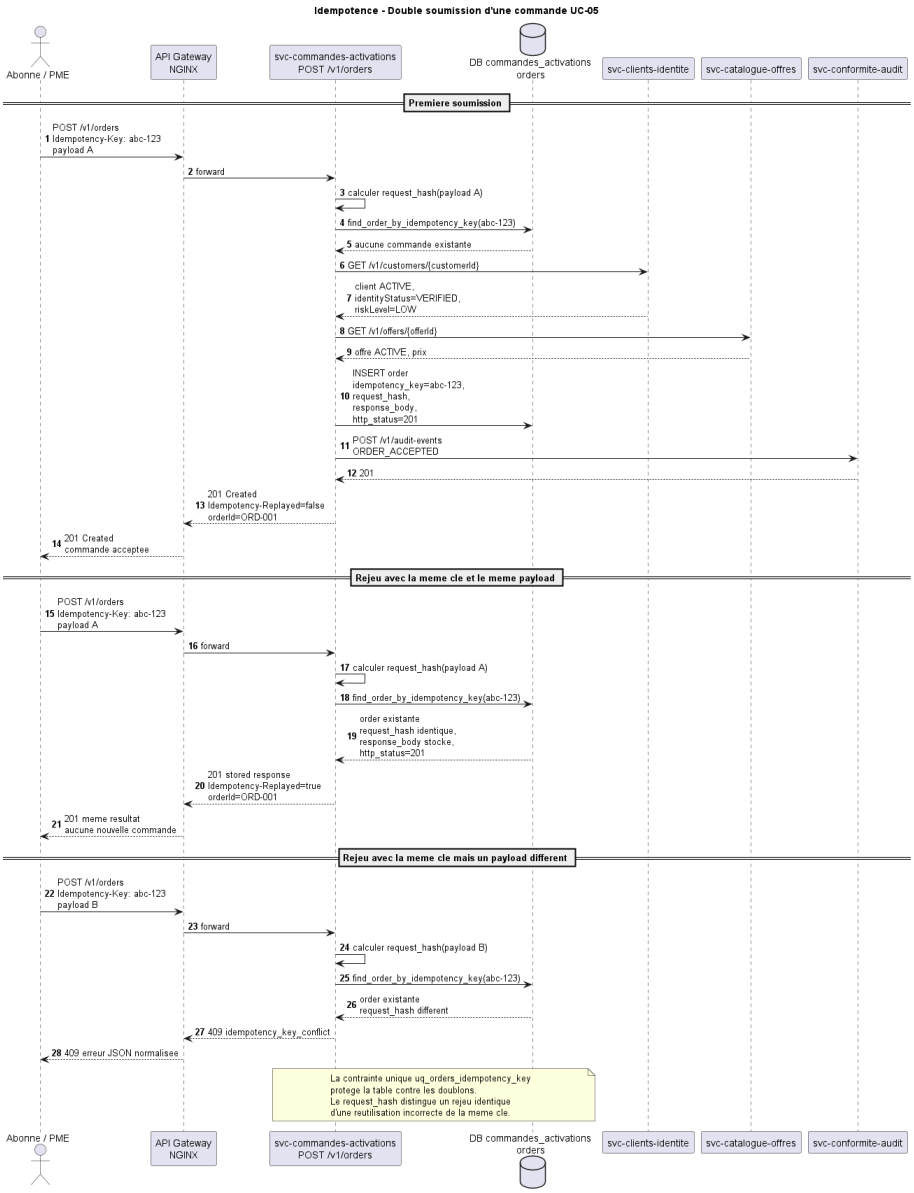


Figure 6.3 — Idempotence d'une commande UC-05

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

6.3.2 Paiement de facture et exactly-once pratique

Pour le paiement d'une facture, **svc-usage-facturation** applique le même principe d'idempotence avec une contrainte unique sur **payments.idempotency_key**. Le service vérifie aussi que la facture existe, appartient au client, est encore ouverte, et que le montant et la devise correspondent.

Lorsque le paiement est accepté par la passerelle simulée, le service insère le paiement et marque la facture comme **PAID** dans une même transaction locale. Un rejeu avec la même clé et le même payload retourne le paiement déjà enregistré sans appliquer un second paiement. Une réutilisation de la même clé avec un payload différent retourne **409 idempotency_key_conflict.**

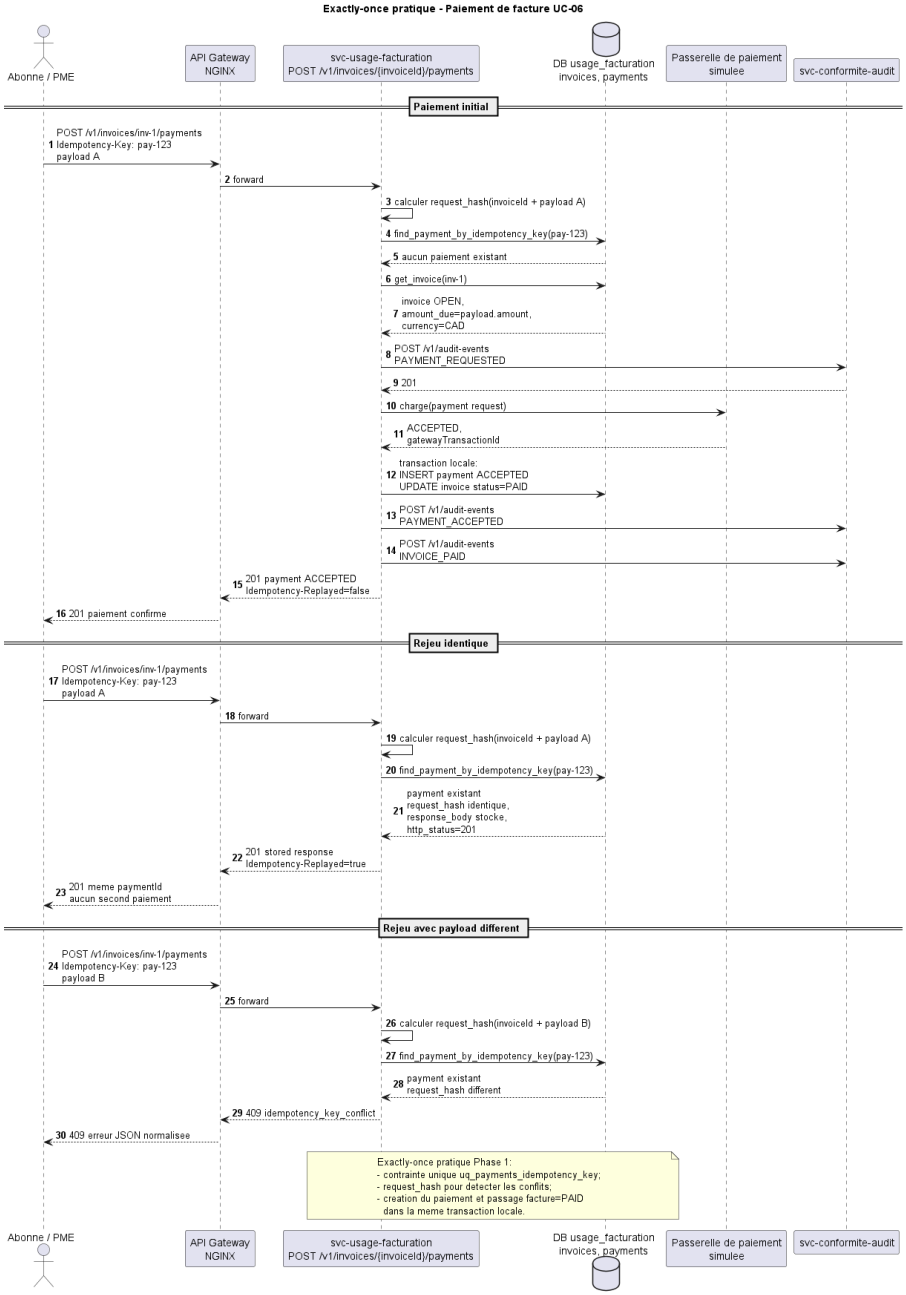


Figure 6.4 — Exactly-once pratique d'un paiement UC-06

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

7. Vue de déploiement

Objectif Arc42 — *Infrastructure technique : nœuds, conteneurs, répartition des artefacts. Couvre la vue Déploiement de 4+1.*

Repères de l'énoncé

Tâches de l'énoncé : 8.1 conteneurisation ; 6.1 API Gateway ; 5.3 load balancing.

Vue 4+1 : Déploiement.

Livrables attendus :

- Topologie docker-compose : services + DB + Prometheus + Grafana + Gateway + cache + seed ; healthchecks /health.
- API Gateway (Kong/KrakenD/Spring Cloud Gateway) : routage, en-têtes/clé API, CORS, throttling.
- Load balancing (NGINX/HAProxy/Traefik) pour N = 1..4 instances.

Critères d'acceptation (à cocher)

☑ docker compose up lance l'ensemble ; healthchecks OK.

☑ Appels fonctionnels via la Gateway ; configuration versionnée.

7.1 Topologie de déploiement

La plateforme est déployée sur une VM ou machine locale au moyen de Docker Compose. L'entrée publique de l'API est le conteneur **api-gateway**, basé sur NGINX, exposé par défaut sur le port **8080**. Il route les appels **/v1/*** vers les six services Flask/Gunicorn: **svc-clients-identite, svc-catalogue-offres, svc-commandes-activations, svc-lignes-services, svc-usage-facturation** et **svc-conformite-audit**.

La persistance est centralisée dans un conteneur MySQL 8.4, avec un volume persistant **mysql-data**. Les scripts SQL de **infra/mysql/init** initialisent les schémas et les données de démonstration. Le conteneur **mysql-bootstrap** complète la création des bases et des privilèges par service. Redis est utilisé comme cache pour les offres du catalogue.

L'observabilité est fournie par Prometheus et Grafana. Prometheus collecte les métriques **/metrics** exposées par les services applicatifs, et Grafana charge les dashboards et datasources depuis **infra/grafana**. Swagger UI expose le contrat OpenAPI central **docs/api/openapi.yaml**.

Les tests de charge sont exécutés avec Locust via le profil Compose **load-test**. La Gateway NGINX utilise des upstreams Docker DNS, **least_conn** et **proxy_next_upstream**, ce qui permet de démontrer le load balancing lorsque certains services sont répliqués de N = 1 à N = 4 instances.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

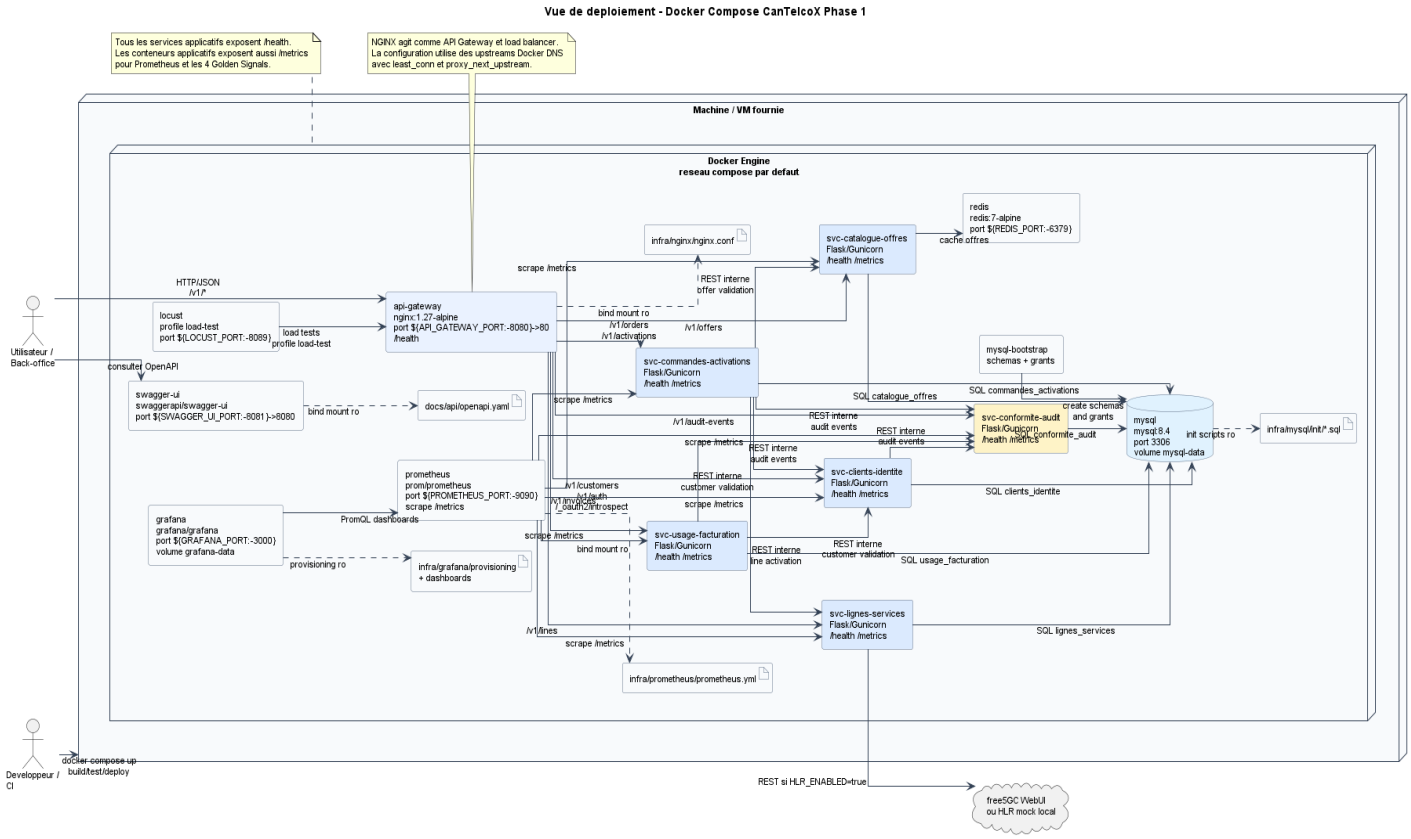


Figure 7.1 — Topologie de déploiement Docker Compose de CanTelcoX Phase 1

7.2 API Gateway

Capacité Gateway	Configuration
Routage	La Gateway expose un point d'entrée unique sur le port **API_GATEWAY_PORT**, **8080** par défaut. Elle redirige **/v1/customers et /v1/auth** vers **svc-clients-identite**, **/v1/offers** vers **svc-catalogue-offres**, **/v1/orders et /v1/activations** vers **svc-commandes-activations, /v1/lines vers svc-lignes-services, /v1/invoices** et **/v1/invoices/{invoiceId}/payments** vers **svc-usage-facturation,** et **/v1/audit-events** vers **svc-conformite-audit**.
Authentification	Les routes publiques comme **/v1/customers/register** et **/v1/auth/*** sont accessibles sans introspection. Les routes protégées utilisent **auth_request /_oauth2/introspect,** qui appelle **svc-clients-identite** sur **/v1/auth/introspect**. Le jeton utilisé est le **sessionToken** retourné après MFA.
En-têtes propagés	La Gateway propage **Authorization**, **X-Request-Id**, **TraceId** et **Idempotency-Key**. Elle expose aussi **TraceId**, **X-Request-Id** et **Idempotency-Key** dans les réponses.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Capacité Gateway	Configuration
CORS	La configuration actuelle autorise **Access-Control-Allow-Origin**: * pour simplifier la démonstration. Les méthodes autorisées sont **GET, POST, PUT, PATCH, DELETE** et **OPTIONS.** Les en-têtes autorisés incluent **Authorization**, **Content-Type**, **Idempotency-Key**, **TraceId** et **X-Request-Id**.
Produit retenu	NGINX **1.27-alpine** comme API Gateway / reverse proxy léger. Il sert de point d'entrée unique, applique le routage **/v1/*,** centralise les en-têtes, permet le load balancing et reste compatible avec Docker Compose.
Timeout	Les routes standards utilisent principalement **5s**. Les routes plus longues utilisent **10s** pour **/v1/lines**, **/v1/orders** et **/v1/invoices**, et **15s** pour **/v1/activations**, car l'activation peut appeler le provisionnement réseau simulé/free5GC.
Load balancing	Les upstreams NGINX utilisent Docker DNS avec **least_conn**. **proxy_next_upstream** permet un retry sur **error, timeout, 502, 503** et **504**, avec **proxy_next_upstream_tries 2.**
Healthcheck	La Gateway expose **/health**, utilisé par Docker Compose pour vérifier que le conteneur est prêt.
Throttling / quota (optionnel)	Non implémenté dans la configuration Phase 1. Il reste une extension possible pour protéger les routes sensibles comme l'authentification, le MFA, la commande, l'activation et le paiement.

7.3 Load balancing & tolérance aux pannes

Le load balancing est assuré par NGINX, utilisé comme API Gateway et reverse proxy. Les services applicatifs peuvent être répliqués de N = 1 à N = 4 instances dans Docker Compose pour les campagnes de mesure. NGINX utilise les upstreams Docker DNS avec la stratégie **least_conn**, ce qui répartit les requêtes vers l'instance ayant le moins de connexions actives.

Chaque service applicatif expose un endpoint **/health**, utilisé par Docker Compose pour vérifier l'état des conteneurs. La Gateway expose également **/health**. Pour la tolérance aux pannes, NGINX est configuré avec **proxy_next_upstream** sur **error**, **timeout**, **502**, **503** et **504**, avec deux tentatives maximum.

Élément	Choix retenu
Composant de load balancing	NGINX comme API Gateway / reverse proxy
Stratégie	**least_conn** entre les instances disponibles d'un même service
Services ciblés	Principalement **svc-catalogue-offres, svc-commandes-activations et svc-lignes-services**, car ils sont sollicités par les scénarios UC-03 et UC-05 ainsi que par la consultation d'offres
Nombre d'instances testées	N = 1, 2, 3 et 4 instances

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Healthcheck	Endpoint **/health** exposé par chaque service et par la Gateway ; healthchecks Docker Compose
Tolérance aux pannes	Arrêt manuel d'une instance pendant un test avec N ≥ 2 ; NGINX doit continuer à router vers les instances restantes lorsque possible
Retry Gateway	**proxy_next_upstream error timeout http_502 http_503 http_504**, avec **proxy_next_upstream_tries 2**
Observabilité	Suivi avec Prometheus/Grafana: latence, taux d'erreur, débit et saturation
Résultats	Les résultats détaillés sont présentés au §10.6

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

8. Concepts transversaux

Objectif Arc42 — *Idées et mécanismes appliqués de façon transversale : persistance, intégrité, sécurité, observabilité, gestion d'erreurs, caching.*

Repères de l'énoncé

Tâches de l'énoncé : 3.1–3.2 persistance & intégrité ; 4.4 / 7.2 sécurité ; 5.1 observabilité ; 5.4 caching.

Livrables attendus :

- Schéma de persistance par service (ORM/DAO), contraintes d'intégrité, migrations, seeds.
- Idempotence (idempotency-key), exactly-once (facturation), journal d'audit append-only.
- Sécurité : CORS, Basic/JWT, MFA/OTP, validation, secrets via variables d'environnement.
- Observabilité : logs structurés, métriques Prometheus, dashboards Grafana (4 Golden Signals).
- Caching (mémoire/Redis) sur endpoints coûteux + règles d'invalidation.

Critères d'acceptation (à cocher)

☑ CRUD robuste, rollback sur erreur, double soumission sans effet de bord.

☑ Garantie exactly-once sur écritures de facturation; journal append-only opérationnel.

☑ Aucun secret en clair; MFA fonctionnel sur UC-02; routes sensibles protégées par introspection du jeton via la Gateway.☑ 4 Golden Signals observés (P95/P99, RPS, 4xx/5xx, saturation).

☑ Gains de cache chiffrés (latence, charge DB) + stratégie d'invalidation.

8.1 Persistance & intégrité

La persistance est organisée par service afin de respecter les frontières des bounded contexts. Chaque service possède ses propres tables logiques dans MySQL et accède à ses données par une couche repository/DAO. La logique métier ne dépend pas directement de MySQL : les repositories masquent les détails SQL, les transactions locales et les contraintes techniques.

Élément	Choix retenu
Type de persistance	Base relationnelle MySQL 8.4, avec séparation logique par schéma/service.
Accès aux données	Repositories/DAO par service, implémentés avec SQL direct.
Transactions	Transactions locales par service avec **commit**; les erreurs empêchent la validation de l'écriture courante.
Initialisation	Scripts SQL versionnés dans **infra/mysql/init/*.sql**, exécutés au démarrage MySQL, plus **mysql-bootstrap** pour créer les schémas et privilèges.
Seeds	Offres de démonstration dans **catalogue_offres.offers** ; clients créés via UC-01; factures de démonstration créées à la demande par **svc-usage-facturation**.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Intégrité	Contraintes d'unicité, index par identifiants métier/statuts et clés étrangères internes lorsqu'elles restent dans le même service.
Observabilité BD	Les erreurs remontent via logs structurés et réponses JSON normalisées ; les métriques HTTP exposent latence, trafic et erreurs par service.
Références inter-services	Références logiques par identifiants **(customer_id, offer_id, line_id, trace_id)**, sans clés étrangères physiques entre schémas de services.

Contraintes d'intégrité principals

Domaine	Contraintes
Clients & Identité	**customer_id** unique; **email** unique; **mfa_challenges.customer_id** référence **customers.customer_id**; index sur statut client et MFA. Les documents d'identité et mots de passe sont stockés sous forme de hash.
Lignes & Services	**line_id** unique; **msisdn** unique; **sim_identifier** unique; **imsi** unique; index sur **customer_id** et **status**. Le provisionnement réussi met la ligne en **ACTIVE** et **hlr_status=PROVISIONED**.
Commandes & Activations	**order_id** et **activation_id** uniques; **idempotency_key** unique dans **orders** et **activations**; **request_hash** permet de distinguer un rejeu identique d'un conflit de payload.
Catalogue & Offres	**offer_id** unique; index sur **status**; seules les offres **ACTIVE** sont utilisables dans les commandes/activations.
Usage / Facturation	**invoice_id** et **payment_id** uniques; **payments.idempotency_key** unique; contrainte **uq_successful_payment_per_invoice(invoice_id, status);** paiement et passage de facture à **PAID** dans une transaction locale.
Conformité & Audit	**audit_event_id** unique; index sur **trace_id**, **customer_id + created_at** et **event_type**; triggers SQL interdisant **UPDATE** et **DELETE** sur **audit_events**.

8.2 Idempotence (commandes & activations)

Les opérations de création de commande et d'activation de ligne sont protégées par un mécanisme **Idempotency-Key**. Le client fournit une clé unique dans l'en-tête HTTP **Idempotency-Key** lors des appels sensibles **POST /v1/orders** et **POST /v1/activations**.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Élément	Mécanisme retenu
Portée	La clé est unique dans la table de l'opération concernée: **orders.idempotency_key** ou **activations.idempotency_key.**
Stockage	La clé est stockée avec **request_hash, response_body, http_status** et le statut métier.
Fenêtre de rejeu	Pas d'expiration automatique en Phase 1. La clé reste conservée en base tant que l'enregistrement existe.
Double soumission	Même clé + même payload: retour de la réponse déjà enregistrée avec **Idempotency-Replayed=true**. Même clé + payload différent : **409 idempotency_key_conflict**.
Contrainte DB	Contraintes uniques **uq_orders_idempotency_key** et **uq_activations_idempotency_key**.

8.3 Exactly-once (facturation)

Les paiements de facture utilisent une garantie exactly-once pratique afin d'éviter qu'un paiement soit appliqué plusieurs fois. En Phase 1, cette garantie repose sur l'idempotence, les contraintes d'unicité et une transaction locale.

Élément	Mécanisme retenu
Déduplication	**payments.idempotency_key** est unique. Le service vérifie si un paiement existe déjà avant d'écrire.
Transactions	Le paiement est inséré et la facture passe à **PAID** dans la même transaction locale.
Hash de requête	**request_hash** distingue un rejeu identique d'une réutilisation incorrecte de la même clé.
Contraintes d'unicité	**uq_payments_idempotency_key** et **uq_successful_payment_per_invoice(invoice_id, status).**
Rejeu	Même clé + même payload: retour du paiement déjà enregistré. Même clé + payload différent: **409 idempotency_key_conflict.**
Limite Phase 1	Pas de table **outbox_events** implémentée. Les événements d'audit sont envoyés via **AuditClient** après les opérations métier.

8.4 Journal d'audit append-only (CRTC / Loi 25)

Le journal d'audit est conçu comme un registre append-only. Les événements sensibles sont ajoutés au journal, mais ne peuvent pas être modifiés ni supprimés..

Élément	Mécanisme retenu
Structure	Chaque événement contient **audit_event_id, event_type, actor_id, customer_id, resource_id, trace_id, source_service, result, metadata, created_at** et **inserted_at.**

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Événements consignés	Inscription, login/MFA, commandes, activations, paiements, paiement de facture, erreurs ou refus métier importants.
Immutabilité	Triggers SQL **audit_events_no_update** et **audit_events_no_delete** interdisent **UPDATE** et **DELETE**.
Traçabilité	**trace_id** relie les événements aux requêtes HTTP, logs structurés et métriques..
Conformité	Le journal supporte la traçabilité des opérations sensibles pour la conformité CRTC/Loi 25 et les démonstrations d'audit.

8.5 Sécurité applicative

Mécanisme	Mise en œuvre
Authentification	Authentification par courriel/mot de passe. Après MFA réussi, le service retourne un jeton de démonstration **dev-session-{challengeId}**, utilisé dans **Authorization: Bearer <token>**. Ce n'est pas un JWT de production.
MFA / OTP	UC-02 implémente un OTP simulé. Les routes protégées, comme activation, commande, consultation facture et paiement, passent par l'introspection Gateway **/v1/auth/introspect.**
CORS	La configuration NGINX actuelle autorise **Access-Control-Allow-Origin: *** pour simplifier la démo. Les méthodes autorisées incluent **GET, POST, PUT, PATCH, DELETE** et **OPTIONS.**
Validation	Les entrées sont validées dans les routes et cas d'utilisation : champs obligatoires, valeurs autorisées, formats métier et **Idempotency-Key** requis pour commandes, activations et paiements.
Secrets	Les secrets sont fournis par variables d'environnement ou **.env** non versionné. **.env.example** sert de modèle.
Données sensibles	Les mots de passe et numéros de document d'identité sont hashés ; les logs ne doivent pas exposer de secrets ou données réglementées.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

8.6 Contrôles anti-fraude

Les contrôles anti-fraude de Phase 1 restent volontairement simples. Ils couvrent surtout le niveau de risque client, les signaux de risque au login/MFA et le blocage des commandes ou activations pour les clients à risque élevé. Les scénarios avancés comme SIM swap complet, roaming fraud, détection d'usage anormal par CDR et alertes fraude dédiées restent des extensions futures..

Risque détecté	Règles de détection	Réaction du système	Tests associés
Usurpation / client à risque élevé	Le client possède un riskLevel. Les commandes et activations vérifient que le client est **ACTIVE**, que **identityStatus=VERIFIED** et que **riskLevel** n'est pas **HIGH**.	Refus de la commande ou activation avec erreur JSON normalisée ; événement d'audit.	Tests **test_high_risk_customer_blocks_order_and_audits** et **test_high_risk_customer_blocks_activation_and_audits_decision**.
Connexion à risque	UC-02 évalue des **riskSignals** simulés comme **newDevice**, **roaming**, **unusualLocation**, **simSwapRecent**.	Retour d'un **mfaStrength** standard ou renforcé ; journalisation des événements login/MFA.	Tests UC-02 MFA/login.
Tentatives MFA invalides ou expirées	Le challenge MFA peut être invalide, expiré ou déjà utilisé.	Retour **401**, **409** ou **422** selon le cas; événement d'audit pour échec ou expiration.	Tests **test_auth_mfa.py**.
SIM swap suspect	Non implémenté comme workflow complet en Phase 1. Le signal **simSwapRecent** peut influencer le risque MFA, mais il n'existe pas de changement SIM/eSIM dédié.	Extension future: MFA renforcé, blocage temporaire, alerte fraude.	Futur
Usage anormal / roaming fraud	Non implémenté en Phase 1, car la médiation CDR et UC-04 usage détaillé sont limitées ou hors scope officiel.	Extension future : détection CDR, alerte fraude, back-office.	Futur

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

8.7 Gestion d’erreurs & versionnement

Les services utilisent un format d’erreur JSON uniforme afin d’assurer une cohérence entre l’API Gateway et les services. Chaque erreur contient un code applicatif, un message lisible, des détails, un identifiant de corrélation et un horodatage.

Format d’erreur JSON unifié

```json
{
  "code": "idempotency_key_conflict",
  "message": "Idempotency-Key was already used with a different payload",
  "details": {},
  "traceId": "trace-demo-1",
  "timestamp": "2026-07-08T12:00:00Z"
}
```

Figure 8.1 — Exemple de format d’erreur JSON normalisé.

Code HTTP	Utilisation
400 Bad Request	Requête invalide, champ manquant ou format incorrect.
401 Unauthorized	Authentification absente, invalide, MFA invalide ou jeton invalide.
403 Forbidden	Client authentifié mais non admissible à l’opération.
404 Not Found	Ressource inexistante : client, challenge MFA, offre, ligne ou facture.
405 Method Not Allowed	Méthode HTTP non supportée.
409 Conflict	Conflit métier : clé d’idempotence réutilisée avec payload différent, MSISDN déjà utilisé, facture déjà payée, compte temporairement incompatible.
422 Unprocessable Entity	Requête syntaxiquement valide mais impossible selon l’état métier, par exemple MFA expiré.
423 Locked	Compte temporairement verrouillé après échecs de connexion.
500 Internal Server Error	Erreur technique non prévue.
503 Service Unavailable	Dépendance ou simulateur temporairement indisponible, par exemple HLR/free5GC ou passerelle de paiement simulée.

Cohérence inter-services

Tous les services retournent une structure d’erreur JSON normalisée et propagent le TraceId reçu de l’API Gateway. Si aucun TraceId n’est fourni, la Gateway ou le service appelé peut en générer un. Cela permet de relier une erreur HTTP aux logs applicatifs, aux métriques Prometheus et aux événements d’audit.

Politique de versionnement

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Les API exposées sont versionnées avec le préfixe **/v1**, par exemple **/v1/customers**, **/v1/auth**, **/v1/orders, /v1/activations, /v1/invoices** et **/v1/invoices/{invoiceId}/payments**. Les changements compatibles restent dans **/v1.** Les changements incompatibles nécessiteront une nouvelle version majeure, par exemple **/v2**, afin de préserver la compatibilité avec les clients existants.

8.8 Observabilité (4 Golden Signals)

Chaque service applicatif expose **/health** et **/metrics**. Prometheus scrape les métriques HTTP exposées par les six services, et Grafana fournit un dashboard pour les 4 Golden Signals. Les logs structurés incluent **timestamp, level, message, service, traceId, method, path, statusCode et durationMs.**

Golden Signal	Métrique	Cible
Latence	Histogramme **cantelcox_http_request_duration_seconds**, P95/P99 dans Grafana/Prometheus.	Objectif de démonstration : P95 ≤ 500 ms sur les parcours mesurés si l'environnement local le permet.
Trafic	Compteur **cantelcox_http_requests_total** agrégé en RPS.	Mesurer le débit atteint durant Locust ; ne pas déclarer ≥ 600 ops/s sans preuve.
Erreurs	**cantelcox_http_requests_total** par statut HTTP.	Suivi des 4xx/5xx ; objectif : 5xx faible pendant les scénarios nominaux.
Saturation	Ressources conteneurs/VM, logs, latence sous charge.	À documenter avec les captures Docker/Grafana/Locust disponibles.

Les résultats, captures Prometheus/Grafana et rapports Locust sont consignés au §10.5. Dans l'implémentation Phase 1, UC-04 usage détaillé et UC-08 cycle mensuel sont limités ; les mesures exploitent donc les parcours disponibles, notamment UC-02, UC-03, UC-05 et UC-06 paiement/factures.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

8.9 Caching

En Phase 1, le cache est volontairement limité au catalogue d'offres. Redis est utilisé par **svc-catalogue-offres** pour réduire la latence et la charge MySQL sur les lectures fréquentes. Les réponses exposent l'en-tête **X-Cache** avec **MISS**, **HIT** ou **BYPASS**.

Endpoint / donnée	Stratégie de cache	TTL	Invalidation
GET /v1/offers	Cache Redis de la liste des offres actives, clé **catalogue:offers:active.**	**OFFER_CACHE_TTL_SECONDS**, défaut 60 s.	Expiration TTL. Le catalogue est seed/read-only en Phase 1.version d'offre.
GET /v1/offers/{offerId}	Cache Redis par offre, clé **catalogue:offer:{offerId}.**	**OFFER_CACHE_TTL_SECONDS**, défaut 60 s.	Expiration TTL. Une future API d'administration devra invalider les clés concernées.

Les écritures critiques, comme la création de commande, l'activation de ligne, le paiement et la facturation, ne sont pas servies depuis le cache. La consultation d'usage **/v1/usage** et le cache des factures ne sont pas implémentés en Phase 1.

Les gains mesurés du cache sont présentés au §10.7, notamment la comparaison de latence avec cache activé et désactivé.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

9. Décisions d’architecture (ADR)

Objectif Arc42 — *Consigner les décisions structurantes au format ADR (statut, contexte, décision, conséquences), chacune traçable à une exigence du cahier.*

Repères de l’énoncé

Tâches de l’énoncé : 2.4 Consigner les décisions structurantes (≥ 3 ADR).

Livrables attendus :

- ADR-001 — Style architectural & découpage en services.
- ADR-002 — Persistance & transactions (idempotence / exactly-once / journal append-only — conformité CRTC/Loi 25).
- ADR-003 — Stratégie d’erreurs & versionnement d’API.

Critères d’acceptation (à cocher)

☑ Format ADR complet (statut, contexte, décision, conséquences).

☑ Chaque ADR explicitement traçable à une exigence du cahier.

ADR-001 — Style architectural & découpage en services

Statut	Accepté
Contexte	CanTelcoX doit offrir plusieurs capacités métier distinctes : gestion des clients, catalogue d’offres, commandes, activation de lignes, usage/facturation et conformité/audit. Le système doit être démontrable dans une VM, exposer des API REST, supporter l’observabilité, l’idempotence et une évolution future par domaine métier. Une architecture trop monolithique rendrait le découpage DDD, les tests, le déploiement et l’évolution plus difficiles.
Décision	Adopter une architecture orientée services, proche microservices, découpée selon les bounded contexts DDD: **svc-clients-identite, svc-catalogue-offres, svc-commandes-activations, svc-lignes-services, svc-usage-facturation** et **svc-conformite-audit**. Chaque service possède sa responsabilité métier, son API REST, sa couche domaine, sa
Conséquences	Le découpage améliore la séparation des responsabilités, la traçabilité entre cas d’utilisation et domaines, l’évolutivité, l’observabilité par service et la possibilité de répliquer certains services indépendamment. Il facilite aussi l’application du style hexagonal et la documentation OpenAPI par service. En contrepartie, l’architecture ajoute de la complexité opérationnelle : plusieurs conteneurs, plusieurs APIs, communication inter-services, configuration Gateway et observabilité distribuée. Des mécanismes supplémentaires sont donc nécessaires pour gérer les erreurs, l’idempotence, les timeouts et la cohérence des données entre services.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Exigence du cahier tracée	Découpage architectural, documentation Arc42/4+1/ADR, exposition d'API REST, conteneurisation Docker Compose, observabilité et séparation en services alignés sur les domaines métier.

ADR-002 — Persistance & transactions (idempotence / exactly-once / append-only)

Statut	Accepté
Contexte	CanTelcoX traite des opérations sensibles comme l'inscription, la création de commandes, l'activation de lignes, les paiements et les événements de conformité. Ces opérations ne doivent pas produire de doublons en cas de retry, timeout ou double soumission. Le système doit aussi conserver une trace fiable des opérations critiques pour répondre aux exigences de conformité télécom, CRTC simulé et protection des renseignements personnels.
Décision	Utiliser une persistance MySQL avec schémas isolés par service. Les commandes, activations et paiements utilisent des colonnes d'idempotence: **idempotency_key**, **request_hash** et **response_body** rejouable. Les paiements appliquent une garantie exactly-once pratique par contraintes d'unicité, déduplication et transaction locale. Les événements critiques sont enregistrés dans un journal d'audit append-only dans **svc-conformite-audit,** sans modification ni suppression des événements existants.
Conséquences	Cette décision réduit les effets de bord lors des doubles soumissions et améliore l'intégrité des données. Une tentative répétée de commande, d'activation ou de paiement peut retourner le résultat original si la requête est identique. Le journal append-only améliore la traçabilité et la conformité. En contrepartie, la cohérence inter-services reste gérée par orchestration claire, statuts métier et garde-fous locaux plutôt que par transactions distribuées.
Exigence du cahier tracée	Persistance, intégrité transactionnelle, idempotence, exactly-once pratique pour la facturation/paiement, journal d'audit append-only, conformité CRTC simulée et Loi 25 / protection des renseignements personnels.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

ADR-003 — Stratégie d’erreurs & versionnement d’API

Statut	Accepté
Contexte	CanTelcoX expose plusieurs API REST à travers une API Gateway et plusieurs services internes. Les clients, les tests, les logs et les dashboards doivent pouvoir interpréter les erreurs de manière cohérente. Sans format commun, chaque service pourrait retourner des erreurs différentes, ce qui compliquerait le débogage, l’observabilité et la documentation OpenAPI. Le système doit aussi pouvoir évoluer sans briser les clients existants.
Décision	Définir un format JSON d’erreur uniforme contenant **code**, **message**, **details**, **traceId** et **timestamp**. Les erreurs sont mappées vers des codes HTTP standards, notamment **400, 401, 403, 404, 405, 409, 422, 423, 500** et **503**. Les API exposées sont versionnées avec le préfixe **/v1**.
Conséquences	Cette décision améliore la cohérence entre les services, facilite les tests automatisés, la documentation OpenAPI, la corrélation avec les logs et l’analyse dans Prometheus/Grafana. Le **traceId** permet de suivre une requête de bout en bout. En contrepartie, tous les services doivent respecter la même convention d’erreur et maintenir la compatibilité des routes **/v1**. Les changements incompatibles devront être introduits dans une nouvelle version, par exemple **/v2**.
Exigence du cahier tracée	API REST, contrat OpenAPI central, gestion d’erreurs JSON normalisée, observabilité, traçabilité des requêtes et politique de versionnement **/v1**.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

10. Exigences de qualité

Objectif Arc42 — *Arbre de qualité et scénarios de qualité concrets ; cibles NFR, stratégie de validation (tests) et résultats mesurés. Couvre la vue Scénarios de 4+1.*

Repères de l'énoncé

Tâches de l'énoncé : 5.2 tests de charge ; 5.3 load balancing ; 5.4 caching ; 6.2 direct vs Gateway ; 7.1 stratégie de tests.

Vue 4+1 : Scénarios.

Livrables attendus :

- Arbre de qualité + scénarios de qualité mesurables.
- Cibles NFR et stratégie de tests (pyramide unit → intégration → E2E).
- Plans de campagnes de charge (UC-03/04/05/08) et de comparaison (N=1..4, cache, Gateway).

Critères d'acceptation (à cocher)

☑ Paliers NFR cibles atteints (ou écarts argumentés) : P95 ≤ 500 ms, ≥ 600 ops/s, dispo 95 %.

☑ Couverture ≥ 80 % sur le domaine critique ; ≥ 1 scénario E2E via la Gateway.

10.1 Arbre de qualité

Attribut de qualité	Priorité	Justification
Performance	Haute	Les opérations critiques comme la commande, l'activation et le paiement doivent répondre dans un délai acceptable pour une démonstration BSS.
Capacité	Haute	Le système doit supporter plusieurs utilisateurs simulés via Locust et exposer ses limites de charge.
Disponibilité	Moyenne	Le système doit rester stable sous charge nominale, mais la tolérance complète aux pannes multi-instances reste limitée en Phase 1.
Sécurité	Haute	Les opérations sensibles utilisent authentification, MFA, jeton Bearer, validation d'entrée et erreurs normalisées.
Auditabilité	Haute	Les actions critiques doivent être traçables avec **TraceId** et journalisées dans un audit log append-only.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Attribut de qualité	Priorité	Justification
Maintenabilité	Moyenne	Les services suivent une séparation domaine/application/adapters afin de faciliter l'évolution des cas d'utilisation.
Reproductibilité	Haute	Le correcteur doit pouvoir lancer le système avec Docker Compose et rejouer les tests/Postman.

10.2 Scénarios de qualité

Attribut	Stimulus	Réponse attendue	mesure / preuve
UC-05 — Consultation des offres	Un abonné consulte les forfaits actifs via **/v1/offers.**	Le catalogue répond via l'API Gateway avec une liste d'offres actives.	Postman, Swagger, logs Gateway, Locust.
UC-05 — Création de commande	Un abonné crée une commande avec **Idempotency-Key**.	La commande est créée une seule fois, rejouable sans doublon.	Postman, base MySQL, audit events.
UC-03 — Activation de ligne	Une activation est demandée pour un client et une offre valides.	La ligne reçoit un MSISDN, une SIM/eSIM, un statut clair et un audit.	Postman, MySQL, free5GC/adapter HLR, audit.
UC-06 — Paiement de facture	Un abonné paie une facture ouverte.	Le paiement est accepté via passerelle simulée, la facture passe à PAID.	Postman, MySQL, audit, tests.
UC-02 — Authentification MFA	L'abonné se connecte puis valide le code MFA.	Une session de démonstrationest créée après MFA réussi.	Postman, logs, tests automatisés.
Traçabilité audit	Une requête porte un **TraceId**.	Le même TraceId apparaît dans les logs multi-services et les événements d'audit.	Docker logs, endpoint audit.
Reproductibilité Docker	Le correcteur lance **docker compose up --build.**	Les services, bases, Gateway, Swagger, Prometheus et Grafana démarrent.	**docker compose ps,** healthchecks.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

10.3 Cibles NFR & résultats

Indicateur	Cible	Mesuré	Verdict
Latence P95	≤ 500 ms	**260 ms** sur GET /v1/offers via Gateway optimisé; **300 ms** au palier 3 du scénario mixte; saturation au palier 4 avec **730 ms**	Atteint sous charge nominale / optimisée ; saturation identifiée au palier 4
Débit	≥ 600 ops/s	**668.77 ops/s** sur GET /v1/offers via Gateway optimisé ; environ **347 ops/s** sur le scénario mixte complet	Atteint sur le scénario catalogue optimisé ; partiel sur le scénario mixte complet
Disponibilité	95 %	**99.9976 %** pendant le test de failover; 1 échec sur 41 509 requêtes	Atteint

Synthèse ; le détail des mesures et les captures figurent aux §10.5 à §10.8.

La section 10.3 constitue la synthèse finale du chapitre 10 et a été complétée après les mesures détaillées des sections 10.5 à 10.8. La baseline de 10.5 identifie le seuil de saturation avant optimisation. Les sections 10.6 et 10.7 montrent ensuite les effets du load balancing et du caching. Le débit cible de 600 ops/s est atteint sur le scénario optimisé de consultation du catalogue via Gateway, tandis que le scénario mixte complet reste inférieur à cette cible en raison des écritures, de l'idempotence, de l'audit et des appels interservices. Cet écart est donc documenté et argumenté.

10.4 Stratégie de tests & couverture

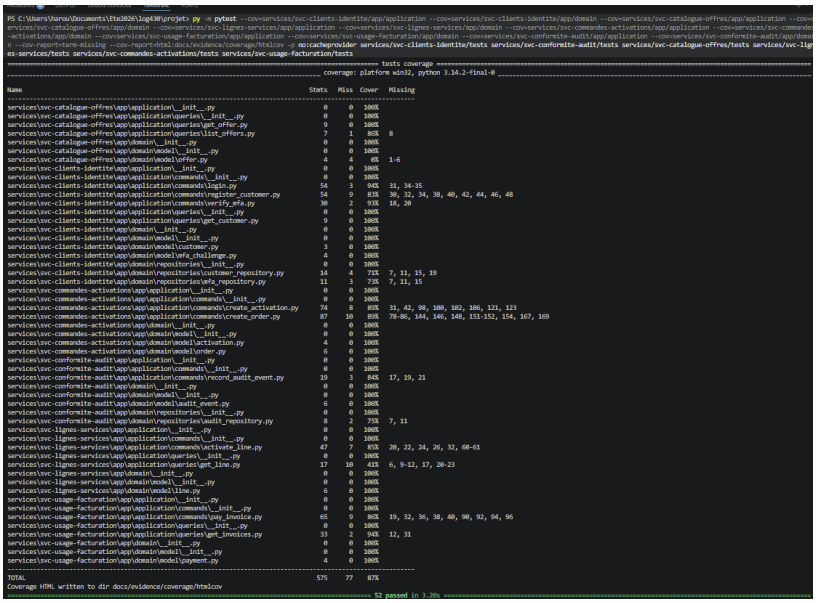
La stratégie de tests suit une pyramide en trois niveaux : tests unitaires, tests d'intégration et tests bout en bout. L'objectif est de valider les règles métier critiques au niveau le plus bas possible, puis de confirmer l'intégration technique avec les bases de données, les adaptateurs et l'API Gateway.

1. Tests unitaires

Les tests unitaires ciblent principalement les couches Application et Domain, qui représentent le domaine critique du projet. Ces couches contiennent les règles métier liées à l'inscription, la vérification d'identité, l'authentification MFA, la validation d'offre, l'activation de ligne, l'idempotence des commandes et activations, le paiement de facture, **l'exactly-once** pratique et l'audit **append-only.**

La couverture est mesurée uniquement sur les couches Application et Domain. Les adaptateurs d'infrastructure ne sont pas inclus dans cette mesure principale, car ils sont validés par les tests d'intégration.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)



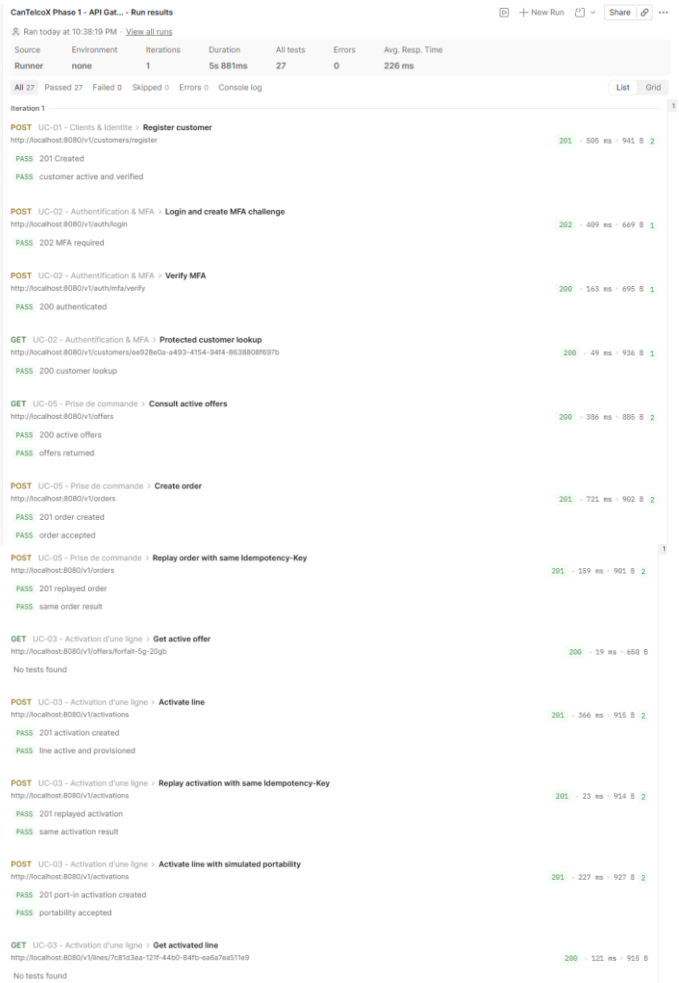
```
PS C:\Users\harou\Documents\Ete2026\log430\projet> py -m pytest --cov=services/svc-clients-identite/app/application --cov=services/svc-clients-identite/app/domain --cov=services/svc-catalogue-offres/app/application --cov=services/svc-catalogue-offres/app/domain --cov=services/svc-lignes-services/app/application --cov=services/svc-lignes-services/app/domain --cov=services/svc-commandes-activations/app/application --cov=services/svc-commandes-activations/app/domain --cov=services/svc-usage-facturation/app/application --cov=services/svc-usage-facturation/app/domain --cov=services/svc-conformite-audit/app/application --cov=services/svc-conformite-audit/app/domain --cov-report=term-missing --cov-report=html:docs/evidence/coverage/htmlcov -p no:cacheprovider services/svc-clients-identite/tests services/svc-conformite-audit/tests services/svc-catalogue-offres/tests services/svc-lignes-services/tests services/svc-commandes-activations/tests services/svc-usage-facturation/tests
============================= tests coverage ==============================
_____ coverage: platform win32, python 3.14.2-final-0 _____

Name                                                
```

MSISDN, l'exactly-once pratique des paiements, l'audit append-only et la passerelle de paiement simulée. Le test ignoré correspond à l'intégration live free5GC, dépendante du simulateur. persistance sont validés dans l'environnement de test.

3. Tests bout en bout

Le scénario E2E passe par l'API Gateway uniquement (`http://localhost:8080`). Il couvre le parcours critique télécom: inscription → authentification → MFA → consultation des offres → prise de commande → activation de ligne → paiement de facture → audit par TraceId. Ce scénario est démontré avec la collection Postman officielle et complété par un test E2E automatisé pytest.



© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

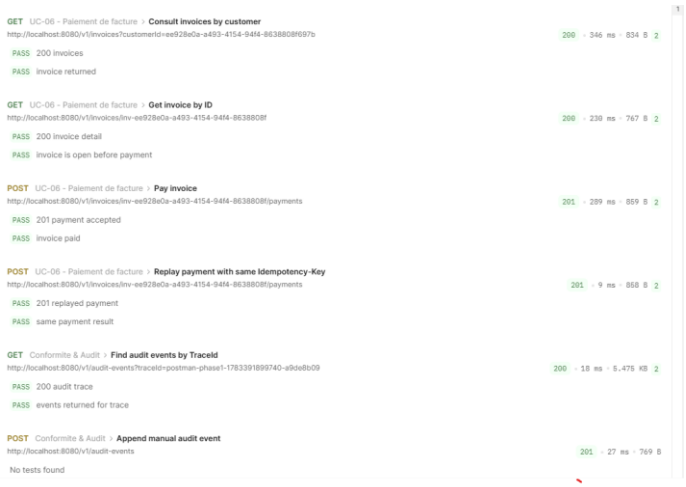


Figure 10.4-3 — Exécution du scénario E2E via Postman

La capture montre l'exécution de la collection Postman **CanTelcoX Phase 1 - API Gateway**. Les requêtes passent par l'API Gateway et valident les étapes principales du parcours : inscription client, création du challenge MFA, vérification MFA, accès protégé, consultation des offres et création de commande. Le résultat indique **27 tests réussis**, **0 échec** et **0 erreur**, ce qui confirme le bon fonctionnement du scénario E2E via Gateway.

10.5 Résultats des campagnes de charge

La section 10.5 présente la campagne de référence, réalisée avant optimisation. Elle sert à établir le comportement initial du système, à observer les 4 Golden Signals et à identifier le seuil de saturation. Les résultats optimisés sont présentés ensuite aux sections 10.6 et 10.7, puis synthétisés en 10.3. Conformément à la consigne, les campagnes se concentrent sur les parcours UC-03 et UC-05, avec UC-02 utilisé pour préparer l'authentification MFA nécessaire aux appels protégés. Les UC-04 et UC-08 sont conservés dans le tableau de suivi, mais ils ne sont pas exécutés car ils ne sont pas implémentés dans la version Phase 1.

Les campagnes de charge ont été exécutées avec Locust via l'API Gateway. Pour chaque scénario, la charge a été augmentée progressivement par paliers afin d'identifier le seuil de saturation.

Les paliers utilisés sont les suivants :

- Palier 1 : 25 utilisateurs, spawn rate 5/s — charge faible
- Palier 2 : 50 utilisateurs, spawn rate 10/s — charge nominale
- Palier 3 : 100 utilisateurs, spawn rate 20/s — charge élevée
- Palier 4 : 200 utilisateurs, spawn rate 20/s — stress

Le seuil de saturation est défini comme le premier palier où la latence P95 dépasse **500 ms**, où le taux d'erreurs augmente de façon significative, ou où des timeouts apparaissent. Les résultats détaillés Locust et les captures Grafana sont joints sous le tableau.

Scénario de charge	Charge nominale	Seuil de saturation	P95
UC-04 — Consultation usage	Non exécuté — UC-04 non implémenté dans la version Phase 1	Non mesuré	Non mesuré

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

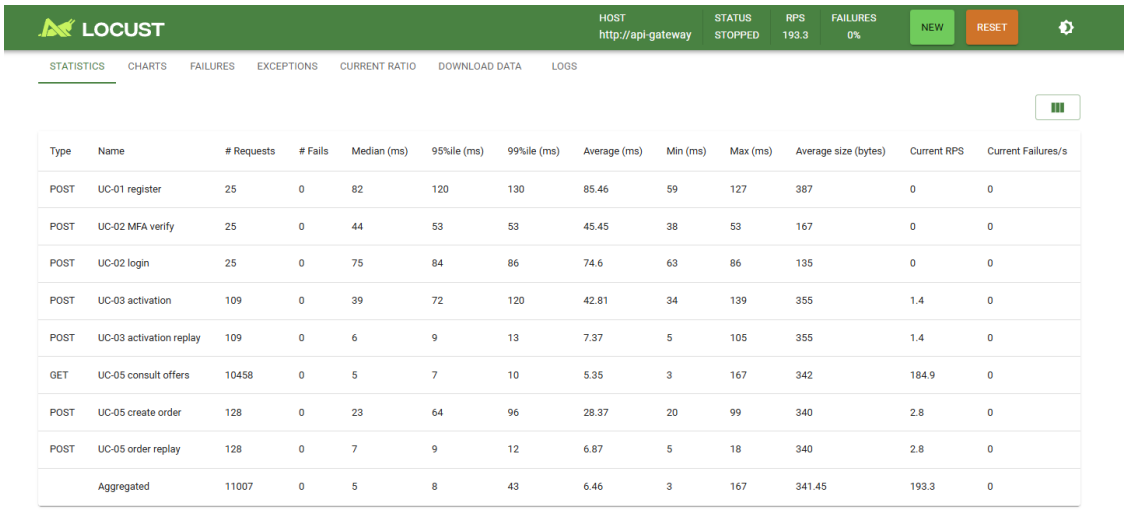
Scénario de charge	Charge nominale	Seuil de saturation	P95
UC-05 — Prise de commande	Palier 3 — 100 utilisateurs, spawn rate 20/s; 0% échec	Palier 4 — 200 utilisateurs, P95 > 500 ms	930 ms
UC-03 — Activation	Palier 3 — 100 utilisateurs, spawn rate 20/s; 0% échec	Palier 4 — 200 utilisateurs, P95 > 500 ms	810 ms
UC-02 — Authentification MFA (support aux scénarios protégés)	Palier 2 — 50 utilisateurs, spawn rate 10/s; 0% échec	Palier 3 — 100 utilisateurs; login P95 > 500 ms	Login 520 ms / MFA 450 ms
UC-08 — Facturation	Non exécuté — UC-08 non implémenté dans la version Phase 1	Non mesuré	Non mesuré

La figure présente le dashboard Grafana CanTelcoX Golden Signals utilisé pendant les campagnes de charge. Il couvre les quatre signaux requis : trafic **RPS**, taux d'erreurs **4xx/5xx**, latence **P95/P99** et saturation **CPU/RAM** par service. Les résultats Locust donnent les mesures par endpoint, tandis que Grafana permet de vérifier l'état interne des services pendant la montée en charge.



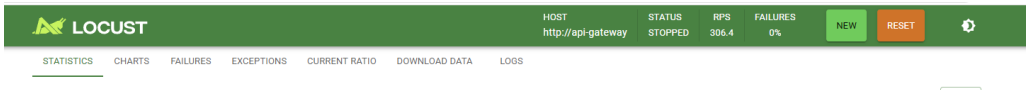
Figure 10.5-1 — Dashboard Grafana CanTelcoX Golden Signals pendant la campagne de charge.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)



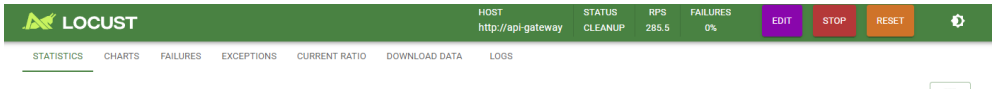
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	UC-01 register	25	0	82	120	130	85.46	59	127	387	0	0
POST	UC-02 MFA verify	25	0	44	53	53	45.45	38	53	167	0	0
POST	UC-02 login	25	0	75	84	86	74.6	63	86	135	0	0
POST	UC-03 activation	109	0	39	72	120	42.81	34	139	355	1.4	0
POST	UC-03 activation replay	109	0	6	9	13	7.37	5	105	355	1.4	0
GET	UC-05 consult offers	10458	0	5	7	10	5.35	3	167	342	184.9	0
POST	UC-05 create order	128	0	23	64	96	28.37	20	99	340	2.8	0
POST	UC-05 order replay	128	0	7	9	12	6.87	5	18	340	2.8	0
	Aggregated	11007	0	5	8	43	6.46	3	167	341.45	193.3	0

Figure 10.5-2 — Résultats Locust du palier 1



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	UC-01 register	50	0	90	160	180	100.49	68	176	387	0	0
POST	UC-02 MFA verify	50	0	63	140	140	76.78	32	140	167	0	0
POST	UC-02 login	50	0	110	160	180	111.94	61	184	135	0	0
POST	UC-03 activation	252	0	77	250	330	96.06	35	413	355	3.9	0
POST	UC-03 activation replay	252	0	7	66	150	15.75	5	208	355	3.9	0
GET	UC-05 consult offers	19105	0	6	81	160	14.75	3	258	342	290.2	0
POST	UC-05 create order	258	0	44	180	240	62.72	20	265	340	4.2	0
POST	UC-05 order replay	257	0	8	78	150	16.27	5	181	340	4.2	0
	Aggregated	20274	0	6	90	170	17.01	3	413	341.44	306.4	0

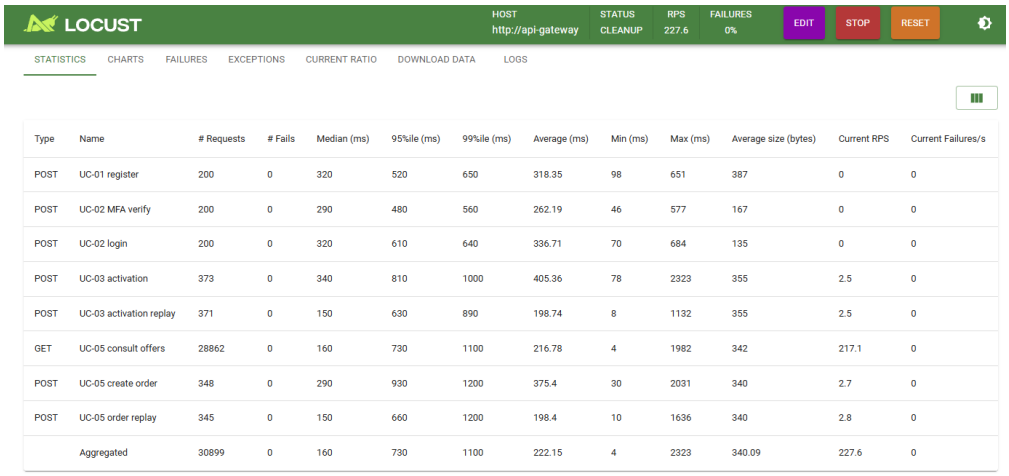
Figure 10.5-3— Résultats Locust du palier 2



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	UC-01 register	100	0	280	540	580	286.54	102	584	387	0	0
POST	UC-02 MFA verify	100	0	240	450	520	236.25	55	518	167	0	0
POST	UC-02 login	100	0	370	520	590	342.99	98	592	135	0	0
POST	UC-03 activation	333	0	140	430	670	187.6	56	831	355	2.4	0
POST	UC-03 activation replay	333	0	21	250	320	54.88	6	463	355	2.4	0
GET	UC-05 consult offers	28570	0	19	270	320	58.17	4	533	342	271.8	0
POST	UC-05 create order	363	0	88	420	630	150.57	34	824	340	4.5	0
POST	UC-05 order replay	363	0	22	270	320	64.7	6	448	340	4.4	0
	Aggregated	30262	0	19	270	350	63.03	4	831	341.12	285.5	0

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Figure 10.5-4— Résultats Locust du palier 3



Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	UC-01 register	200	0	320	520	650	318.35	98	651	387	0	0
POST	UC-02 MFA verify	200	0	290	480	560	262.19	46	577	167	0	0
POST	UC-02 login	200	0	320	610	640	336.71	70	684	135	0	0
POST	UC-03 activation	373	0	340	810	1000	405.36	78	2323	355	2.5	0
POST	UC-03 activation replay	371	0	150	630	890	198.74	8	1132	355	2.5	0
GET	UC-05 consult offers	28862	0	160	730	1100	216.78	4	1982	342	217.1	0
POST	UC-05 create order	348	0	290	930	1200	375.4	30	2031	340	2.7	0
POST	UC-05 order replay	345	0	150	660	1200	198.4	10	1636	340	2.8	0
	Aggregated	30899	0	160	730	1100	222.15	4	2323	340.09	227.6	0

Figure 10.5-5 — Résultats Locust du palier 4

10.6 Load balancing (N = 1..4) & tolérance aux pannes

Cette section constitue la première optimisation après la baseline de 10.5. Elle évalue l'effet de la mise à l'échelle horizontale et du load balancing NGINX sur le débit, la latence, les erreurs et la saturation.

Les campagnes de load balancing ont été exécutées avec Locust via l'API Gateway. L'objectif était de comparer le comportement du système avec 1, 2, 3 et 4 instances des services répliqués, puis de vérifier la tolérance aux pannes par l'arrêt manuel d'une instance pendant la charge.

N instances	RPS	P95 (ms)	Erreurs	Saturation
1	345.44	85	0	Non observée: P95 < 500 ms, aucun échec
2	342.42	93	0	Non observée: P95 < 500 ms, aucun échec
3	342.39	170	0	Non observée: P95 < 500 ms, aucun échec
4	341.86	78	0	Non observée: P95 < 500 ms, aucun échec

Les résultats obtenus pour N = 1 à N = 4 sont très proches. Le débit reste stable autour de **342 à 345 RPS**, avec **0 % d'erreur** pour toutes les campagnes. La latence P95 demeure également inférieure à la cible NFR de **500 ms** dans tous les cas. Ces résultats montrent que la charge

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

nominale utilisée ne sature pas l'instance unique. Par conséquent, l'ajout d'instances ne produit pas de gain de débit mesurable dans cette campagne. Le load balancing est donc validé principalement comme mécanisme de répartition, de déploiement multi-instance et de tolérance aux pannes, mais le gain de scalabilité horizontale reste limité à cette charge.

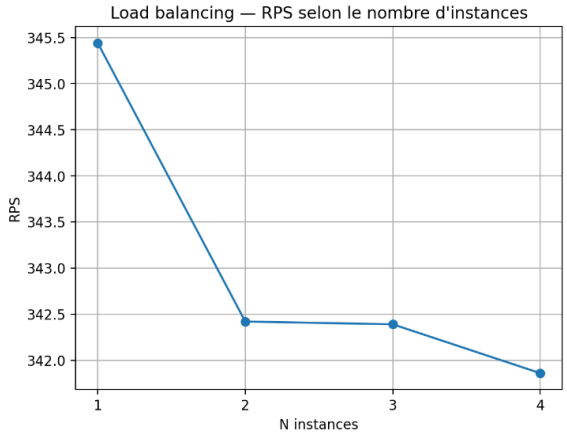


Figure 10.6-1 — Évolution du débit RPS selon le nombre d'instances

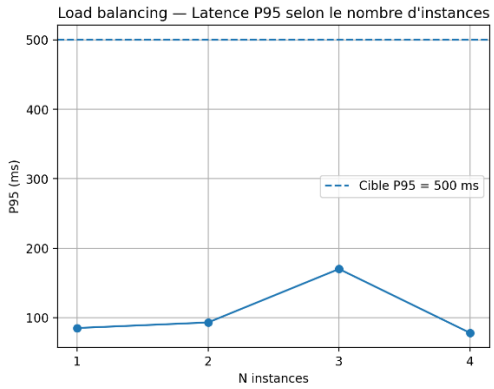


Figure 10.6-2 — Évolution de la latence P95 selon le nombre d'instances

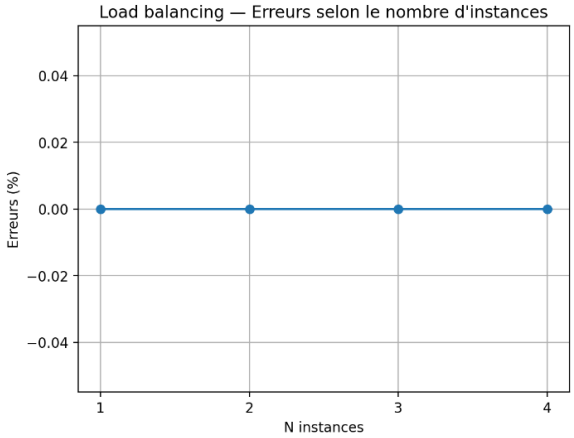


Figure 10.6-3 — Évolution du taux d'erreurs selon le nombre d'instances

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)



Figure 10.6-4 — Saturation observée selon le nombre d'instances

10.6.2 Tolérance aux pannes : arrêt d'une instance en charge

Un test de tolérance aux pannes a été réalisé afin de vérifier le comportement du système lorsqu'une instance applicative est arrêtée pendant une campagne de charge. Pour ce test, le service **svc-commandes-activations** était déployé avec 2 instances actives derrière le load balancer.

Pendant la campagne Locust, l'instance **projet-svc-commandes-activations-2** a été arrêtée manuellement avec la commande `docker stop`. Le trafic continuait de passer par l'API Gateway et le load balancer devait rediriger les requêtes vers l'instance restante du service **svc-commandes-activations.**

Le test a généré 41 509 requêtes au total. Un seul échec a été observé, soit une réponse **HTTP 502** sur POST UC-03 activation, cohérente avec une requête en cours au moment exact où l'instance a été arrêtée. La disponibilité observée est donc: **Disponibilité = (41 509 − 1) / 41 509 × 100 = 99,9976 %**

Le résultat dépasse largement la cible de disponibilité de 95 %. Le système conserve donc une disponibilité acceptable malgré la perte d'une instance sur deux pendant la charge.

La capture montre les conteneurs applicatifs disponibles avant le test de failover. Le service svc-commandes-activations est déployé avec **2** instances actives derrière l'API Gateway / load balancer NGINX.

```
PS C:\Users\harou\Documents\Ete2026\log430\projet> docker compose ps svc-catalogue-offres svc-commandes-activations svc-lignes-services
NAME                                  IMAGE                                 COMMAND                  SERVICE                       CREATED          STATUS                    PORTS
projet-svc-catalogue-offres-1         projet-svc-catalogue-offres           "sh -c 'gunicorn -b …"   svc-catalogue-offres          5 hours ago      Up 5 hours (healthy)
projet-svc-catalogue-offres-2         projet-svc-catalogue-offres           "sh -c 'gunicorn -b …"   svc-catalogue-offres          29 minutes ago   Up 28 minutes (healthy)
projet-svc-catalogue-offres-3         projet-svc-catalogue-offres           "sh -c 'gunicorn -b …"   svc-catalogue-offres          21 minutes ago   Up 21 minutes (healthy)
projet-svc-catalogue-offres-4         projet-svc-catalogue-offres           "sh -c 'gunicorn -b …"   svc-catalogue-offres          14 minutes ago   Up 14 minutes (healthy)
projet-svc-commandes-activations-1    projet-svc-commandes-activations      "sh -c 'gunicorn -b …"   svc-commandes-activations     5 hours ago      Up 5 hours (healthy)
projet-svc-commandes-activations-2    projet-svc-commandes-activations      "sh -c 'gunicorn -b …"   svc-commandes-activations     29 minutes ago   Up 28 minutes (healthy)
projet-svc-commandes-activations-3    projet-svc-commandes-activations      "sh -c 'gunicorn -b …"   svc-commandes-activations     21 minutes ago   Up 21 minutes (healthy)
projet-svc-commandes-activations-4    projet-svc-commandes-activations      "sh -c 'gunicorn -b …"   svc-commandes-activations     14 minutes ago   Up 14 minutes (healthy)
projet-svc-lignes-services-1          projet-svc-lignes-services            "sh -c 'gunicorn -b …"   svc-lignes-services           5 hours ago      Up 5 hours (healthy)
projet-svc-lignes-services-2          projet-svc-lignes-services            "sh -c 'gunicorn -b …"   svc-lignes-services           29 minutes ago   Up 28 minutes (healthy)
projet-svc-lignes-services-3          projet-svc-lignes-services            "sh -c 'gunicorn -b …"   svc-lignes-services           21 minutes ago   Up 21 minutes (healthy)
projet-svc-lignes-services-4          projet-svc-lignes-services            "sh -c 'gunicorn -b …"   svc-lignes-services           14 minutes ago   Up 14 minutes (healthy)
PS C:\Users\harou\Documents\Ete2026\log430\projet>
```

Figure 10.6-5 — Instances disponibles avant le test de tolérance aux pannes

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

La capture montre l'identification des instances du service svc-commandes-activations, puis l'arrêt manuel de l'instance projet-svc-commandes-activations-2 avec la commande docker stop. Cette opération est effectuée pendant que Locust génère du trafic afin de simuler la panne d'une instance. Après l'arrêt, le load balancer doit rediriger les requêtes vers l'instance restante.

```
PS C:\Users\harou\Documents\Ete2026\log430\projet> docker ps --filter "name=svc-commandes-activations" --format "{{.Names}}"
projet-svc-commandes-activations-4
projet-svc-commandes-activations-3
projet-svc-commandes-activations-2
projet-svc-commandes-activations-1
PS C:\Users\harou\Documents\Ete2026\log430\projet> docker stop projet-svc-commandes-activations-2
projet-svc-commandes-activations-2
PS C:\Users\harou\Documents\Ete2026\log430\projet>
```

Figure 10.6-6 — Arrêt manuel d'une instance pendant la charge

La capture Locust présente les résultats de la campagne de charge pendant le test de failover. Le système a traité 41 509 requêtes avec un seul échec, correspondant à une requête **POST UC-03 activation**. Le débit agrégé atteint 346,55 RPS et le taux d'échec reste quasi nul. La disponibilité calculée est de 99,9976 %, ce qui confirme que le système respecte la cible de 95 % malgré l'arrêt d'une instance.

Locust Test Report

During: 2026-07-06, 6:00:41 p.m. - 2026-07-06, 6:02:41 p.m.
Target Host: http://api-gateway
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
POST	UC-01 register	50	0	130.7	85	220	387	0.42	0
POST	UC-02 MFA verify	50	0	72.3	50	104	167	0.42	0
POST	UC-02 login	50	0	121.94	85	246	135	0.42	0
POST	UC-03 activation	480	1	116.09	19	572	354.59	4.01	0.01
POST	UC-03 activation replay	479	0	16	5	225	355	4	0
GET	UC-05 consult offers	39360	0	14.37	3	380	342	328.6	0
POST	UC-05 create order	520	0	70.53	27	573	340	4.34	0
POST	UC-05 order replay	520	0	14.06	5	169	340	4.34	0
	Aggregated	41509	1	16.61	3	573	341.84	346.55	0.01

Figure 10.6-7 — Résultat Locust après arrêt d'une instance

10.7 Caching (on / off)

Cette section constitue la seconde optimisation après la baseline: le cache Redis/L1 est utilisé pour réduire les lectures répétées sur le catalogue et améliorer la latence des endpoints de consultation.

Le cache **Redis/L1** a été appliqué à l'endpoint de consultation du catalogue des offres, soit **GET /v1/offers**. L'objectif était de réduire les lectures répétées vers la base MySQL et de stabiliser la latence pour un endpoint fortement sollicité pendant les campagnes Locust.

Les résultats montrent une amélioration mesurable: le P95 de **GET /v1/offers** passe de **81 ms sans cache** à **71 ms avec cache**, soit un gain d'environ **12,3 %**. Le P95 agrégé passe également de **89 ms** à **84 ms**, soit un gain d'environ **5,6 %**. Le gain reste modéré, car le catalogue de démonstration contient peu d'offres et la base MySQL répond déjà rapidement. L'intérêt principal du cache est

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

donc de réduire la charge de lecture répétée sur la base de données et de limiter la dégradation de latence sous charge.

Le scénario UC-04 — Consultation d'usage n'étant pas implémenté dans la version Phase 1, aucun test de cache n'a été exécuté sur l'usage.

Endpoint	P95 sans cache	P95 avec cache	Charge DB	Gain
Catalogue /v1/offers	81 ms	71 ms	Réduite après MISS initial; HIT Redis/L1 pendant TTL	12.3%
Usage	Hors périmètre UC-04	Hors périmètre UC-04	Non mesuré	N/A

La capture montre la campagne Locust avec cache activé. Le système traite 41 526 requêtes, avec 0 échec, un débit agrégé de 347,18 RPS et un P95 agrégé de 84 ms. Pour GET /v1/offers, le débit atteint 328,92 RPS avec un P95 de 71 ms.

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
POST	UC-01 register	130	140	150	220	540	540	560	560
POST	UC-02 MFA verify	81	86	94	100	150	170	200	200
POST	UC-02 login	120	120	130	150	160	170	180	180
POST	UC-03 activation	88	96	110	140	190	270	410	1400
POST	UC-03 activation replay	8	9	11	14	35	78	140	240
GET	UC-05 consult offers	6	7	8	11	26	71	130	580
POST	UC-05 create order	52	58	68	89	120	160	270	630
POST	UC-05 order replay	8	9	11	14	28	58	120	170
	Aggregated	6	7	9	13	41	84	150	1400

Figure 10.7-1 — Résultat Locust avec cache activé sur le catalogue

La capture présente la campagne Locust exécutée avec le cache désactivé pour l'endpoint GET /v1/offers. Ce scénario sert de référence avant optimisation. Le P95 mesuré pour la consultation du catalogue est de 81 ms, avec un P95 agrégé de 89 ms

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
POST	UC-01 register	140	150	180	220	270	270	440	440
POST	UC-02 MFA verify	87	92	98	120	140	160	190	190
POST	UC-02 login	130	130	150	200	370	370	370	370
POST	UC-03 activation	80	86	95	110	150	200	290	330
POST	UC-03 activation replay	7	8	9	12	37	74	150	170
GET	UC-05 consult offers	5	6	7	10	42	81	160	300
POST	UC-05 create order	45	51	57	70	120	170	240	310
POST	UC-05 order replay	7	8	9	12	35	77	150	230
	Aggregated	6	6	8	12	54	89	170	440

Figure 10.7-2 — Résultat Locust avec cache désactivé sur le catalogue

Après redémarrage **de svc-catalogue-offres** et suppression de la clé Redis **catalogue:offers:active**, le premier appel à **GET /v1/offers** retourne **X-Cache: MISS.** Le second appel identique retourne **X-Cache: HIT**. Cela confirme que la consultation du catalogue est servie depuis le cache **Redis/L1** après le premier accès, pendant la durée du TTL configuré.

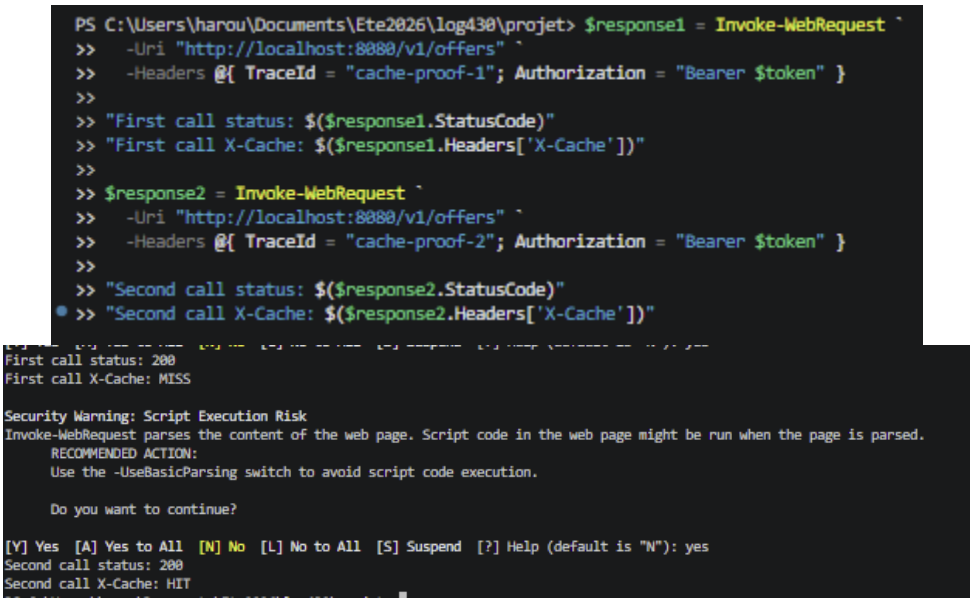


Figure 10.7-3 — Vérification du cache HIT sur GET /v1/offers

Le principal risque du cache est de servir des données périmées lorsque le catalogue est modifié. Pour limiter ce risque, le cache utilise un TTL et peut être invalidé lors des mises à jour du catalogue. En Phase 1, l'invalidation est simple : suppression de la clé Redis **catalogue:offers:active** au redémarrage ou lors d'une modification catalogue.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

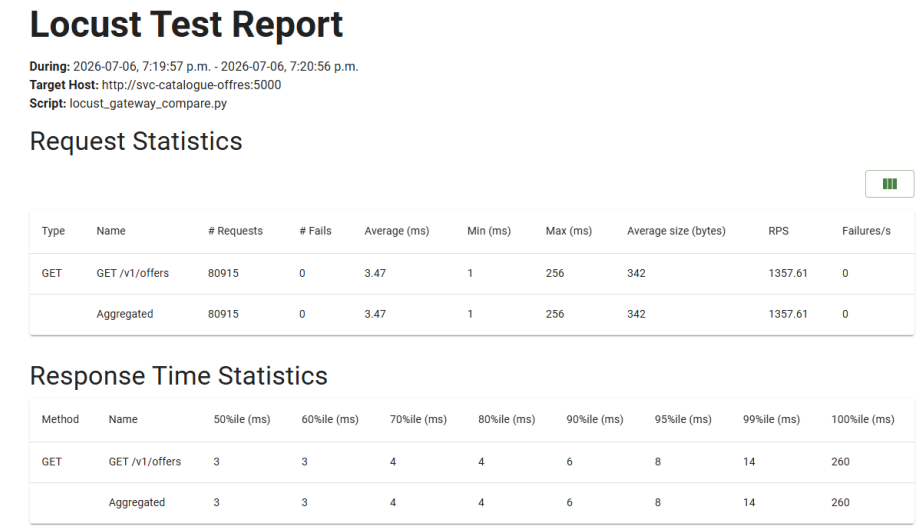
10.8 Appels directs vs via Gateway

Trajet	P95 (ms)	Erreurs	Traçabilité
Direct	8 ms	0 %	**TraceId** envoyé directement à **svc-catalogue-offres:5000**. Pas de point d'entrée central et pas de logs Gateway.
Via Gateway	260 ms	0 %	**TraceId** propagé via NGINX. Point d'entrée unique, logs Gateway, conservation des headers **Authorization / TraceId.**

La comparaison utilise le même endpoint GET /v1/offers afin d'isoler l'impact de l'API Gateway. L'appel direct vers **svc-catalogue-offres:5000** atteint 1357 RPS avec un P95 de 8 ms et 0 % d'échec. L'appel via NGINX Gateway optimisé atteint 668,77 RPS avec un P95 de 260 ms et 0 % d'échec.

L'appel direct est donc plus rapide, ce qui est normal puisqu'il contourne le routage Gateway et les traitements transverses. Cependant, l'appel via Gateway reste sous la cible NFR de **500 ms** au P95 pour ce scénario. La Gateway apporte aussi les mécanismes requis pour la démonstration officielle: point d'entrée unique, routage vers les microservices, propagation des en-têtes transverses, journalisation centralisée et load balancing.

La capture montre le test Locust exécuté directement contre http://svc-catalogue-offres:5000 sur l'endpoint GET /v1/offers. Ce trajet contourne l'API Gateway. Le test atteint 1357 RPS, avec un P95 de 8 ms et 0 % d'échec.



Locust Test Report

During: 2026-07-06, 7:19:57 p.m. - 2026-07-06, 7:20:56 p.m.
Target Host: http://svc-catalogue-offres:5000
Script: locust_gateway_compare.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /v1/offers	80915	0	3.47	1	256	342	1357.61	0
	Aggregated	80915	0	3.47	1	256	342	1357.61	0

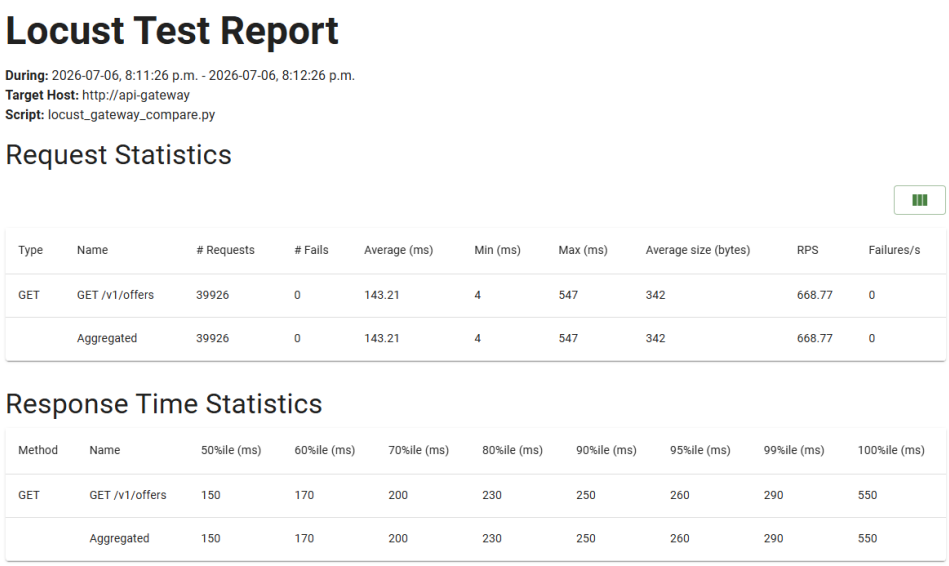
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /v1/offers	3	3	4	4	6	8	14	260
	Aggregated	3	3	4	4	6	8	14	260

Figure 10.8-1 — Test Locust en appel direct vers svc-catalogue-offres

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

La capture montre le test Locust exécuté via http://api-gateway sur le même endpoint GET /v1/offers. Ce trajet passe par NGINX, qui agit comme point d'entrée unique, route la requête vers le service catalogue et propage les en-têtes transverses. Le test atteint 668,77 RPS, avec un P95 de 260 ms et 0 % d'échec.



Locust Test Report

During: 2026-07-06, 8:11:26 p.m. - 2026-07-06, 8:12:26 p.m.
Target Host: http://api-gateway
Script: locust_gateway_compare.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
GET	GET /v1/offers	39926	0	143.21	4	547	342	668.77	0
	Aggregated	39926	0	143.21	4	547	342	668.77	0

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /v1/offers	150	170	200	230	250	260	290	550
	Aggregated	150	170	200	230	250	260	290	550

Figure 10.8-2 — Test Locust via NGINX Gateway

Validation du jeton et MFA simulée

NGINX open source est utilisé comme API Gateway pour le routage, la journalisation, le load balancing et la propagation des en-têtes TraceId, Authorization et Idempotency-Key. Il ne valide pas directement les jetons, car NGINX open source ne supporte pas nativement la validation OAuth2/OIDC/JWT.

Dans cette Phase 1, la validation du jeton est déléguée aux services applicatifs. Le cas d'utilisation **UC-02 — Authentification MFA** utilise un fournisseur MFA simulé: le service d'identité crée un challenge OTP, valide le code MFA, puis émet un jeton de session de développement de type Bearer **dev-session** Ce jeton est ensuite transmis dans le header Authorization.

Ce choix permet de démontrer le flux d'authentification MFA et la protection des endpoints sans introduire un fournisseur OAuth2/OIDC complet dans le périmètre de la Phase 1. Une amélioration future consisterait à déplacer la validation du jeton au niveau de la Gateway avec NGINX njs, NGINX Plus ou une Gateway spécialisée compatible OAuth2/OIDC.

Justification du choix de NGINX

NGINX a été retenu comme API Gateway pour sa simplicité, sa compatibilité avec Docker Compose et sa capacité à fournir rapidement un point d'entrée unique vers les microservices. Il permet de centraliser le routage **/v1/...,** de conserver les en-têtes transverses, de produire des logs d'accès et de répartir la charge entre plusieurs instances. Le compromis assumé est que la validation OAuth2/OIDC complète n'est pas réalisée par NGINX dans cette phase; elle est déléguée aux services applicatifs.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

11. Risques et dette technique

Objectif Arc42 — *Identifier les risques d'architecture connus et la dette technique, avec leurs mesures d'atténuation.*

Risque / dette	Impact	Probabilité	Atténuation
Point unique de défaillance de l'API Gateway NGINX	Si la Gateway tombe, les APIs publiques /v1 deviennent indisponibles même si les services internes fonctionnent.	Moyenne	Healthcheck **/health**, configuration NGINX simple et versionnée, redémarrage Docker Compose. En Phase 2: plusieurs instances Gateway derrière un load balancer externe.
MySQL unique dans Docker Compose	Perte de disponibilité si le conteneur MySQL tombe ; risque de perte de données si les volumes ne sont pas sauvegardés.	Moyenne	Volume Docker persistant, scripts SQL reproductibles, tests d'intégration. En Phase 2: sauvegardes automatisées, réplication ou service DB managé.
Données périmées dans le cache Redis catalogue	Les offres retournées par GET /v1/offers peuvent rester périmées pendant la durée du TTL.	Faible	TTL court via **OFFER_CACHE_TTL_SECONDS**. Catalogue principalement read-only en Phase 1. En Phase 2: invalidation explicite lors des mises à jour catalogue.
Références inter-services non garanties par FK physiques	Les références comme **customer_id**, **offer_id** ou **line_id** peuvent devenir incohérentes si les services évoluent sans coordination.	Moyenne	Validation applicative par appels REST avant commandes critiques, tests E2E, contrats OpenAPI. En Phase 2: tests de contrats et événements d'intégrité.
Idempotence basée sur clés uniques locales	Protège contre les doubles soumissions dans un service, mais ne couvre pas tous les scénarios distribués complexes.	Moyenne	Contraintes uniques sur **idempotency_key**, validation du **request_hash**, tests d'intégration. En Phase 2: patron Outbox, saga ou orchestration robuste.
Exactly-once facturation limité à des garde-fous pratiques	Risque résiduel si un paiement externe réel réussit mais que la persistance locale échoue.	Moyenne	Transaction DB locale, **idempotency_key**, unicité du paiement par facture, passerelle de paiement simulée. En Phase 2: webhooks PSP idempotents et journal transactionnel.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Risque / dette	Impact	Probabilité	Atténuation
Audit append-only dépendant de MySQL	L'append-only est protégé par triggers, mais reste dans la même infrastructure MySQL de démonstration.	Faible à moyenne	Triggers **no_update** / **no_delete,** tests d'intégration append-only, logs structurés. En Phase 2: stockage WORM, export immuable ou infrastructure audit séparée.
Sécurité simplifiée pour la démonstration	Tokens, MFA et secrets sont adaptés à la Phase 1, mais pas à une production réelle.	Moyenne	**.env.example** sans secrets réels, **.env** ignoré, MFA simulée, endpoints protégés par Bearer token. En Phase 2: OIDC, TLS, rotation des secrets et vault.
Tests de charge dépendants de la machine locale	Les résultats Locust peuvent varier selon CPU/RAM, Docker Desktop et la machine utilisée.	Moyenne	Rapports Locust conservés comme preuves, scénarios documentés et reproductibles. En Phase 2 : environnement de benchmark fixe.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

12. Glossaire

Objectif Arc42 — *Définir le langage ubiquitaire (DDD) et les acronymes pour une compréhension commune.*

Repères de l'énoncé

Tâches de l'énoncé : 1.2 ubiquitous language.

Livrables attendus :

- Glossaire métier (langage ubiquitaire) + acronymes.

Critères d'acceptation (à cocher)

☐ Glossaire validé.

12.1 Langage métier

Terme	Définition	Bounded context
Abonné	Client individuel ou PME utilisant les services mobiles CanTelcoX.	Clients & Identité
Client	Profil commercial et d'identité associé à une personne ou organisation.	Clients & Identité
Identité vérifiée	État indiquant que les informations d'identité du client ont passé les contrôles de validation Phase 1.	Clients & Identité
Risque client	Niveau de risque simplifié utilisé pour bloquer ou accepter certaines opérations sensibles.	Clients & Identité
MFA Challenge	Défi d'authentification créé après login et validé par OTP.	Clients & Identité
Session token	Jeton retourné après MFA et utilisé comme Bearer token sur les endpoints protégés.	Clients & Identité
Forfait	Offre mobile commercialisable, avec prix mensuel, volume de données et statut actif/inactif.	Catalogue & Offres
Offre active	Offre disponible pour commande ou activation.	Catalogue & Offres
Catalogue	Ensemble des forfaits et options publiés par l'opérateur.	Catalogue & Offres
Ligne mobile	Service télécom associé à un client, un MSISDN, une SIM/eSIM et un forfait.	Lignes & Services
MSISDN	Numéro public de ligne mobile, unique dans le système.	Lignes & Services
SIM	Carte SIM physique associée à une ligne mobile.	Lignes & Services

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

eSIM	Profil SIM numérique associé à une ligne mobile.	Lignes & Services
IMSI	Identifiant réseau de l'abonné mobile utilisé pour le provisionnement.	Lignes & Services
SUPI	Identifiant permanent du souscripteur dans le contexte 5G/free5GC.	Lignes & Services
Provisionnement réseau	Action de créer ou activer la ligne dans le simulateur réseau HLR/HSS/UDM.	Lignes & Services
Activation	Commande idempotente qui valide le client et l'offre, crée/provisionne la ligne et produit un état actif ou échoué.	Commandes & Activations
Commande	Demande commerciale de souscription ou changement d'offre. En Phase 1, NEW_PLAN est implémenté.	Commandes & Activations
Idempotency-Key	Clé fournie par le client API pour rejouer une commande sans créer de doublon.	Commandes & Activations / Usage & Facturation
Rejeu idempotent	Réponse retournée depuis une commande déjà traitée avec la même clé et le même contenu.	Commandes & Activations / Usage & Facturation
Facture	Document de facturation associé à un client, une période et un montant dû.	Usage & Facturation
Paiement	Commande idempotente visant à régler une facture.	Usage & Facturation
Paiement accepté	Paiement validé par la passerelle simulée et persisté localement.	Usage & Facturation
Exactly-once pratique	Ensemble de garde-fous locaux évitant les doubles écritures critiques, notamment contraintes uniques et transactions.	Usage & Facturation
Événement d'audit	Enregistrement append-only décrivant une action métier ou technique traçable.	Conformité & Audit
TraceId	Identifiant de corrélation utilisé pour suivre une requête entre gateway, services et audit.	Transverse / Observabilité
Journal append-only	Journal où les événements peuvent être ajoutés mais pas modifiés ni supprimés.	Conformité & Audit
Gateway API	Point d'entrée public NGINX exposant les routes /v1.	API Gateway / Transverse
Healthcheck	Endpoint ou commande permettant de vérifier l'état opérationnel d'un service.	Observabilité

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Golden Signals	Latence, trafic, erreurs et saturation utilisées pour observer le comportement du système.	Observabilité
Seed data	Données initiales chargées automatiquement au démarrage de la base, par exemple les forfaits de démonstration.	Persistance / Catalogue

12.2 Acronymes

Acronyme	Signification
BSS	Business Support System
CRTC	Conseil de la radiodiffusion et des télécommunications canadiennes
HLR/HSS	Home Location Register / Home Subscriber Server
MSISDN	Mobile Station International Subscriber Directory Number
MFA / OTP	Authentification multifacteur / mot de passe à usage unique
NFR	Exigence non fonctionnelle
ADR	Architecture Decision Record
TMF	TM Forum (Open APIs)
API	Application Programming Interface
CI	Continuous Integration
CD	Continuous Deployment / Continuous Delivery
DAO	Data Access Object
DB	Database
DDD	Domain-Driven Design
ER	Entity-Relationship
UDM	Unified Data Management
IMSI	International Mobile Subscriber Identity
PII	Personally Identifiable Information
PIPEDA / LPRPDE	Personal Information Protection and Electronic Documents Act / Loi sur la protection des renseignements personnels et les documents électroniques
Law 25 / Loi 25	Loi québécoise modernisant la protection des renseignements personnels
REST	Representational State Transfer
SLA	Service Level Agreement
SME / PME	Small and Medium Enterprise / Petite et moyenne entreprise
SIM	Subscriber Identity Module
eSIM	Embedded SIM
SUPI	Subscription Permanent Identifier
UC	Use Case
UML	Unified Modeling Language
VM	Virtual Machine
WORM	Write Once Read Many

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Annexe A — Contrats d’API & catalogue d’endpoints

Référez le fichier OpenAPI/Swagger publié et la collection Postman (chemins dans le dépôt).

Méthode + Route (/v1)	UC	Service	Auth / MFA
POST /v1/customers/register	UC-01 — Inscription & vérification d’identité	svc-clients-identite	Public, TraceId; MFA non requise
POST /v1/auth/login	UC-02 — Authentification & MFA	svc-clients-identite	Public; crée un challenge MFA
POST /v1/auth/mfa/verify	UC-02 — Authentification & MFA	svc-clients-identite	Vérifie le code MFA; retourne un sessionToken
GET /v1/customers/{customerId}	Support UC-03 / UC-05 / UC-06 — validation client	svc-clients-identite	Bearer token obtenu après MFA
GET /v1/offers	Support UC-05 — consultation des offres disponibles	svc-catalogue-offres	Bearer token obtenu après MFA
GET /v1/offers/{offerId}	*Support UC-03 / UC-05 — validation de l’offre*	svc-catalogue-offres	*Bearer token obtenu après MFA*
POST /v1/orders	*UC-05 — Prise de commande / souscription à un forfait*	*svc-commandes-activations*	*Bearer token + Idempotency-Key*
POST /v1/activations	*UC-03 — Activation d’une ligne mobile*	*svc-commandes-activations*	*Bearer token + Idempotency-Key*
GET /v1/lines/{lineId}	*Support UC-03 — consultation de l’état de ligne*	*svc-lignes-services*	*Bearer token obtenu après MFA*
GET /v1/invoices?customerId={customerId}	*Support UC-04 / UC-06 — consultation des factures*	*svc-usage-facturation*	*Bearer token obtenu après MFA*

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Méthode + Route (/v1)	UC	Service	Auth / MFA
GET /v1/invoices/{invoiceId}	*Support UC-04 / UC-06 — détail de facture*	*svc-usage-facturation*	*Bearer token obtenu après MFA*
POST /v1/invoices/{invoiceId}/payments	*UC-06 — Paiement de facture mobile*	*svc-usage-facturation*	*Bearer token + Idempotency-Key*
GET /v1/audit-events?traceId={traceId}	*Support conformité & audit — traçabilité réglementaire*	*svc-conformite-audit*	*Bearer token obtenu après MFA*
POST /v1/audit-events	*Support conformité & audit — journal append-only*	*svc-conformite-audit*	*Bearer token obtenu après MFA*

Swagger/OpenAPI = docs/api/openapi.yaml

Swagger UI local = http://localhost:8081

Postman = docs/api/postman/CanTelcoX-Phase1-Gateway.postman_collection.json

Base URL gateway = http://localhost:8080

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Annexe B — Schéma de persistance

La persistance est organisée par service et par contexte métier. Chaque service possède son propre schéma logique et accède uniquement aux tables de son bounded context. En Phase 1, les bases sont initialisées automatiquement par les scripts SQL versionnés sous **infra/mysql/init**, ce qui permet de recréer l'environnement de manière reproductible avec Docker Compose.

Les références entre services sont **logiques** et non modélisées par des clés étrangères inter-bases. Par exemple, une commande peut contenir un **customer_id** ou un **offer_id**, mais l'intégrité entre services est validée par appels applicatifs plutôt que par contrainte SQL inter-schémas. Ce choix respecte la séparation des responsabilités entre microservices.

Modèle logique par service

Service	Base / schéma	Tables principales	Rôle
svc-clients-identite	clients_identite	customers, mfa_challenges	Profils clients, identité, statut, MFA
svc-conformite-audit	conformite_audit	audit_events	Journal append-only pour conformité et preuves
svc-catalogue-offres	catalogue_offres	offers	Catalogue des forfaits et offres
svc-lignes-services	lignes_services	service_lines	Lignes mobiles, MSISDN, SIM/eSIM, IMSI, état HLR
svc-commandes-activations	commandes_activations	orders, activations	Commandes UC-05, activations UC-03, idempotence
svc-usage-facturation	usage_facturation	invoices, payments	Factures, paiements, exactly-once pratique

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Ce diagramme regroupe les schémas MySQL détenus par chaque service et montre les références logiques entre contextes bornés. Les liens inter-services ne sont pas des clés étrangères physiques; ils sont validés par API REST et adapters.

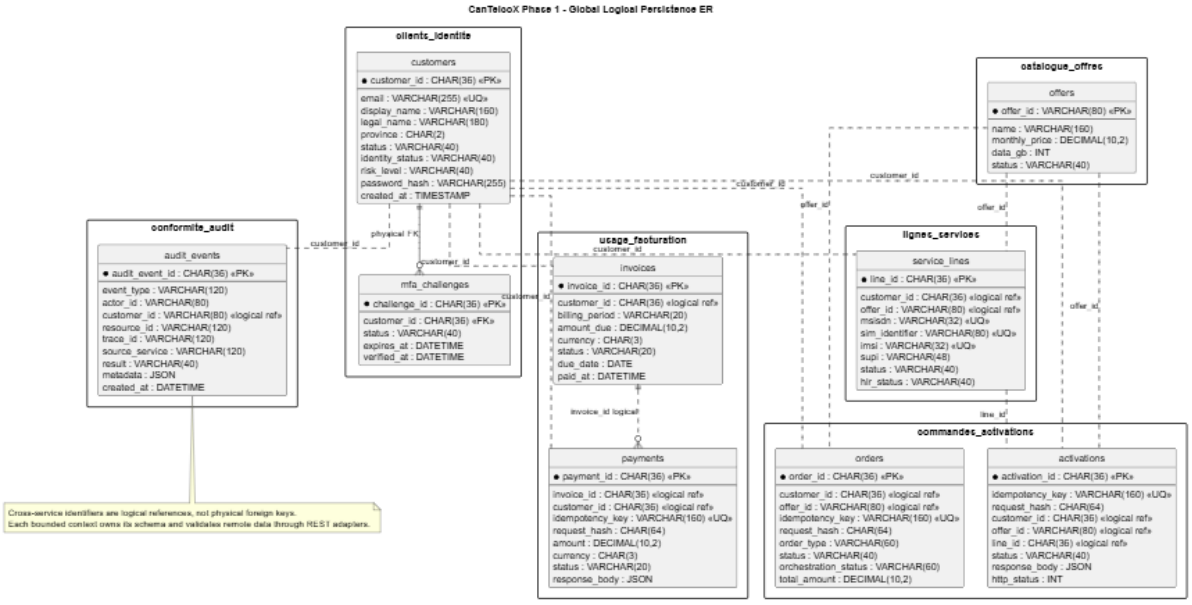


Figure B-1 — Schéma ER logique global de la persistance CanTelcoX Phase 1.

Ce diagramme présente les tables `customers` et `mfa_challenges`, incluant l'unicité de l'adresse courriel, la relation physique MFA-client, et les champs nécessaires à l'identité, au risque de connexion et au cycle MFA.

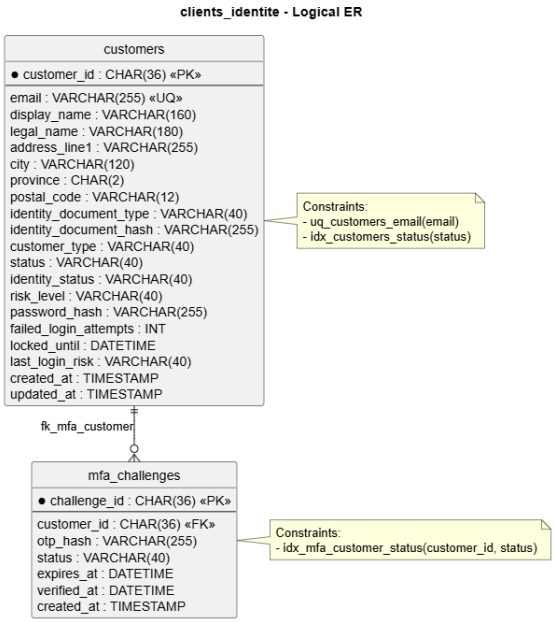


Figure B-2 — Schéma ER du service Clients & Identité.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Ce diagramme présente la table `audit_events`, les index de recherche par client, trace et type d'événement, ainsi que les déclencheurs SQL empêchant toute modification ou suppression afin de garantir un journal append-only.

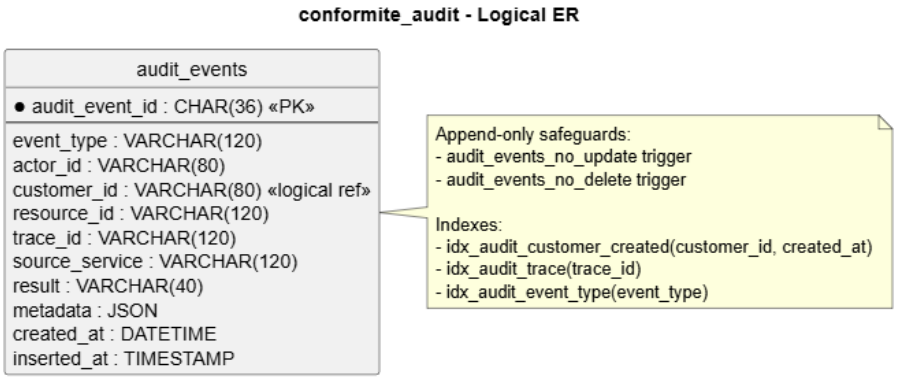


Figure B-3 — Schéma ER du service Conformité & Audit.

Ce diagramme présente la table `offers`, utilisée pour publier les forfaits actifs et inactifs. Les données seed incluent les offres de démonstration utilisées par les parcours UC-03 et UC-05

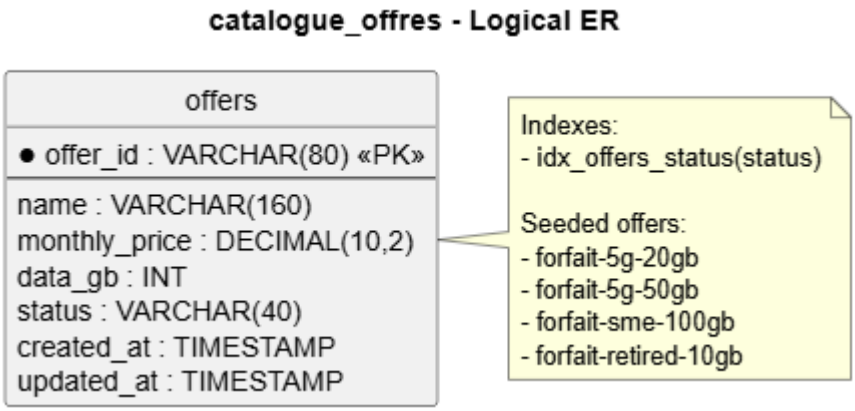


Figure B-4 — Schéma ER du service Catalogue & Offres.

Ce diagramme présente la table `service_lines`, responsable de l'état des lignes mobiles, MSISDN, SIM/eSIM, IMSI/SUPI et statut de provisionnement HLR. Les contraintes d'unicité protègent contre les doublons de numéro, SIM et IMSI.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

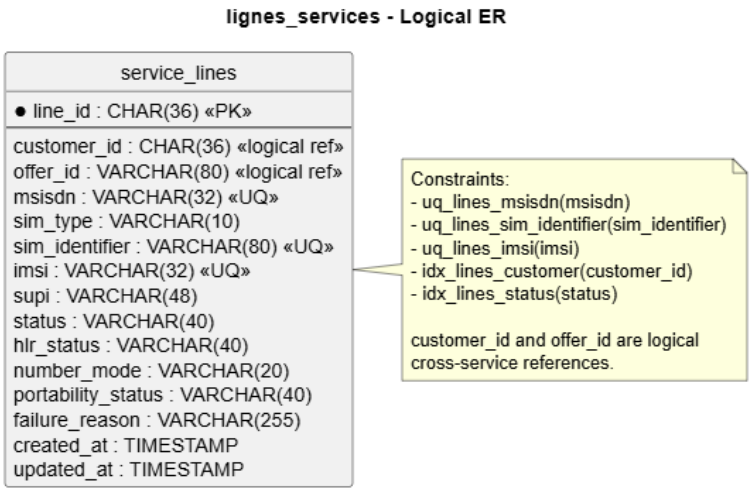


Figure B-5 — Schéma ER du service Lignes & Services.

Ce diagramme présente les tables `orders` et `activations`, utilisées pour UC-05 et UC-03. Les clés d'idempotence uniques empêchent les doubles créations lors des rejouements de commandes ou d'activations.

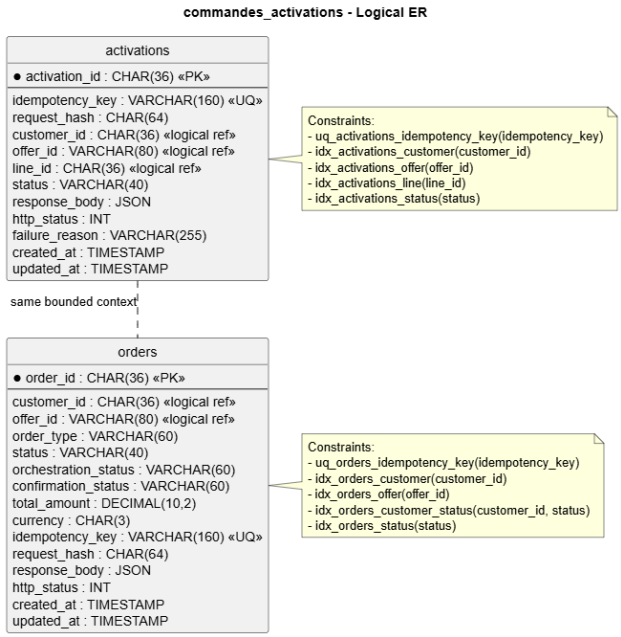


Figure B-6 — Schéma ER du service Commandes & Activations.

Ce diagramme présente les tables `invoices` et `payments`, incluant les contraintes d'idempotence des paiements et l'unicité pratique d'un paiement réussi par facture afin de limiter les écritures en double.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

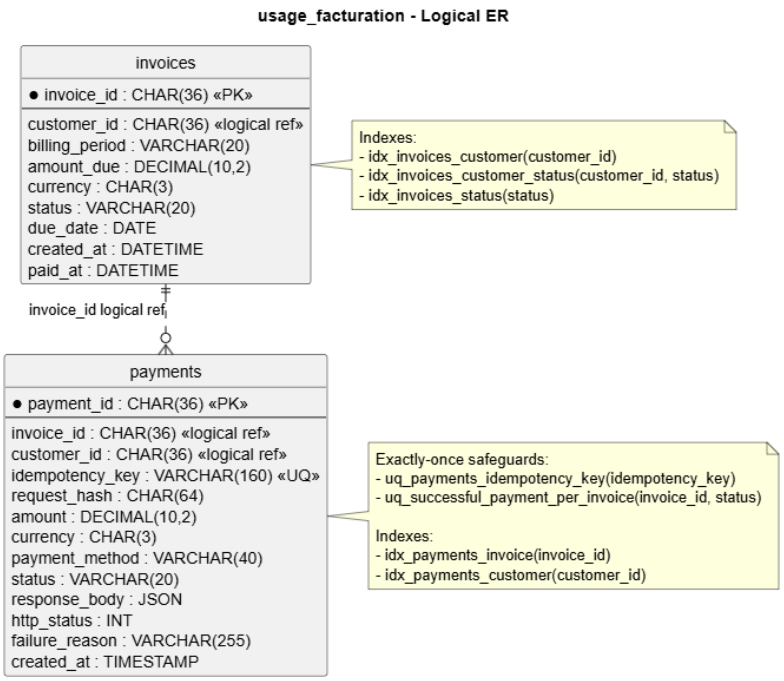


Figure B-7 — Schéma ER du service Usage & Facturation.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Élément	Détail
Contraintes d'intégrité	Les principales contraintes sont: **customers.email** unique; **mfa_challenges.customer_id** lié au cclient;**service_lines.msisdn**, **sim_identifier** et **imsi** uniques; activations.idempotency_key unique; **orders.idempotency_key** unique; **payments.idempotency_key** unique; contrainte sur **payments (invoice_id, status)** pour éviter deux paiements réussis sur la même facture. Des index sont aussi prévus sur les champs de recherche fréquents comme **customer_id**, **status** et **trace_id**.
Migrations	Aucun outil ORM/migration lourd n'est utilisé en Phase 1. Les migrations sont reproductibles par scripts SQL versionnés sous **infra/mysql/init/*.sql**. Ces scripts sont exécutés automatiquement par le conteneur MySQL au premier démarrage du volume. Le bootstrap Docker Compose crée également les bases/schémas nécessaires par service
Choix ORM/DAO	Aucun ORM n'est utilisé. L'accès aux données se fait par DAO/repositories MySQL avec PyMySQL, dans les adaptateurs sortants de persistance. Les principaux repositories sont: **MysqlCustomerRepository, MysqlMfaRepository, MysqlOfferRepository, MysqlLineRepository, MysqlActivationRepository, MysqlUsageBillingRepository et MysqlAuditRepository**
Données seed	Les offres sont seedées dans **infra/mysql/init/030-svc-catalogue-offres.sql**, notamment **forfait-5g-20gb**, **forfait-5g-50gb**, **forfait-sme-100gb** et **forfait-retired-10gb**. Les clients de démonstration sont créés via UC-01. Les factures de démonstration UC-06 sont créées de manière déterministe par le service de facturation lors de la consultation des factures pour un client valide.
Tests d'intégration	Les tests d'intégration utilisent la base MySQL conteneurisée et sont regroupés dans **tests/integration/test_mysql_repository_boundaries.py**. Ils valident la persistance clients/MFA, la lecture des offres seedées, l'unicité MSISDN/SIM/IMSI, l'idempotence des commandes et activations, l'exactly-once pratique des paiements et le caractère append-only de l'audit. Commande: **python -m pytest -p no:cacheprovider tests/integration** après **docker compose up -d.**

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Annexe C — CI/CD, conteneurisation & exploitation

Élément	Détail
Dockerfiles	Des Dockerfiles multi-stage sont fournis pour les 6 services sous **services/svc-*/Dockerfile.** Chaque image utilise un stage **builder** pour installer les dépendances Python, puis un stage **runtime** slim contenant le code applicatif et Gunicorn.
docker-compose.yml	Le fichier **docker-compose.yml** lance toute la stack Phase 1: les 6 services Flask, MySQL 8.4, les scripts d'initialisation/seed sous **infra/mysql/init**, Redis pour le cache, NGINX comme API Gateway, Swagger UI, Prometheus, Grafana et Locust via le profil optionnel **load-test**. Commande de démarrage : **cp .env.example .env**, puis **docker compose up -d --build**.
Healthchecks	Chaque service Flask expose un endpoint **/health**, vérifié par Docker Compose. MySQL et Redis possèdent aussi leurs healthchecks. L'API Gateway expose également **/health** et possède un healthcheck Compose sur **http://127.0.0.1/health**.
Pipeline CI	La CI est basée sur GitHub Actions avec un runner **ubuntu-latest,** via le workflow **.github/workflows/phase1-ci.yml**. Les étapes sont: installation de Python 3.12, lint avec **compileall** et **ruff**, validation des fichiers Postman JSON, validation **docker compose config**, tests unitaires, build Docker, démarrage Compose, tests d'intégration, test E2E via Gateway et publication des artefacts JUnit/logs/config. Le timeout est fixé à 10 minutes. Le badge CI est inclus dans **README.md**.
Job CD (VM)	Le déploiement cible un environnement Docker propre, équivalent à une VM fraîche. Le script principal est **scripts/deploy-vm.sh deploy**; un équivalent Windows local est fourni avec **scripts/deploy-vm.ps1**. Le script crée **.env** s'il est absent, valide la configuration Compose, construit les images, lance **docker compose up -d** et attend les healthchecks. Un rollback simple est disponible avec **scripts/deploy-vm.sh rollback**, qui revient à la révision Git précédente enregistrée dans **.deploy/previous_revision**. Un workflow manuel optionnel est fourni dans **.github/workflows/deploy-vm.yml**.
Runbook & guide de démo	Le runbook **docs/runbooks/deploiement-vm.md** documente les prérequis VM/Docker, le démarrage reproductible, les commandes Windows/Linux, le déploiement en une commande, le rollback, la consultation du statut et des logs, le reset des volumes, la CI, le CD manuel et les preuves à capturer. La reproductibilité visée est de moins de 30 minutes sur un environnement Docker propre.

Les figures suivantes fournissent la preuve d'exécution du pipeline CI GitHub Actions. Elles complètent la description du workflow .**github/workflows/phase1-ci.yml** en montrant une exécution réussie, la durée du pipeline, l'artefact produit et le détail des étapes automatisées.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

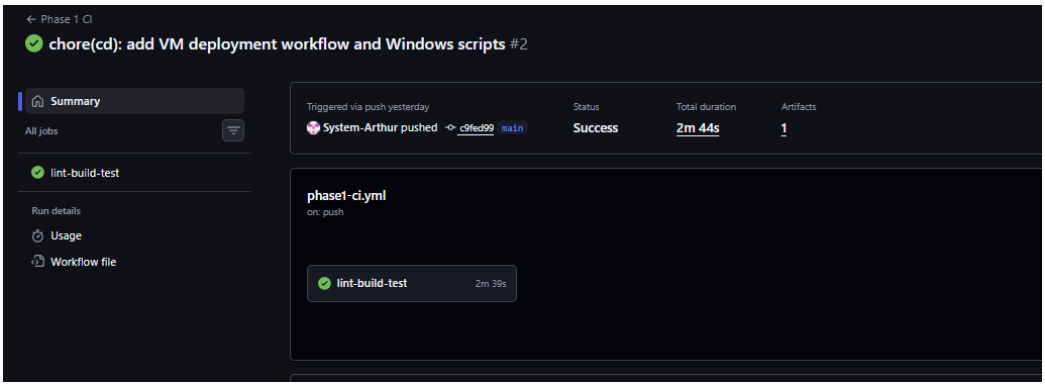


Figure C.1 — Exécution réussie du workflow GitHub Actions Phase 1 CI

La capture montre l'exécution du workflow GitHub Actions **Phase 1 CI** sur la branche **main**. Le workflow se termine avec le statut **Success**, une durée totale de 2 min 44 s et un artefact publié. Cette preuve confirme que le pipeline CI est exécutable automatiquement à partir du dépôt et qu'il termine correctement dans la limite de 10 minutes fixée pour la Phase 1.

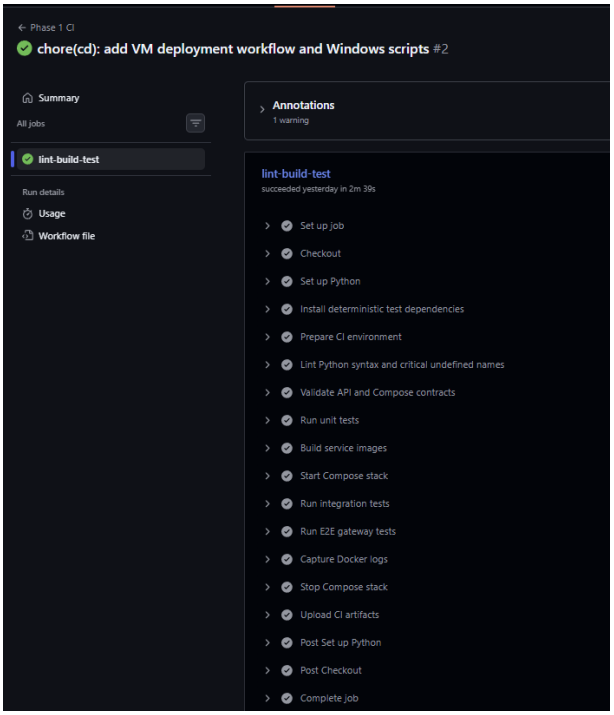


Figure C.2 — Détail des étapes du job CI `lint-build-test

La capture détaille les étapes exécutées par le job **lint-build-test.** Le pipeline effectue le checkout du dépôt, installe Python et les dépendances de test, prépare l'environnement CI, valide la syntaxe Python, vérifie les contrats API/Postman et Docker Compose, exécute les tests unitaires, construit les images Docker, démarre la stack Compose, lance les tests d'intégration, exécute les tests E2E via l'API Gateway, capture les logs Docker, arrête la stack puis publie les artefacts CI. Toutes les étapes sont réussies, ce qui valide la chaîne CI complète pour la Phase 1.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Annexe D — Traçabilité, Définition de Fini & auto-évaluation

D.1 Matrice de traçabilité (livrables & critères d'acceptation)

Cochez chaque case (☐) lorsque le livrable est produit et le critère d'acceptation satisfait ; indiquez la preuve (figure, section ou chemin dans le dépôt). « Empl. » renvoie à la section du présent document.

Tâche	Livrables → empl.	Critères d'acceptation → empl.	Preuve	Fait
1) Analyse métier & DDD				
1.1 UC Must	§1.1, §1.2	§1.2 — ≥ 5 UC validés	*§1.1–§1.2, §6.1, Postman collection docs/api/postman/CanTelcoX-Phase1-Gateway.postman_collection.json, tests/e2e*	☑ Livrable ☑ CA
1.2 DDD	§3.1, §3.3, §5.3, §12.1	§12.1 ; §5.3 ; §3.3+§4.1	*§3.3, §5.3, docs/architecture/diagrams/context-map-ddd-cantelcox.puml*	☑ Livrable ☑CA
2) Architecture & décisions				
2.1 Services	§4.1, §4.2, §5.2	§4.2, §5.1 ; §4.1	*§4.1–§4.2, §5.1, docs/architecture/diagrams/building-blocks-level1-cantelcox.puml, services/*	☑ Livrable ☑ CA
2.2 API REST	§4.3, §4.4, Annexe A	Annexe A ; §5.2/§5.4	*§4.3, docs/api/openapi.yaml, Postman collection, infra/nginx/nginx.conf*	☑ Livrable ☑ CA
2.3 4+1 / Arc42	§5, §6, §7, §1.2/§10 ; §1→§12	figures : §3.1, §5.1, §5.2, §6, §7.1	*diagrams in docs/architecture/diagrams/, sections §3, §5, §6, §7, §10*	☑ Livrable ☑ CA
2.4 ADR (≥ 3)	§9 (ADR-001/002/003)	§9 — tracé au cahier	*docs/adr/ADR-001...ADR-004*	☑ Livrable ☑ CA
3) Persistance & intégrité				
3.1 Schéma	§8.1, Annexe B	Annexe B — migrations, seeds	*infra/mysql/init/, docker-compose.yml, Annexe B*	☑ Livrable ☑ CA
3.2 Intégrité	§8.2, §8.3, §8.4, Annexe B	§8.1–8.2 ; §6.3	*§8.2–§8.4, §6.3, tests/integration/test_mysql_repository_boundaries.py, ADR-002*	☑ Livrable ☑ CA
4) Implémentation & API REST sécurisée				
4.1 Domaine	§5.3, §8.5, §8.6	§5.3 ; §10.4	*services/*/app/domain, services/*/tests, §5.3, coverage §10.4*	☑ Livrable ☑ CA
4.2 Ports/adapt.	§3.2, §5.2	§5.2/§5.4	*§5.2, docs/architecture/diagrams/building-blocks-level2-commandes-activations.puml*	☑ Livrable ☑CA
4.3 API E2E	§6.1, Annexe A	§6.1 ; §8.7	*§6.1, tests/e2e, Postman collection*	☑

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Tâche	Livrables → empl.	Critères d'acceptation → empl.	Preuve	Fait
				Livrable ☑ CA
4.4 Sécurité	§8.5	§8.5 — MFA UC-02/03	*§8.5, §6.2, sequence-uc02-auth-mfa.puml, infra/nginx/nginx.conf*	☑ Livrable ☑CA
5) Observabilité, charge & optimisation				
5.1 Observ.	§8.8, §10.5	§10.5 ; §10.3	*§8.8, §10.5, infra/prometheus/, infra/grafana/*	☑ Livrable ☑ CA
5.2 Charge	§10.5	§10.5 ; §10.3	*§10.5, tests/load/locustfile.py, docs/evidence/load/*	☑ Livrable ☑ CA
5.3 Load balancing	§7.3, §10.6	§10.6	*§7.3, §10.6, infra/nginx/nginx.conf, docs/architecture/load-balancing-cache-evidence.md*	☑ Livrable ☑ CA
5.4 Caching	§8.9, §10.7	§8.9 ; §10.7	*§8.9, §10.7, svc-catalogue-offres, Redis evidence*	☑ Livrable ☑ CA
6) API Gateway				
6.1 Gateway	§7.2	§7.2	*§7.2, infra/nginx/nginx.conf, deployment diagram*	☑ Livrable ☑ CA
6.2 Direct vs GW	§10.8	§10.8 ; §10.5	*§10.8, tests/load/locust_gateway_compare.py*	☑ Livrable ☑ CA
7) Qualité, tests & sécurité				
7.1 Tests	§10.4	§10.4 ; §6.1	*§10.4, tests/integration, tests/e2e, service unit tests*	☑ Livrable ☑ CA
7.2 Séc./erreurs	§8.5, §8.6, §8.7	§8.5/§8.6/§8.7	*§8.5–§8.7, OpenAPI error format, NGINX auth introspection*	☑ Livrable ☑ CA
8) CI/CD & conteneurisation				
8.1 Conteneurs	§7.1, Annexe C	Annexe C — compose up, /health	*docker-compose.yml, service Dockerfiles, §7.1*	☑ Livrable ☑ CA
8.2 CI	Annexe C	Annexe C — CI < 10 min	*.github/workflows/phase1-ci.yml*	☑ Livrable ☑ CA

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Tâche	Livrables → empl.	Critères d'acceptation → empl.	Preuve	Fait
8.3 CD	Annexe C	Annexe C — déploiement 1 commande	*.github/workflows/deploy-vm.yml, docs/runbooks/*	☑ Livrable ☑ CA
9) Documentation finale & démo				
9.1 Doc finale	Document entier, Annexe C	Annexe C ; Annexe D.2 (DoD)	*full report, diagrams README, runbook*	☑ Livrable ☑ CA
9.2 Comparatif	§10.3, §10.6–10.8	§10.3 ; §10.5–10.8	*§10.3, §10.5–§10.8*	☑ Livrable ☑CA

D.2 Définition de Fini (DoD)

☑ ≥ 5 UC Must implémentés bout-en-bout via l'API REST, erreurs normalisées.

☑ ≥ 2 microservices conteneurisés + DB + Prometheus + Grafana + API Gateway, déployables via docker-compose.

☑ Idempotence (commandes/activations), exactly-once (facturation), journal append-only opérationnels.

☑ Tests automatisés en CI (unit, intégration, E2E), couverture ≥ 80 % sur le domaine critique.

☑ 4 Golden Signals observés ; paliers NFR atteints ou écarts argumentés (P95 ≤ 500 ms, ≥ 600 ops/s, dispo 95 %).

☑ Campagnes de charge avec comparatifs (avant/après, N = 1..4), tolérance aux pannes démontrée.

☑ LB et cache en production (compose) avec impacts chiffrés.

☑ API Gateway opérationnelle ; direct vs gateway comparés et argumentés.

☑ 4+1 cohérent, Arc42 (sections cœur), ≥ 3 ADR approuvés ; CI/CD fonctionnels.

☑ Logs structurés ; runbook & guide de démo à jour.

☑ Rapport PDF + dépôt projet (zip) remis sur Moodle ; reproductibilité < 30 min sur VM.

D.3 Auto-évaluation (grille de l'énoncé)

Critère	Pond.	Section(s) de preuve	Auto-éval
1. Analyse métier & DDD	12 %	**§1, §3.3, §5.3, §12**	*A*
2. Architecture, décisions & API REST	18 %	**§4, §5, §9, Annexe A**	*A*
3. Persistance & intégrité	12 %	**§8.1–8.4, Annexe B**	*A*
4. Microservices & API Gateway	18 %	**§5.1, §7.2, §10.8**	*A*
5. Observabilité & tests de charge	10 %	**§8.8, §10.5**	*A*

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Critère	Pond.	Section(s) de preuve	Auto-éval
6. Load balancing & caching	10 %	**§7.3, §8.9, §10.6–10.7**	*A*
7. Qualité, tests & sécurité	10 %	**§8.5–8.7, §10.4**	*A-*
8. CI/CD, documentation & démo	10 %	**§7.1, Annexe C**	*A*

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

Annexe E — Soutenance et démonstration (20 %)

Durée : 12 à 15 minutes de présentation + questions. Objectif : démontrer que l'architecture décrite fonctionne et que les décisions sont défendables.

E.1 Déroulé attendu

Étape	Contenu	Durée
1. Contexte & objectifs de qualité	Mission CanTelcoX, cibles NFR (P95 ≤ 500 ms, ≥ 600 ops/s, dispo 95 %)	1–2 min
2. Tour d'architecture	Carte de contextes (DDD) et choix structurants	3–4 min
3. Démo en direct	Scénario de bout en bout : déclenchement via une API, propagation entre contextes	4–5 min
4. Défense d'un ADR	Pourquoi cette décision, quelles alternatives écartées	2 min
5. Questions	Échange avec le jury	3–4 min

E.2 Grille de soutenance

Critère de soutenance / démo	Pts
Maîtrise de l'architecture présentée (vocabulaire, justesse)	5
Démonstration fonctionnelle d'un scénario de bout en bout	5
Justification des décisions (au moins un ADR défendu)	4
Qualité des réponses aux questions	3
Clarté, structure et respect du temps	3
TOTAL — SOUTENANCE	**20**

E.3 Plan de soutenance (à préparer)

Scénario E2E démontré	Inscription client → authentification → MFA → consultation des offres → création d'une commande avec Idempotency-Key → activation d'une ligne → consultation/paiement d'une facture ouverte → vérification de l'audit par TraceId. Le scénario passe par l'API Gateway NGINX et utilise les routes /v1 exposées dans la collection Postman.
ADR défendu	ADR-001 — Découpage en services aligné sur les bounded contexts DDD. Décision défendue : séparer Clients & Identité, Catalogue & Offres, Commandes & Activations, Lignes & Services, Usage / Facturation et Conformité & Audit. Alternatives écartées: monolithe unique, découpage purement technique par couches, intégration directe entre domaine et systèmes externes sans adapters.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)

	ADR secondaire possible: ADR-002 — Persistance, idempotence et audit append-only. Décision défendue : utiliser des contraintes MySQL, des clés d'idempotence, des transactions locales et un journal audit append-only plutôt que Kafka/outbox/saga complets en Phase 1.
Répartition des intervenants	Intervenant 1 : contexte métier, objectifs qualité, carte DDD — 3 min. Intervenant 2 : architecture technique, Gateway, services, persistance, sécurité — 4 min. Intervenant 3 : démonstration E2E Postman/OpenAPI + audit — 5 min. Intervenant 4 : résultats NFR, load balancing, cache, ADR défendu — 3 min.
Environnement de démo	VM ou poste local avec Docker Desktop / Docker Engine. Commande de démarrage : **docker compose up -d --build**. Point d'entrée : **http://localhost:8080** via API Gateway. Contrat OpenAPI : **docs/api/openapi.yaml**. Collection Postman: **docs/api/postman/CanTelcoX-Phase1-Gateway.postman_collection.json**. Données seed: offres catalogue, bases MySQL initialisées par **infra/mysql/init,** facture de démonstration créée/accessible pour UC-06. Observabilité : Prometheus et Grafana pour les 4 Golden Signals.
Plan B (si la démo échoue)	Utiliser les captures du rapport: Postman E2E, pytest E2E, Locust, Grafana, cache **HIT/MISS** et failover. Montrer les preuves dans le dépôt **: tests/e2e, tests/integration**, **tests/load**, **docs/evidence/load, docs/architecture/diagrams**. Démonstration dégradée : lancer uniquement la collection Postman déjà validée ou montrer les réponses enregistrées avec les captures de la section 10.

© Pierre Gauthier, Fabio Petrillo — Été 2026 · École de technologie supérieure (ÉTS)